

Embedded Systems Firmware

The Complete Guide - From Beginner to Expert

Architecture · Toolchains · Bare Metal · Peripherals · Interrupts · DMA ·
RTOS · Real-Time Analysis · Drivers · Debugging · Low Power ·
Bootloaders · Security · Functional Safety · Embedded Linux · Production

Contents	38 chapters in 6 parts, plus 3 appendices
Level	First blink to shipping a certified, field-updatable product
Style	How the hardware behaves, then the code, then the numbers that prove it
Worked examples	Baud-rate error, timer prescalers, ADC resolution, stack-frame fault decoding, rate-monotonic analysis, battery life, CRC by hand - all computed and checkable
Code	C (C11) on ARM Cortex-M, with notes for RISC-V, AVR and Linux; FreeRTOS and Zephyr idioms
Assumed	Some C. No prior hardware knowledge - Chapters 2 and 3 build it.

How To Use This Book

Firmware is the software that runs directly on hardware, usually with no operating system underneath it worth the name, where a mistake does not throw an exception - it hangs a machine, drains a battery, or bricks a product in a customer's hands. This book teaches that discipline from the first instruction after reset to the last field update of a product's life.

Every chapter follows the same three beats: **what the hardware actually does**, because firmware bugs are usually misunderstandings of a peripheral rather than of C; **the code**, written the way production firmware is written rather than the way tutorials write it; and **the numbers**, because 'it should be fast enough' is not an engineering statement.

The six parts

Part	Chapters	What you get out of it
I - Foundations	1-6	What firmware is, how a microcontroller is built, how bits represent the physical world, the C that firmware really uses, the toolchain, and what happens between reset and <code>main()</code> .
II - The Hardware Interface	7-16	Registers, clocks, GPIO, interrupts, timers, analogue, UART/SPI/I2C, CAN/USB/Ethernet, DMA, and non-volatile storage - each with its failure modes.
III - Firmware Architecture	17-24	Superloops, state machines, lock-free buffers, RTOS internals, synchronisation, real-time analysis, memory discipline, and driver and HAL design that survives a second product.
IV - Making It Work	25-30	Debugging with SWD and trace, decoding a hard fault by hand, testing firmware on a host, reliability and watchdogs, low-power engineering, and performance optimisation.
V - Shipping and Trusting It	31-35	Bootloaders and OTA update, security and secure boot, functional safety and certification, embedded Linux, and manufacturing.
VI - Practice	36-38	Connected-device and edge-ML firmware, professional working practice, and a complete board-to-product walkthrough.

Reading paths

- **Complete beginner:** 1 -> 2 -> 3 -> 4 -> 5 -> 6, then 7 -> 8 -> 9 -> 10. Stop after each and make the hardware do the thing described. Reading firmware without a board is like reading about swimming.
- **Programmer moving from applications to embedded:** skim 1-3, read 4 (the C you think you know), 5-6 (toolchain and startup) carefully, then 9 (interrupts), 17-18 (architecture), 21-22 (RTOS), 25 (debugging).
- **Electronics engineer moving into firmware:** skim 2-3, read 4-6, then Part III in full - the architecture chapters are where hardware people gain the most.
- **Bringing up a new board next week:** 6 (startup), 8 (clocks), 25 (debug), and the bring-up checklist in Chapter 38.
- **Chasing a bug that happens once a day:** 9 (shared data), 19 (synchronisation), 24 (memory and stack), 25 (fault post-mortem), 28 (reliability).
- **Shipping a connected product:** 31 (bootloader and OTA), 32 (security), 35 (manufacturing), 36 (fleet architecture).
- **Interview preparation:** 4, 9, 17, 19, 20, 21, 24, 25, plus the glossary in Appendix B.

Conventions used throughout

Style	Meaning
<code>REG, 0x40021000</code>	Register names and addresses in monospace; all addresses are examples from a Cortex-M class device unless stated.
MHz, us, ns	Written as ASCII: 'us' means microseconds, 'ohm' means the unit.

Style	Meaning
Cortex-M	The default target for code examples. Differences for RISC-V, AVR, and application processors are called out where they matter.
Coloured boxes	KEY IDEA = the one thing to remember. MATH = an arithmetic derivation you can check. INTUITION = the mental picture. PRACTICAL TIP = what to actually do. PITFALL = a mistake that reaches production. EXPERT CORNER = depth you can skip on a first read.

THE ONE HABIT THAT SEPARATES GOOD FIRMWARE ENGINEERS

Measure instead of believing. A scope on a GPIO pin toggled at the top and bottom of a function answers, in ten seconds and beyond argument, a question that can otherwise consume an afternoon of reasoning about what the compiler 'probably' did. Every chapter in this book ends up recommending some version of this.

Table of Contents

PART I - Foundations	11
1. What Firmware Is	12
1.1 A first program, and everything it implies	12
1.2 The landscape: five classes of embedded system	12
1.3 Firmware, drivers, and the words people argue about	13
1.4 What makes firmware different to write	13
1.5 The shape of a firmware project	13
1.6 How to read this book with a board on your desk	14
2. The Machine: Computer Architecture for Firmware Engineers	15
2.1 The core: fetch, decode, execute	15
2.2 The programmer's model	15
2.3 Which core is in front of you	15
2.4 Memory, buses and the address map	16
2.5 Alignment, endianness, and the two bugs they cause	16
2.6 Caches, TCM and the DMA coherency trap	17
2.7 MPU, MMU and what protection you actually get	17
2.8 The core peripherals you get for free	17
3. Bits, Numbers and the Physical World	19
3.1 Bit manipulation: the six idioms	19
3.2 Integers in C, where firmware bugs live	19
3.3 Fixed point: real numbers without a floating-point unit	20
3.4 Structures, padding and data that crosses a boundary	21
3.5 Checksums and CRCs	21
3.6 From counts to physical units	22
4. The C You Actually Need: Firmware Dialect	23
4.1 volatile: what it does, and the three things it does not	23
4.2 Undefined behaviour is not theoretical	23
4.3 Storage, scope and linkage	24
4.4 The standard library, selectively	24
4.5 Patterns that make firmware readable	24
4.6 C++ in firmware: the useful subset	25
5. The Toolchain: From Source to an Image on Flash	27
5.1 The five stages	27
5.2 Sections: where everything lives	27
5.3 The linker script, annotated	27
5.4 Reading a map file	28
5.5 Flags that matter, and what they cost	28

6. From Reset to main(): The Boot Path	30
6.1 What resets a microcontroller	30
6.2 The vector table and the first two words	30
6.3 The reset handler, line by line	30
6.4 What SystemInit must do, in order	31
6.5 The RAM picture at run time	31
6.6 Handing over: bootloader to application	32

PART II - The Hardware Interface **33**

7. Registers and Memory-Mapped I/O	34
7.1 Registers are not memory	34
7.2 Ordering: the compiler, the write buffer and the bus	34
7.3 Two things to do before touching any peripheral	35
7.4 How much abstraction to buy	35
7.5 Bit-banding, shadow registers and other access patterns	35
7.6 Reading a datasheet register description	36
8. Clocks, Power Domains and Pins	38
8.1 Where clocks come from	38
8.2 The clock tree, and the PLL arithmetic	38
8.3 Power domains and what stays alive	39
8.4 Configuring a pin, completely	39
8.5 The electrical facts firmware must respect	40
9. Interrupts and Exceptions	41
9.1 The exception model	41
9.2 Priorities: the number that means the opposite	41
9.3 What the hardware does on entry and exit	42
9.4 Masking and critical sections	42
9.5 Writing a handler that will not haunt you	43
9.6 Faults: the interrupts you did not ask for	43
10. Timers, PWM and Keeping Time	45
10.1 Why the delay loop in Chapter 1 was wrong	45
10.2 Timer anatomy	45
10.3 What the modes are for	46
10.4 Software timers on top of hardware	46
10.5 Clocks, calendars and drift	46
10.6 Watchdogs	47
11. Analogue: ADC, DAC and Real Sensors	48
11.1 How a SAR ADC actually samples	48
11.2 Resolution, noise and what you really get	48
11.3 Sampling correctly	49
11.4 Getting analogue out: DAC and PWM	49

11.5 Sensors, and the maths they need	49
12. Serial Buses I: UART, SPI and I2C	51
12.1 Choosing between them	51
12.2 UART: framing and the baud-rate arithmetic	51
12.3 SPI: four modes and the details that bite	52
12.4 I2C: the bus that fails in interesting ways	52
12.5 A driver structure that works for all three	53
13. Serial Buses II: CAN, USB, Ethernet and the Rest	54
13.1 CAN and CAN-FD	54
13.2 USB, in the amount a firmware engineer needs	54
13.3 Ethernet and IP on a microcontroller	55
13.4 The rest of the bus zoo	55
14. Wireless: BLE, Wi-Fi, LoRa and Cellular	57
14.1 Two architectures, and the choice between them	57
14.2 The technologies, compared honestly	57
14.3 Design rules that apply to every radio	57
14.4 Enough RF to make good decisions	58
15. DMA: Moving Data Without the CPU	60
15.1 What DMA buys, in numbers	60
15.2 The mental model	60
15.3 The five DMA bugs	61
15.4 Circular reception: the pattern for an unknown message length	61
15.5 Arbitration, bursts and FIFOs	62
16. Non-Volatile Memory and Storage	64
16.1 How flash behaves	64
16.2 Storing configuration so it survives a power cut	64
16.3 File systems, and whether you need one	65
16.4 External flash and execute-in-place	65
PART III - Firmware Architecture	67
17. Architecture Without an Operating System	68
17.1 The superloop, and where it breaks	68
17.2 Step one: a time-triggered cooperative scheduler	68
17.3 State machines: the core tool	69
17.4 Events, queues and decoupling	69
17.5 Bare metal or RTOS?	70
18. Concurrency, Shared Data and Lock-Free Patterns	71
18.1 Where the concurrency comes from	71
18.2 What is atomic, and what only looks atomic	71
18.3 The three safe mechanisms	71
18.4 The single-producer single-consumer ring buffer	72

18.5 Publishing multi-word data without a lock	72
18.6 Reentrancy	73
18.7 Multi-core, briefly	73
18.8 Finding races before they find you	73
19. RTOS Fundamentals: Tasks, Scheduling and Synchronisation	75
19.1 Tasks and the scheduler	75
19.2 The synchronisation primitives, and what each is for	75
19.3 Priority inversion, and why mutexes are special	76
19.4 RTOS rules that prevent most defects	77
20. Real-Time Analysis: Proving the Deadlines Are Met	78
20.1 Vocabulary, precisely	78
20.2 Measuring C: what to trust	78
20.3 The quick check: utilisation bounds	78
20.4 Response time analysis, worked end to end	78
20.5 Jitter, and where it comes from	79
20.6 Designing for overload	80
21. Inside an RTOS: Context Switching and Porting	81
21.1 What a context switch actually is	81
21.2 The tick, and doing without it	81
21.3 The interrupt priority rule that breaks kernels	82
21.4 Debugging a running RTOS	82
21.5 Building a task's first stack	82
21.6 What a port actually contains	83
21.7 Measuring the kernel's own cost	83
22. Protocols and Middleware: Framing, Parsing and Reliability	85
22.1 Layer the protocol, even a small one	85
22.2 Framing: finding message boundaries in a byte stream	85
22.3 Reliability on top of framing	86
22.4 Message payloads that survive version skew	86
22.5 When to use a standard stack instead	86
23. Drivers, HALs and Portable Firmware	88
23.1 The three layers	88
23.2 API design rules	88
23.3 The vendor HAL question	88
23.4 Portability without ifdef sprawl	89
23.5 A complete driver, end to end	89
24. Memory: Budgets, Stacks, Heaps and Protection	91
24.1 Write the budget down	91
24.2 Static allocation, and pools when you need flexibility	91
24.3 Sizing stacks properly	92
24.4 Catching overflow instead of debugging corruption	92

24.5 Placing memory deliberately	93
--	----

PART IV - Making It Work 94

25. Debugging: Probes, Traces and Post-Mortems 95

25.1 The debug hardware	95
25.2 Halting versus not halting	95
25.3 Logging that does not distort what it measures	95
25.4 Decoding a hard fault by hand	96
25.5 A method, not a ritual	96

26. Testing Firmware 98

26.1 The firmware test pyramid	98
26.2 Making firmware testable	98
26.3 The layers above	98
26.4 Static analysis and build hygiene	99
26.5 Continuous integration for firmware	99

27. Board Bring-Up 101

27.1 Before power is applied	101
27.2 The bring-up sequence	101
27.3 Reading hardware symptoms	102
27.4 First-light firmware: what to write before the drivers	102
27.5 Measuring what you cannot see	102

28. Reliability: Failing Well 104

28.1 A strategy for errors, not a habit	104
28.2 Assertions, and what they should do in the field	104
28.3 Watchdogs as an architecture, not a line of code	104
28.4 Layered recovery	105
28.5 Evidence from the field	105
28.6 Data integrity over years	105

29. Low-Power Firmware 107

29.1 The only equation that matters	107
29.2 Where the microamps actually go	107
29.3 Design patterns for low power	108
29.4 Entering and leaving deep sleep, correctly	108
29.5 Energy per operation, and what it implies	109

30. Performance and Optimisation 110

30.1 Measure first, and measure the right thing	110
30.2 Where the cycles actually go	110
30.3 The order to try things	111
30.4 Optimising for size	111
30.5 When to stop	111

PART V - Shipping and Trusting It	113
31. Bootloaders and Firmware Update	114
31.1 Memory layout is the design	114
31.2 The update sequence, step by step	114
31.3 Rules for the bootloader itself	115
31.4 Rolling out to a fleet	115
32. Embedded Security	116
32.1 Threat model first	116
32.2 Secure boot: the root of trust	116
32.3 Cryptography that fits on a microcontroller	117
32.4 Attack surface in ordinary firmware	117
32.5 Regulation is now part of the job	118
33. Functional Safety and Certification	119
33.1 The landscape	119
33.2 What certification actually demands	119
33.3 Techniques worth borrowing even without an auditor	119
33.4 Self-tests and diagnostic coverage in practice	120
34. When a Microcontroller Is Not Enough: Embedded Linux	122
34.1 The trade, stated plainly	122
34.2 The boot chain	122
34.3 Device tree, in one page	122
34.4 Building the system	123
34.5 User space or a kernel driver?	123
34.6 Real-time on Linux, honestly	124
35. Production: Manufacturing, Provisioning and the Field	125
35.1 Getting firmware onto the board	125
35.2 Factory test firmware	125
35.3 Identity, keys and calibration	125
35.4 The field, and what comes back	126
PART VI - Practice	127
36. Connected Devices and Edge Intelligence	128
36.1 The anatomy of connected-device firmware	128
36.2 Machine learning on a microcontroller	128
36.3 On-device adaptation, briefly	129
37. Professional Practice	130
37.1 Version control and the things that are not source	130
37.2 Reviewing firmware	130
37.3 Documentation that is worth writing	130
37.4 Estimation, honestly	131

37.5 Working with the hardware team	131
38. A Complete Product, Stage by Stage	132
38.1 Stage 0 - Requirements, before anything else	132
38.2 Stage 1 - Choose the parts and the software architecture	132
38.3 Stage 2 - Review the schematic while it can still change	132
38.4 Stage 3 - Repository, toolchain and CI on day one	132
38.5 Stage 4 - Bring up the board	133
38.6 Stage 5 - Drivers, one at a time, each with tests	133
38.7 Stage 6 - The timing budget, written down and checked	133
38.8 Stage 7 - Power engineering, measured not assumed	133
38.9 Stage 8 - Storage, configuration and calibration	133
38.10 Stage 9 - Bootloader and OTA, built before they are needed	134
38.11 Stage 10 - Security and provisioning	134
38.12 Stage 11 - Testing, on the host and in the loop	134
38.13 Stage 12 - Production test and calibration	134
38.14 Stage 13 - Release, roll out, and watch	134
38.15 Stage 14 - Maintain	134
38.16 The timeline, honestly	134
38.17 The one-page master checklist	135

APPENDICES 137

Appendix A. Formula and Reference Sheet	139
Clocks, timers and serial	139
Analogue	139
Power and energy	139
Real-time	139
Cortex-M quick reference	140
Bit manipulation	140
Toolchain commands	141
Appendix B. Glossary	142
Appendix C. Study Roadmap, Projects and Tools	145
A 16-week study plan	145
A ladder of projects	145
The tools worth owning	145
How to keep learning	146

PART I

Foundations

What firmware is, the machine it runs on, how bits stand for voltages and time, the C that firmware actually uses, the toolchain that turns it into an image, and everything that happens between the reset pin and the first line of `main()`.

1. What Firmware Is

Firmware is the software that gives a piece of hardware its behaviour. It is the code in your washing machine, your car's brake controller, your earbuds, the drone's flight stack, the insulin pump, and the keyboard you are typing on - which contains a microcontroller running a program that someone had to write, debug, and update.

1.1 A first program, and everything it implies

```
/* Blink an LED on a Cortex-M microcontroller. */
#include <stdint.h>

#define RCC_AHB1ENR (*(volatile uint32_t *)0x40023830u)
#define GPIOA_MODER (*(volatile uint32_t *)0x40020000u)
#define GPIOA_ODR (*(volatile uint32_t *)0x40020014u)

int main(void)
{
    RCC_AHB1ENR |= (1u << 0);          /* clock on for GPIO port A */
    GPIOA_MODER &= ~(3u << (5 * 2));    /* clear the 2 bits for pin 5 */
    GPIOA_MODER |= (1u << (5 * 2));     /* 01 = general purpose output */

    for (;;) {
        GPIOA_ODR ^= (1u << 5);        /* toggle the pin */
        for (volatile uint32_t i = 0; i < 400000u; i++) { }
    }
}
```

Thirteen lines that contain most of this book in miniature.

Look at what that program assumes. It writes to fixed numeric addresses - there is no operating system to ask for memory, and those numbers are wired into the silicon. It enables a **clock** before touching a peripheral, because an unclocked peripheral silently ignores writes. It does read-modify-write on a register whose other bits belong to other pins. It uses **volatile** because the value at that address changes outside the C abstract machine. It never returns, because there is nothing to return to. And its delay loop is wrong in three ways we will fix in Chapter 10.

THE DEFINITION TO CARRY THROUGH THE BOOK

Firmware is software written with the memory map, the clock tree, the interrupt controller and the electrical timing of a specific device as first-class concerns. Application software is written against abstractions that hide all four. Everything that makes firmware difficult, and everything that makes it interesting, follows from that sentence.

1.2 The landscape: five classes of embedded system

Class	Typical device	Memory	Software model
8/16-bit MCU	AVR, PIC, MSP430; a thermostat, a toy	1-16 KB flash, 128 B - 2 KB RAM	Superloop, no OS, hand-written drivers
32-bit MCU (the centre of gravity)	Cortex-M0/M4/M7, RISC-V; sensors, motor control, wearables	32 KB - 2 MB flash, 8-512 KB RAM	Bare metal or RTOS; vendor HAL plus your own drivers
MCU + radio (connected)	BLE/Wi-Fi/LoRa SoC; IoT devices	256 KB - 4 MB flash	RTOS plus a vendor protocol stack; OTA update mandatory
Application processor (MPU)	Cortex-A, RISC-V 64; gateways, infotainment, cameras	512 MB+ DRAM, eMMC storage	Embedded Linux: bootloader, kernel, device tree, user space

Class	Typical device	Memory	Software model
Safety / real-time controller	Automotive ECU, PLC, medical pump	Varies, often lockstep cores	Certified RTOS or bare metal, MISRA C, full traceability

This book centres on the second and third rows - the 32-bit microcontroller, with and without a radio - because that is where most firmware is written, and because everything learned there transfers upward to Linux (Chapter 34) and downward to 8-bit parts. Where a topic is specific to one class, it says so.

1.3 Firmware, drivers, and the words people argue about

Term	Working definition
Firmware	Software stored in non-volatile memory on the device it controls, shipped as part of the hardware product.
Bare metal	Firmware with no operating system: one program owns the CPU, memory and every peripheral.
RTOS	A small scheduler and synchronisation library linked into your firmware. Not an operating system in the desktop sense: no processes, no memory protection by default, no shell.
Embedded Linux	A full kernel with virtual memory, processes and drivers, running on an application processor. Different discipline, covered in Chapter 34.
Driver	The layer that turns a peripheral's registers into an API. In firmware you frequently write it yourself.
HAL / BSP	Vendor code abstracting a chip family (HAL) or adapting software to one board (BSP). Useful and frequently over-trusted - Chapter 23.
Real-time	Correctness depends on when the answer arrives, not only on the answer. Hard real-time means a late answer is a failure; soft real-time means it is merely bad.

REAL-TIME DOES NOT MEAN FAST

A system that responds in 50 ms, always, without exception, is hard real-time. A system that usually responds in 1 ms but occasionally takes 200 ms is not - and it is the one that will fail its acceptance test. The whole of Chapter 20 exists to turn 'usually fast' into 'provably bounded'.

1.4 What makes firmware different to write

Constraint	Consequence for your code
Kilobytes of RAM, not gigabytes	Static allocation, fixed-size buffers, no <code>malloc()</code> in steady state (Chapter 24)
No screen, no shell, often no console	Debugging is done with a debug probe, a pin, and a logic analyser (Chapter 25)
The program must never exit or crash	Every error path must lead somewhere defined; watchdogs catch what escapes (Chapter 28)
Concurrency arrives whether you want it or not	An interrupt can preempt any instruction; shared data needs the discipline of Chapters 9 and 18
Timing is part of correctness	Deadlines, jitter and latency get budgets and measurements (Chapter 20)
Energy is a currency	Microamps at 3 V decide whether a product runs for a day or a year (Chapter 29)
Updates are risky and sometimes impossible	A failed update can brick the unit; hence dual-bank bootloaders and rollback (Chapter 31)
The hardware is also new	Half of early firmware bugs are hardware bugs, footprint errors, or datasheet errata

1.5 The shape of a firmware project

application logic	state machines, control loops, UX
services	protocol stacks, storage, logging, OTA update, telemetry
RTOS or superloop	scheduling, timers, synchronisation
device drivers	UART, SPI, I2C, ADC, flash, sensors
HAL / register access	CMSIS headers, vendor HAL, your own
startup + linker	vector table, .data/.bss init, map
SILICON	core, buses, peripherals, clocks

The layering every firmware project converges on, whether or not anyone drew it. Chapters 5-6 build the bottom two layers, Part II the next two, Part III the top three.

1.6 How to read this book with a board on your desk

You need three things: a development board (any Cortex-M board with a debug probe on it will do), a debugger - the vendor's IDE or `arm-none-eabi-gdb` with OpenOCD - and a way to see signals. A twenty-dollar USB logic analyser is transformative; an oscilloscope is better. Do not attempt Part II by reading alone. Firmware is a laboratory subject.

Your first hour with a new board

- ☐ Build and flash the vendor's blink example unmodified. Confirm it runs.
- ☐ Attach the debugger, halt the core, and read the program counter.
- ☐ Set a breakpoint in `main()`, reset, and confirm it is hit.
- ☐ Find the datasheet, the reference manual, and the errata sheet. Download all three - they are different documents.
- ☐ Locate the board schematic and identify which pin the LED is on and whether it is active-high or active-low.
- ☐ Toggle a spare pin in a loop and look at it with an analyser.

Exercises

- 1 Get the blink program above running on any board, then change the delay constant and measure the resulting frequency on a pin. Explain the difference from what you predicted.
- 2 Remove the `volatile` from the delay loop counter, rebuild with optimisation enabled, and observe what happens. Keep this result: it is the shortest possible explanation of Chapter 4.
- 3 List every microcontroller in the room you are sitting in. Most people find between five and fifty.
- 4 For a product you own, write down what its firmware must do in the first 100 ms after power-on. Compare with Chapter 6 afterwards.

2. The Machine: Computer Architecture for Firmware Engineers

You cannot write good firmware for a machine you cannot picture. This chapter builds that picture: the core, its registers, the buses, the memory map, and the handful of core peripherals that every Cortex-M device shares. None of it is optional knowledge - every item here appears later as the explanation of a bug.

2.1 The core: fetch, decode, execute

A CPU repeats one cycle forever: fetch the instruction at the program counter, decode it, execute it, and advance. Pipelining overlaps those stages so that a new instruction starts before the previous one finishes - a three-stage pipeline on Cortex-M3/M4, six on Cortex-M7 with branch prediction and dual issue.

cycle:	1	2	3	4	5	6
instr A	fetch	decode	exec			
instr B		fetch	decode	exec		
instr C			fetch	decode	exec	
instr D				fetch	decode	exec

A taken branch discards the instructions already fetched:
the 'branch penalty' - typically 2-3 cycles on a Cortex-M.

Pipelining is why an instruction's cost is not a fixed number, and why a tight loop with an unpredictable branch is slower than its instruction count suggests.

THE PC READS AHEAD

On the Cortex-M, reading the program counter gives the address of the current instruction plus 4, because two instructions have already been fetched. This matters exactly twice: when hand-writing position-relative assembly, and when decoding a fault (Chapter 25).

2.2 The programmer's model

Register	Role
R0-R12	General purpose. R0-R3 pass the first four arguments and R0 returns the result (the AAPCS calling convention); R4-R11 must be preserved by a called function.
R13 / SP	Stack pointer. Two banked copies: MSP (main, used by handlers and by default) and PSP (process, used by RTOS tasks).
R14 / LR	Link register: the return address of a call. Inside an exception it holds EXC_RETURN , an encoded value that says which stack and mode to return to.
R15 / PC	Program counter. Bit 0 must be 1 on Cortex-M - it selects Thumb state, and clearing it causes an immediate usage fault.
xPSR	Status: N/Z/C/V flags, the executing exception number, and the Thumb bit.
PRIMASK, BASEPRI, FAULTMASK	Interrupt masking (Chapter 9). PRIMASK disables all configurable interrupts; BASEPRI disables only those below a chosen priority.
CONTROL	Selects MSP/PSP and privileged/unprivileged execution in thread mode.

The core is in one of two modes at all times: **thread mode**, running your `main()` or an RTOS task, and **handler mode**, running an exception or interrupt. Handler mode is always privileged and always uses the MSP. This split is the hardware foundation of every RTOS context switch in Chapter 21.

2.3 Which core is in front of you

Core	Notable features	Where it is used
Cortex-M0/M0+	ARMv6-M, ~3-stage, no divide (M0), tiny gate count, no bit-banding	Cost- and power-critical: sensors, peripherals, second cores
Cortex-M3	ARMv7-M, hardware divide, bit-banding, MPU option	General purpose control
Cortex-M4/M4F	M3 plus DSP (SIMD, MAC) and optional single-precision FPU	Motor control, audio, sensor fusion - the workhorse
Cortex-M7	Superscalar, caches, TCM, double-precision FPU option	High-throughput: imaging, high-rate control
Cortex-M23/M33/M55	ARMv8-M with TrustZone-M security extension; M55 adds Helium vector extensions	Secure IoT, edge ML
RISC-V (RV32IMAC)	Open ISA; machine/user modes, CLINT/PLIC instead of NVIC	Increasingly common in new silicon
8-bit AVR / PIC	Harvard, tiny register files, no interrupt priorities	Legacy and ultra-low-cost designs

2.4 Memory, buses and the address map

Cortex-M devices use a single flat 4 GB address space that is carved into fixed regions by the architecture. Everything - flash, RAM, peripheral registers, even the core's own control registers - is at an address you can dereference.

0xFFFF_FFFF	+-----+	
	Vendor / system	
0xE000_0000	+-----+	Private Peripheral Bus
	NVIC, SysTick, SCB, DWT, ITM	(core peripherals)
0xA000_0000	+-----+	
	External device / SDRAM	
0x6000_0000	+-----+	external RAM
0x4000_0000	+-----+	PERIPHERALS (APB/AHB)
	GPIO, UART, SPI, timers, DMA	
0x2000_0000	+-----+	SRAM
	.data .bss heap stack	
0x0800_0000	+-----+	FLASH (typical)
	vector table, .text, .rodata	
0x0000_0000	+-----+	aliased boot region

The canonical Cortex-M map. Vendors place flash and RAM inside these windows; the top region above 0xE0000000 is architectural and identical across every Cortex-M part.

Behind that flat view is a **bus matrix**: the core, the DMA controllers and other masters contend for slaves (flash, each SRAM bank, each peripheral bus). Two consequences follow, and both surprise people:

- **Peripheral writes are posted.** A write to a peripheral register may still be in flight when the next instruction executes. If you clear an interrupt flag at the end of an ISR and return immediately, the interrupt can fire a second time. The fix is to read the register back (which stalls until the write lands) or to issue a `__DSB()`.
- **Bandwidth is shared.** A DMA transfer streaming into the same SRAM bank the CPU is using steals cycles from the CPU. Placing DMA buffers in a different bank than the stack is a real optimisation, not superstition.

FLASH WAIT STATES, AND WHY YOUR CODE IS NOT 3X FASTER

Flash on a typical MCU has an access time near 25-30 ns regardless of core speed. At 24 MHz (41.7 ns per cycle) zero wait states are needed. At 168 MHz a cycle is 5.95 ns, so a 30 ns access needs $\text{ceil}(30 / 5.95) - 1 = 5$ **wait states**: every instruction fetch that misses the prefetch buffer costs six cycles instead of one. This is why vendors add prefetch buffers, instruction caches and ART accelerators, why raising the clock without configuring wait states **hard-faults instantly**, and why the same loop is fast from RAM (zero wait states) and slow from flash.

2.5 Alignment, endianness, and the two bugs they cause

Cortex-M is little-endian by default: the least significant byte is at the lowest address. The 32-bit value 0x12345678 stored at 0x2000'0000 appears in memory as 78 56 34 12. Network protocols are usually big-endian, so every packet parser needs explicit byte swapping - never a cast of a byte pointer to a `uint32_t *`.

```
/* Portable, correct, and compiles to a single REV instruction */
/* on ARM when the compiler recognises the idiom. */
static uint32_t be32_read(const uint8_t *b)
{
    return ((uint32_t)b[0] << 24) | ((uint32_t)b[1] << 16) |
           ((uint32_t)b[2] << 8) | ((uint32_t)b[3]);
}

/* WRONG: alignment and endianness both undefined behaviour here */
/* uint32_t v = *(uint32_t *)&buffer[1]; */
```

Unaligned access is permitted for normal loads and stores on ARMv7-M but **not** for **LDM/STM**, not for anything in the peripheral region, and not at all on ARMv6-M (Cortex-M0). Since the compiler chooses those instructions, an unaligned pointer that works today breaks after an optimisation change. Copy bytes, or use `memcpy` - modern compilers turn a fixed-size `memcpy` into the right instruction anyway.

2.6 Caches, TCM and the DMA coherency trap

Cortex-M7 and application processors have data and instruction caches. They make code faster and introduce a class of bug that does not exist on smaller parts: **the CPU and the DMA controller can disagree about memory**.

```
CPU writes buffer -> lands in D-cache, not yet in SRAM
DMA reads SRAM    -> sends stale data out of the UART

DMA writes SRAM   -> CPU reads its cached copy
                  -> sees the old contents

Fix: SCB_CleanDCache_by_Addr()    before DMA reads your buffer
     SCB_InvalidateDCache_by_Addr() after DMA writes your buffer
     (address 32-byte aligned, length a multiple of 32)
```

Cache maintenance around DMA. The alternative is to place DMA buffers in a region configured as non-cacheable by the MPU, which is simpler and usually fast enough.

Tightly-coupled memory (ITCM/DTCM) is SRAM wired directly to the core with no cache and no bus arbitration: deterministic single-cycle access. Put the hottest interrupt handlers and their data there when jitter matters more than average speed.

2.7 MPU, MMU and what protection you actually get

	MPU (Cortex-M)	MMU (Cortex-A, Linux)
Granularity	A handful of regions (8-16), each a power of two	4 KB pages, arbitrarily many
Translation	None - physical addresses only	Virtual to physical translation
Typical use	Catch stack overflow, make a region read-only or execute-never, isolate an RTOS task	Full process isolation
Cost of a violation	MemManage fault	SIGSEGV in one process
Realistic benefit	Turns silent corruption into an immediate, debuggable fault - worth configuring even minimally	Complete isolation between programs

2.8 The core peripherals you get for free

Block	Address	What it does
NVIC	0xE000E100	Enable, prioritise and pend interrupts (Chapter 9)
SysTick	0xE000E010	A 24-bit down-counter with an interrupt: the RTOS tick or a simple millisecond timebase
SCB	0xE000ED00	System control: vector table offset, fault status registers, reset request, sleep behaviour
MPU	0xE000ED90	Region-based memory protection
DWT	0xE0001000	Data watchpoints and the cycle counter - the best profiling tool on the chip (Chapter 30)
ITM/TPIU	0xE0000000	Instrumentation trace: printf-speed logging over SWO with almost no CPU cost (Chapter 25)

ENABLE THE CYCLE COUNTER ON DAY ONE

Three writes give you an exact, free, always-available stopwatch: `CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk; DWT->CYCCNT = 0; DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;`. After that, `uint32_t t0 = DWT->CYCCNT;` around any code gives its cost in cycles. Almost every performance argument in a firmware team can be settled with this in five minutes.

Exercises

- 1 Find your device's reference manual and write down the base address of GPIO port A, one timer, and the SRAM. Verify them against the linker script.
- 2 Enable the DWT cycle counter and measure the cost of an empty function call, a 32-bit divide, and a `float` multiply. Compare with and without the FPU enabled.
- 3 Compute the required flash wait states for your device at its maximum clock, then check what the vendor's clock configuration code actually programs.
- 4 Write the same 32-bit big-endian read three ways - byte shifts, `memcpy` plus a byte swap, and a pointer cast - and compare the disassembly at -O2.
- 5 If your part has a cache, deliberately reproduce the DMA coherency bug and then fix it twice: once with cache maintenance, once with an MPU non-cacheable region.

3. Bits, Numbers and the Physical World

Firmware's job is to move between two representations: voltages, currents and times on one side, integers on the other. This chapter is the arithmetic of that translation - bit manipulation, integer behaviour that C gets subtly wrong, fixed-point maths, data packing, and the checksums that tell you whether the bytes survived the wire.

3.1 Bit manipulation: the six idioms

```
#define BIT(n)          (1u << (n))
#define SET_BIT(r, m)   ((r) |= (m))
#define CLR_BIT(r, m)   ((r) &= ~(m))
#define TGL_BIT(r, m)   ((r) ^= (m))
#define TST_BIT(r, m)   (((r) & (m)) != 0u)

/* Extract a field: value of bits [hi:lo] */
#define FIELD_GET(r, lo, width) \
    (((r) >> (lo)) & ((1u << (width)) - 1u))

/* Insert a field without disturbing the neighbours */
#define FIELD_SET(r, lo, width, v) \
    ((r) = ((r) & ~((1u << (width)) - 1u) << (lo)) | \
     ((v) & ((1u << (width)) - 1u)) << (lo))
```

Write these once, in one header, and never open-code a shift again. The bugs they prevent - forgetting to mask before shifting in, clobbering adjacent bits - are the most common register-access defects in firmware.

Trick	Expression	Use
Lowest set bit	<code>x & -x</code>	Priority encoders, allocators
Clear lowest set bit	<code>x & (x - 1)</code>	Iterating set bits
Power of two?	<code>x && !(x & (x - 1))</code>	Buffer size assertions
Round up to a power of two multiple	<code>(x + (a-1)) & ~(a-1)</code>	Alignment (a must be a power of two)
Count leading zeros	<code>__CLZ(x)</code>	Single-cycle log2 on Cortex-M3+; the fast way to find the highest priority pending bit
Bit reverse / byte reverse	<code>__RBIT(x), __REV(x)</code>	Endianness and CRC hardware

3.2 Integers in C, where firmware bugs live

C's integer rules were designed for portability across machines that no longer exist, and they bite hardest on 8- and 16-bit values, which is exactly what firmware handles. Three rules explain almost every surprise:

- **Integer promotion:** any operand smaller than `int` is promoted to `int` (32-bit signed on Cortex-M) before arithmetic. So `uint8_t a = 200, b = 100; (a + b)` is 300, not 44.
- **The usual arithmetic conversions:** mixing signed and unsigned of the same rank converts the signed operand to unsigned. `if (-1 < sizeof(x))` is `false`.
- **Shifting:** shifting a signed value left into the sign bit is undefined behaviour, and shifting by more than the width - including `1u << 32` - is undefined too, not zero.

THE 16-BIT SHIFT BUG, IN FULL

`uint16_t crc = 0xFFFF; crc = (crc << 8) | (crc >> 8);` looks like a byte swap. What happens: `crc` promotes to `int`, `crc << 8` becomes `0x00FFFF00` in 32 bits, the OR gives `0x00FFFFFF`, and the assignment truncates to `0xFFFF`. On a 16-bit target the shift would instead be undefined behaviour on the sign bit. The correct form masks explicitly: `crc = (uint16_t)(((crc << 8) | (crc >> 8)) & 0xFFFFu)`; **Always cast the result back to the target width, and always use unsigned types for anything bit-oriented.**

Type	Use it for
<code>uint8_t, uint16_t, uint32_t</code>	Registers, protocol fields, anything with a defined width - always from <code><stdint.h></code>
<code>int</code>	Loop counters and general arithmetic where width does not matter; it is the machine's natural size and often the fastest
<code>uint_fast8_t</code>	A small counter you want fast rather than small
<code>size_t</code>	Sizes and array indices
<code>bool (<stdbool.h>)</code>	Truth values - not <code>uint8_t</code> , so the reader knows
<code>char</code>	Text only. Its signedness is implementation-defined; never use it for raw bytes

3.3 Fixed point: real numbers without a floating-point unit

A fixed-point number is an integer with an implied binary point. In **Q15**, a 16-bit signed integer represents a value in $[-1, 1)$ with a scale factor of 2^{-15} : the stored integer 16384 means 0.5.

Qm.n: **real value = stored_integer / 2ⁿ**
Add/sub: **same Q -> just add the integers**
Multiply: **Qm.n * Qp.q = Q(m+p).(n+q) -> shift right by n to**
 return to Qm.n; use a 32-bit intermediate for Q15
Divide: **shift the numerator left by n first**
Range Q15: **-1.0 to 0.999969, resolution 1/32768 = 3.05e-5**

A Q15 MULTIPLY, STEP BY STEP

Multiply 0.75 by 0.5 in Q15. 0.75 becomes `round(0.75 x 32768) = 24576`; 0.5 becomes 16384. The product `24576 x 16384 = 402,653,184`, which needs 32 bits - this is why the intermediate must be `int32_t`, and on a 16-bit machine why it must be explicit. Converting back: `402,653,184 >> 15 = 12,288`, and `12,288 / 32768 = 0.375`, which is correct. Note the two hazards: (1) shifting right truncates towards negative infinity, so add `1 << 14` before shifting to round to nearest; (2) the single value `-1.0 x -1.0` overflows Q15, which is why DSP libraries use saturating instructions.

```
typedef int16_t q15_t;

static inline q15_t q15_mul(q15_t a, q15_t b)
{
    int32_t prod = (int32_t)a * (int32_t)b;    /* Q30 */
    prod = (prod + (1 << 14)) >> 15;          /* round, back to Q15 */
    if (prod > 32767) prod = 32767;            /* saturate */
    if (prod < -32768) prod = -32768;
    return (q15_t)prod;
}
```

On a Cortex-M4 the DSP extension does this in one instruction (**SMULBB** plus **SSAT**, or **__SSAT** from CMSIS). Use CMSIS-DSP rather than hand-rolling a filter - but understand this function first.

Representation	When it is right	Cost on Cortex-M4F
Integer / fixed point	No FPU, tight loops, exact scaling, safety code that must be deterministic	1 cycle multiply
float with FPU	M4F/M7 with hardware FPU; readability wins	1-3 cycles - as cheap as integer maths
float without FPU	Rare: software emulation	20-100+ cycles, plus several KB of library
double	Almost never on an MCU - single-precision FPUs silently fall back to software	Hundreds of cycles

THE `DOUBLE` TRAP

`float x = 1.0 / 3.0;` computes in **double** and then converts, because unsuffixed floating constants are **double**. On a single-precision FPU that pulls in the software double library and costs hundreds of cycles inside what looked like a one-line expression. Write `1.0f / 3.0f`. Enable `-Wdouble-promotion` and fix every warning it produces; the ones in `printf` arguments are unavoidable, the rest are free performance.

3.4 Structures, padding and data that crosses a boundary

The compiler inserts padding so that each member is naturally aligned. A struct that looks like 7 bytes is often 12.

```
struct { uint8_t a; uint32_t b; uint16_t c; };

offset:  0    1 2 3    4 5 6 7    8 9    10 11
         [a]  [pad pad pad][ b  b  b  b ][ c  c ][pad pad]

sizeof = 12, not 7. Reorder largest-first:
struct { uint32_t b; uint16_t c; uint8_t a; }; -> sizeof = 8
```

Ordering members from largest to smallest is free memory. On a device with 8 KB of RAM and a few hundred structs, it is not a micro-optimisation.

NEVER SEND A STRUCT OVER A WIRE

Writing a struct directly to a UART, a radio or a flash record binds your protocol to one compiler's padding, one endianness and one version of the struct. It works until the day the other end is built differently, and then it fails in the field. **Serialise field by field into a byte buffer** with explicit widths and byte order. It costs twenty lines and removes an entire class of interoperability bug. The same applies to `__attribute__((packed))`, which additionally creates unaligned members whose address cannot legally be taken.

3.5 Checksums and CRCs

Every byte that leaves the chip - over a wire, into flash, through a radio - can come back wrong. A checksum tells you. The choice is between cost and detection strength.

Method	Cost	Detects	Use
Sum / XOR of bytes	Trivial	Single bit errors; misses reordering and many double errors	Legacy protocols only
Fletcher / Adler	Cheap	Better than a sum, weaker than CRC	Software checksums where CRC is unavailable
CRC-8 / CRC-16	Small table or hardware	All 1-2 bit errors, all odd numbers of errors, all bursts up to the CRC width	Sensor buses, short frames
CRC-32	Hardware on most MCUs	As above with a 32-bit burst guarantee	Firmware images, flash records, Ethernet
SHA-256 / HMAC	Thousands of cycles	Deliberate tampering, not just noise	Secure boot and OTA (Chapters 31-32)

A CRC COMPUTED BY HAND

A CRC is the remainder of polynomial division in GF(2), where subtraction is XOR. Take the generator $x^3 + x + 1$, i.e. the bit pattern 1011, and the message 1101. Append 3 zero bits to make 1101000, then XOR the generator in wherever the leading bit is set: 1101000 XOR 1011000 = 0110000; 0110000 XOR 0101100 = 0011100; 0011100 XOR 0010110 = 0001010; 0001010 XOR 0001011 = 0000001. The remainder is **001**, so the transmitted frame is 1101001. The receiver divides the whole frame by 1011 and expects a remainder of **zero** - and flipping any single bit of the frame changes that remainder, which is exactly the property being sought. Real CRCs differ only in the polynomial's width, an initial value, whether the bits are reflected, and a final XOR; get those four parameters wrong and two correct implementations will disagree forever.

```
/* Table-driven CRC-32 (reflected, poly 0xEDB88320) - the standard */
/* used by Ethernet, zlib, and most firmware image formats.      */
uint32_t crc32(const uint8_t *data, size_t len, uint32_t crc)
{
    crc = ~crc;
    while (len--) {
        crc ^= *data++;
        for (int k = 0; k < 8; k++)
            crc = (crc >> 1) ^ (0xEDB88320u & (-(int32_t)(crc & 1)));
    }
    return ~crc;
}
```

The bit-at-a-time version: no table, about 8 cycles per bit. A 256-entry table costs 1 KB of flash and runs 8x faster; many MCUs have a CRC peripheral that does it with zero CPU cost, and you should use it.

3.6 From counts to physical units

ADC: $V_{in} = code * V_{ref} / 2^N$

12-bit, $V_{ref} = 3.300\text{ V}$: $LSB = 3.300 / 4096 = 0.806\text{ mV}$

Sensor with gain G and offset 0 (volts):

$physical = (V_{in} - 0) / G$

Two-point calibration from measurements $(c1, p1)$, $(c2, p2)$:

$physical = p1 + (code - c1) * (p2 - p1) / (c2 - c1)$

Always calibrate in the units the customer cares about, and store the coefficients in non-volatile memory per unit (Chapter 35).

FIXED-POINT SCALING IN PRACTICE

Do not convert an ADC count to volts and then to degrees in floating point at 1 kHz. Keep integers and scale once: store temperature in **millidegrees** as an `int32_t`, current in **microamps**, voltage in **millivolts**. Integer units chosen 1000x finer than the resolution you need eliminate rounding drift, make logs greppable and comparisons exact, and cost nothing.

Exercises

- 1 Write the field get/set macros above and use them to configure a real peripheral register, checking the result in the debugger.
- 2 Reproduce the 16-bit shift bug on your target and then fix it. Look at the disassembly of both versions.
- 3 Implement `q15_mul` and verify the 0.75×0.5 example, then find an input pair where the truncating version and the rounding version differ.
- 4 Compute `sizeof` for three orderings of the same five struct members and explain each result.
- 5 Compute the CRC of 1101 by hand as in the box, then confirm it with a five-line program.
- 6 Take a real sensor, do a two-point calibration, and report the residual error at a third point.

4. The C You Actually Need: Firmware Dialect

Firmware is written in a dialect of C that application programmers rarely use: `volatile` everywhere it matters, no dynamic allocation, undefined behaviour treated as a hazard rather than a curiosity, and a standard library used selectively. This chapter is that dialect.

4.1 volatile: what it does, and the three things it does not

`volatile` tells the compiler that the value at this address can change outside the visible flow of the program, so every read must be a real load and every write a real store. Without it, the optimiser is entitled to cache a register value in a CPU register forever - which is why the un-`volatile` delay loop in Chapter 1 disappears at -O2.

Situation	volatile needed?
Memory-mapped peripheral register	Yes, always
Variable written by an ISR and read by the main loop	Yes - otherwise the loop may never re-read it
Variable shared between two RTOS tasks	Not sufficient - you need a mutex, queue or atomic (Chapter 19). <code>volatile</code> alone does not order or protect anything
A buffer filled by DMA	Yes for the flag; the buffer also needs cache maintenance on cached parts
A local variable	No - unless it must survive <code>setjmp</code>

THE THREE THINGS VOLATILE DOES NOT GIVE YOU

(1) Atomicity. `volatile uint32_t counter; counter++;` is still load, add, store - an interrupt between them loses a count. **(2) Ordering with respect to other variables.** The compiler may not reorder two volatile accesses relative to each other, but the hardware write buffer still can; use `__DMB()/__DSB()` when order across the bus matters. **(3) Protection from other cores.** On multi-core parts you need real atomics (`stdatomic.h`, or `LDREX/STREX`).

```
/* Correct placement matters: read it right-to-left. */
volatile uint32_t *p; /* pointer to volatile data */
uint32_t *volatile p; /* volatile pointer to normal data */
volatile uint32_t *volatile p; /* both */

/* The CMSIS convention for peripheral structs: */
typedef struct {
    __IO uint32_t CR; /* __IO = volatile (rw) */
    __I uint32_t SR; /* __I = volatile const (ro) */
    __O uint32_t DR; /* __O = volatile (wo) */
} UART_TypeDef;
#define UART1 ((UART_TypeDef *)0x40011000u)
```

A peripheral struct overlaid on an address is the standard idiom: it gives named fields, correct offsets by construction, and a single place to fix if a register moves in the next silicon revision.

4.2 Undefined behaviour is not theoretical

Undefined behaviour	How it shows up in firmware
Signed integer overflow	The optimiser deletes your <code>if (x + 1 < x)</code> overflow check entirely
Strict aliasing violation	A value written through a <code>uint32_t *</code> is not seen when read through a <code>float *</code> ; the bug appears only at -O2
Unaligned access	Hard fault on Cortex-M0, or on any core inside an LDM the compiler chose

Undefined behaviour	How it shows up in firmware
Reading an uninitialised variable	Works on the bench because RAM happens to be zero after power-on, fails after a warm reset
Out-of-bounds array write	Silent corruption of whatever the linker placed next - usually another module's state
Shift by $\geq$ width, or 1 $<$ 31 on a signed int	Different results at different optimisation levels
Modifying an object twice between sequence points	<code>i = i++ + 1;</code> - compiles, means nothing

```

/* Type punning: the two legal ways. */
float f;
uint32_t bits;

memcpy(&bits, &f, sizeof bits);      /* always correct; the      */
                                      /* compiler emits one move */
union { float f; uint32_t u; } cvt;  /* legal in C (not in C++) */
cvt.f = f; bits = cvt.u;

/* bits = *(uint32_t *)&f;  <-- strict aliasing violation */

```

4.3 Storage, scope and linkage

- **`static` at file scope** makes a symbol internal: it cannot be linked from another file, it can be optimised harder, and it does not collide with a same-named symbol elsewhere. Make everything static except the deliberate API of the module.
- **`static` inside a function** gives a variable permanent storage. In firmware this is usually right (no stack cost, no allocation) and occasionally a disaster (it makes the function non-reentrant - see Chapter 18).
- **`const`** on a lookup table puts it in flash instead of RAM, which on a device with 20 KB of RAM is the difference between fitting and not. Check the map file to confirm it landed in `.rodata`.
- **Weak symbols** let the vendor's startup file define a default handler that your file silently overrides: `void __attribute__((weak)) TIM2_IRQHandler(void);`. Convenient, and a classic source of 'my ISR never runs' when the name is misspelled by one character.
- **Section attributes** place objects deliberately: `__attribute__((section(".noinit")))` for data that must survive a warm reset, or a fast-RAM section for a hot ISR.

4.4 The standard library, selectively

Facility	Verdict in firmware
<code>memcpy</code> , <code>memset</code> , <code>memmove</code>	Use them - they are optimised assembly and often better than your loop
<code>strcpy</code> , <code>sprintf</code>	Avoid: unbounded. Use <code>strncpy</code> -style bounded helpers and <code>snprintf</code>
<code>printf</code> family	Large (10-40 KB with floats), slow, and blocking. Use a bounded logger over ITM/RTT (Chapter 25); if you must, link <code>nano.specs</code> and no float formatting
<code>malloc/free</code>	Fragmentation and unbounded latency. Allowed at init, forbidden in steady state (Chapter 24)
<code>assert</code>	Yes - but define your own that logs and resets rather than calling <code>abort()</code> (Chapter 28)
<code>errno</code> , <code>rand</code> , <code>strtok</code>	Non-reentrant unless the library is built thread-safe; a classic RTOS corruption source
Floating-point <code>printf</code>	Pulls in ~10 KB and a double library; print fixed-point integers instead

4.5 Patterns that make firmware readable

```

/* 1. Explicit error codes, never magic numbers. */
typedef enum { DRV_OK = 0, DRV_TIMEOUT, DRV_NACK, DRV_BUSY } drv_err_t;

/* 2. Compile-time checks cost nothing at run time. */
_Static_assert(sizeof(frame_t) == 16, "frame layout changed");
_Static_assert((RING_SIZE & (RING_SIZE - 1)) == 0, "power of two");

/* 3. Callbacks carry context - never a bare global. */
typedef void (*evt_cb_t)(void *ctx, uint8_t evt);
struct driver { evt_cb_t cb; void *cb_ctx; };

/* 4. Designated initialisers document intent and zero the rest. */
static const cfg_t cfg = { .baud = 115200u, .parity = PAR_NONE };

/* 5. A named constant for every magic number in a register write. */
#define UART_CR1_UE    (1u << 13)    /* USART enable */

```

MISRA C, USEFULLY SUMMARISED

MISRA C is a coding standard for safety-related C. Its genuinely valuable core is small: no dynamic memory after init, no recursion, one exit style used consistently, explicit types and casts, no reliance on implementation-defined behaviour, every `switch` has a `default`, every `if-else if` chain ends in `else`, no side effects in conditions, and braces on every block. Adopt those even if you are not certifying anything; adopt the full rule set with a checker only when a standard requires it, because deviations then need documenting.

4.6 C++ in firmware: the useful subset

Feature	Verdict	Reason
Classes, references, <code>constexpr</code>, templates	Use	Zero or negative run-time cost; <code>constexpr</code> moves work to compile time
RAII for locks and peripherals	Use	Guarantees the mutex is released and the pin restored on every exit path
<code>std::array</code> , <code>std::span</code> , <code><type_traits></code>	Use	Bounds and types checked with no run-time overhead
Virtual functions	Careful	One indirection plus a vtable per class; fine for a handful of drivers, wrong in a 10 kHz ISR
Exceptions, RTTI	Avoid	Tens of KB of unwind tables and unbounded latency; build with <code>-fno-exceptions -fno-rtti</code>
<code>new/delete</code> , STL containers that allocate	Avoid	Same fragmentation problem as <code>malloc</code>
<code>std::function</code>	Avoid in hot paths	May allocate; use a function pointer plus context, or a fixed-capacity delegate

STATIC INITIALISATION ORDER

Global C++ objects in different translation units are constructed in an unspecified order, by `__libc_init_array` before `main()` - and some of them will want a clock or a peripheral that is not configured yet. Prefer `constexpr`/POD globals plus an explicit `init()` called from `main()`, or a function-local static (constructed on first use).

Exercises

- 1 Take a peripheral register you use and write both a macro-based and a CMSIS-struct access to it. Compare the disassembly.
- 2 Write a loop that polls a flag without `volatile`, build at `-O0` and `-O2`, and diff the assembly.

- 3 Find one place in your codebase where a shared variable is only `volatile` and decide whether it needs a critical section instead.
- 4 Compile your project with `-Wall -Wextra -Wconversion -Wdouble-promotion` and fix every warning. Most teams find a real bug in the first hour.
- 5 Measure the flash cost of `printf("%f")` versus `printf("%d")` versus a hand-written fixed-point formatter.

5. The Toolchain: From Source to an Image on Flash

A firmware engineer who cannot read a map file or a linker script is working blind. This chapter follows one source file all the way to a binary image, and shows what each tool decides on the way.

5.1 The five stages

```
main.c --[ preprocessor ]--> main.i    macros, #includes expanded
      --[ compiler      ]--> main.s    architecture assembly
      --[ assembler     ]--> main.o    ELF object: sections+symbols
startup.o, drivers.o, libc.a |
                             v
      --[ linker + linker script ]--> firmware.elf
      --[ objcopy         ]--> firmware.bin / .hex  (what you flash)
```

firmware.elf also carries DWARF debug info - keep it forever,
it is what turns a field crash address back into a line number.

Each arrow is a program you can run by hand, and doing so once is the fastest way to understand the whole pipeline.

Tool (GNU arm-none-eabi-)	Question it answers
<code>gcc -E</code>	What does the preprocessor actually produce?
<code>gcc -S</code>	What instructions did my C become?
<code>nm --size-sort -S</code>	Which symbols are the biggest?
<code>size</code>	How much flash and RAM does this image use?
<code>objdump -d</code>	Disassemble - the ground truth
<code>readelf -S</code>	Which sections exist and where are they?
<code>addr2line -e fw.elf 0x0800A31C</code>	Which source line is this crash address?
<code>objcopy -O binary</code>	Produce the raw image for the flasher

5.2 Sections: where everything lives

Section	Contents	Stored in	At run time in
<code>.text</code>	Code	Flash	Flash (executed in place)
<code>.rodata</code>	<code>const</code> data, string literals	Flash	Flash
<code>.data</code>	Initialised globals	Flash (the initial image)	RAM - copied by startup
<code>.bss</code>	Zero-initialised globals	Nothing	RAM - zeroed by startup
<code>.noinit</code>	Data that must survive a warm reset	Nothing	RAM - deliberately untouched
<code>.heap / .stack</code>	Dynamic and automatic storage	Nothing	RAM

VMA AND LMA, THE IDEA MOST PEOPLE MISS

Every section has two addresses: the **VMA** (where it is when running) and the **LMA** (where it is stored in the image). For `.text` they are the same. For `.data` the LMA is in flash and the VMA is in RAM, and the startup code copies from one to the other. That single mechanism is why an initialised global costs both flash **and** RAM, and why a big `const` table must be `const` - otherwise it lands in `.data` and consumes RAM equal to its size.

5.3 The linker script, annotated

```

MEMORY {
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 512K
    RAM (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
}
_estack = ORIGIN(RAM) + LENGTH(RAM); /* stack grows downwards */

SECTIONS {
    .isr_vector : { KEEP(*(.isr_vector)) } > FLASH /* never GC this */
    .text : { *(.text*) *(.rodata*) . = ALIGN(4); } > FLASH
    _sidata = LOADADDR(.data); /* LMA: source in flash */
    .data : { _sdata = .; *(.data*) . = ALIGN(4); _edata = .; }
            > RAM AT > FLASH /* VMA in RAM, LMA in FLASH */
    .bss : { _sbss = .; *(.bss*) *(COMMON) _ebss = .; } > RAM
    _user_heap_stack : { . = . + _Min_Heap_Size + _Min_Stack_Size; }
                       > RAM /* fails the link if short */
}

```

The symbols `_sidata`, `_sdata`, `_edata`, `_sbss`, `_ebss` are not magic: the startup code in Chapter 6 reads exactly these to know what to copy and what to zero. `KEEP` protects the vector table from `--gc-sections`, which would otherwise discard it because nothing references it.

5.4 Reading a map file

The map file (`-Wl, -Map=fw.map`) answers 'why is my image this big' and 'what is at address 0x0800A31C'. Read it in three passes: the memory configuration at the top, the per-section placement in the middle, and the cross-reference table at the end.

```

Memory Configuration
Name      Origin      Length      Attributes
FLASH     0x08000000    0x00080000  xr
RAM        0x20000000    0x00020000  xrw

.text      0x08000190    0x9c14
.text.uart_send
            0x08002a10    0x84 build/uart.o
.text.sensor_task
            0x08002a94    0x1c8 build/sensor.o
*fill*     0x08009c5c    0x2
.bss       0x20000090    0x1a40
.bss.rx_ring
            0x20000090    0x400 build/uart.o
.bss.log_buf
            0x20000490    0x1000 build/log.o <-- 4 KB!

```

The `<-- 4 KB` moment is the point of the exercise: a single log buffer using a quarter of your RAM is invisible in source review and obvious here.

Question	Where to look
Total flash and RAM used	<code>arm-none-eabi-size fw.elf</code> : flash = text + data, RAM = data + bss (plus stack and heap)
Biggest functions	<code>nm --print-size --size-sort -r fw.elf</code>
Why is this library included?	The map's cross-reference: it names the symbol that pulled the archive member in
Did <code>--gc-sections</code> work?	Compare sizes with and without <code>-ffunction-sections -fdata-sections -Wl,--gc-sections</code>
What is at this crash address?	<code>addr2line</code> , or search the address ranges in the map

5.5 Flags that matter, and what they cost

Flag	Effect
<code>-Og -g3</code>	The debug build: optimised enough to run at speed, still steppable. Prefer it to <code>-O0</code> , which can be 5-10x slower and hide timing bugs
<code>-Os / -O2</code>	Release. <code>-Os</code> for flash-limited parts, <code>-O2</code> when speed matters. Always test at the release setting - <code>-O0</code> -only testing is how UB bugs reach the field
<code>-ffunction-sections</code> <code>-fdata-sections</code> <code>-Wl,--gc-sections</code>	Discard unused functions and data; typically 10-30% smaller
<code>-flto</code>	Whole-program optimisation; another 5-15%. Occasionally exposes latent UB and can confuse debug info
<code>-Wall -Wextra -Werror</code>	Non-negotiable. Add <code>-Wconversion</code> and <code>-Wdouble-promotion</code> on new code
<code>-fstack-usage</code>	Emits per-function stack usage - the input to the stack budget in Chapter 24
<code>--specs=nano.specs</code>	The small newlib: cuts <code>printf</code> and startup code substantially
<code>-fno-common</code>	Turns duplicate tentative definitions into link errors instead of silently merging them (default in modern GCC)
<code>-mcpu=cortex-m4</code> <code>-mfpv=fpv4-sp-d16</code> <code>-mfloat-abi=hard</code>	Get these wrong and floats go through software emulation - or link against incompatible libraries

MAKE THE BUILD REPRODUCIBLE AND SELF-IDENTIFYING

Embed the git hash, the build date and the version in a `const` struct at a fixed location, and print it at boot and in the crash log. When a field unit misbehaves, the first question is always 'which build is on it?' - and 'the one from Tuesday' is not an answer. Then make the build reproducible (`-ffile-prefix-map`, no `__DATE__` in code paths, pinned toolchain version) so that the same commit produces a byte-identical image on any machine.

Exercises

- 1 Run all five toolchain stages by hand on a single file and inspect each intermediate output.
- 2 Add `-Wl, -Map=fw.map` to your build and find the three largest functions and the three largest RAM objects.
- 3 Move a large table from `.data` to `.rodata` by adding `const`, and measure the RAM saved.
- 4 Turn `--gc-sections` on and off and record the size difference.
- 5 Delete the `KEEP` around `.isr_vector` in a scratch copy of your linker script and observe exactly how the resulting failure manifests.

6. From Reset to main(): The Boot Path

Between the moment power is stable and the first line of `main()`, a few hundred microseconds of code runs that most engineers never read. It sets the stack pointer, copies initialised data into RAM, zeroes the rest, starts the clocks, and enables the floating-point unit. When it is wrong, the symptom is a device that faults before any of your code runs - which is why this chapter exists.

6.1 What resets a microcontroller

Reset source	Meaning	What firmware should do
Power-on reset (POR)	Supply crossed the rising threshold	Full initialisation; RAM contents are undefined
Brown-out reset (BOR)	Supply dipped below a threshold	Log it - repeated BOR means a power design problem, not a firmware one
External NRST pin	Debugger, supervisor chip, or a button	Normal start
Watchdog reset	Firmware stopped servicing the watchdog	Log it prominently ; this is a defect until proven otherwise
Software reset	<code>NVIC_SystemReset()</code> after an update or a fault	Continue the intended flow (e.g. boot the new image)
Low-power exit	Wake from standby	Restore state; peripherals may need reconfiguring

LATCH THE RESET REASON IN THE FIRST TEN LINES

Read the reset-cause register, save it into a `.noinit` variable, clear the flags, and expose it in your log and telemetry. Devices that reset silently in the field are diagnosed by this one number more often than by any other single piece of data.

6.2 The vector table and the first two words

On Cortex-M, the hardware itself performs the first two steps of boot. It reads the 32-bit word at the vector table base into the main stack pointer, and the second word into the program counter. There is no boot loop in software before this - the silicon does it.

address	contents	loaded into
0x0800_0000	0x2002_0000 initial MSP	-> SP
0x0800_0004	0x0800_0191 Reset_Handler	-> PC (bit0 = Thumb)
0x0800_0008	NMI_Handler	
0x0800_000C	HardFault_Handler	
0x0800_0010	MemManage_Handler	
0x0800_0014	BusFault_Handler	
0x0800_0018	UsageFault_Handler	
...	SVC, PendSV, SysTick, then IRQ0..IRQn	

The initial stack pointer is data in flash, not code. If your linker script puts `_estack` outside RAM, the device faults on the first push - before a debugger can usefully stop it.

- Every entry is a **function address with bit 0 set**, because Cortex-M executes Thumb code only. The linker handles this; hand-written tables get it wrong.
- The table must be aligned to a power of two at least as large as the table itself (a 256-entry table needs 1024-byte alignment). Misalignment is a silent failure that shows as random handlers running.
- `VTOR (SCB->VTOR)` relocates the table - this is how a bootloader hands control to an application whose vectors are elsewhere (Chapter 31), and how a table can be copied to RAM to be patched at run time.

6.3 The reset handler, line by line

```
void Reset_Handler(void)
```

```

{
    /* 1. Copy .data from its load address in flash to RAM.          */
    uint32_t *src = &_sidata, *dst = &_sdata;
    while (dst < &_edata) { *dst++ = *src++; }

    /* 2. Zero .bss.                                                */
    for (dst = &_sbss; dst < &_ebss; ) { *dst++ = 0u; }

    /* 3. Chip-level init: clocks, flash wait states, FPU, cache. */
    SystemInit();

    /* 4. C++ static constructors and __attribute__((constructor)) */
    __libc_init_array();

    /* 5. Enter the application. It must never return.            */
    (void)main();
    for (;;) { }          /* if it does, do not fall off the end */
}

```

Fifteen lines that every firmware image runs. Steps 1 and 2 are the reason a global initialised to a non-zero value costs flash as well as RAM, and why a huge zero-initialised array costs boot **time** rather than image size.

HOW LONG DOES BOOT TAKE BEFORE MAIN()?

A device at 168 MHz (5.95 ns per cycle) with 8 KB of **.data** and 40 KB of **.bss**. The copy loop moves 2,048 words at roughly 4 cycles each: 8,192 cycles = **49 us**. The zero loop clears 10,240 words at about 2 cycles each: 20,480 cycles = **122 us**. Add crystal start-up (1-5 ms for a typical HSE) and PLL lock (about 200 us) and the picture is clear: **the oscillator dominates by an order of magnitude**. If you need a fast boot, start on the internal RC oscillator, do useful work, and switch to the crystal when it is ready - and shrink **.bss** before optimising the copy loop.

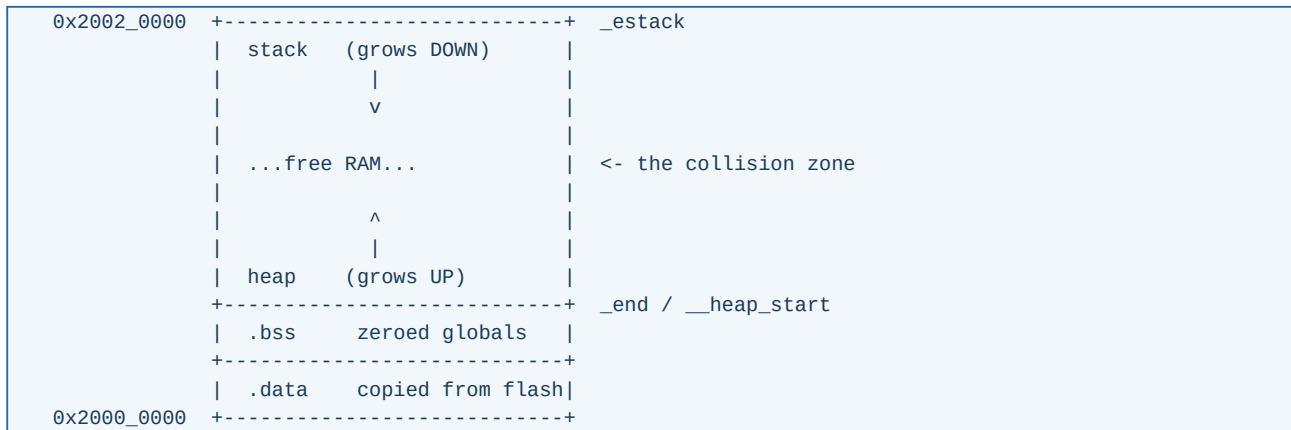
6.4 What SystemInit must do, in order

- **Raise flash wait states before raising the clock** - never the other way round, or the first instruction fetch at the new speed fails.
- Configure the voltage scaling / power mode the target frequency requires.
- Start and wait for the oscillator, configure the PLL, wait for lock, then switch the system clock source and **verify the switch took effect** by reading back the status bits.
- Enable the FPU if present: `SCB->CPACR |= (0xF << 20);`. Forgetting this gives a UsageFault on the first floating-point instruction, typically inside library code you did not know used floats.
- Enable caches and configure the MPU on parts that have them.
- Set `SCB->VTOR` if the table is not at the default address.

THE FIVE BOOT BUGS THAT ACCOUNT FOR MOST BRING-UP TIME

(1) Clock configured after flash wait states are already too low - instant hard fault. (2) **.data** copy omitted in a hand-written startup, so every initialised global reads as garbage. (3) Vector table misaligned or **VTOR** not set after a bootloader jump - the wrong handler runs. (4) FPU not enabled, faulting inside `printf`. (5) The stack overlapping **.bss** because the linker script's heap and stack reservation is missing - the failure looks like random memory corruption and is diagnosed in Chapter 24.

6.5 The RAM picture at run time



Nothing in the hardware prevents the stack from growing into the heap or into **.bss**. An MPU region placed just below the stack, or a painted guard band, turns that silent corruption into an immediate fault (Chapter 24).

6.6 Handing over: bootloader to application

```
static void jump_to_app(uint32_t app_base)
{
    uint32_t sp = *(volatile uint32_t *)(app_base);
    uint32_t pc = *(volatile uint32_t *)(app_base + 4u);

    __disable_irq();
    deinit_all_peripherals(); /* stop DMA, timers, clear NVIC */
    for (int i = 0; i < 8; i++) { NVIC->ICER[i] = 0xFFFFFFFFu;
                                NVIC->ICPR[i] = 0xFFFFFFFFu; }
    SysTick->CTRL = 0; SysTick->LOAD = 0; SysTick->VAL = 0;

    SCB->VTOR = app_base; /* application's vector table */
    __set_MSP(sp); /* application's stack pointer */
    __DSB(); __ISB();
    ((void (*)(void))pc)(); /* never returns */
}
```

Every line here exists because of a bug someone shipped: a live DMA channel writing into the new image's RAM, a pending interrupt firing into a vector table that no longer exists, or a SysTick left running with a handler the application has not installed yet.

Exercises

- 1 Read your project's startup file end to end and map every line to a step above.
- 2 Break at **Reset_Handler**, single-step through the **.data** copy, and watch a known global change from garbage to its initialiser.
- 3 Add a 32 KB zero-initialised array and measure the change in boot time and in image size. Explain both results.
- 4 Deliberately set **__estack** 16 bytes above the top of RAM and observe the failure.
- 5 Latch and print the reset cause, then trigger each reset source in turn and confirm you can tell them apart.

PART II

The Hardware Interface

Registers, clocks, GPIO, interrupts, timers, analogue conversion, the serial protocols, DMA and non-volatile storage - what each peripheral does, how to drive it correctly, and the failure mode it is famous for.

7. Registers and Memory-Mapped I/O

Every peripheral is a set of registers at fixed addresses. Driving hardware is reading and writing those addresses in the right order, with the right widths, while other agents - interrupts, DMA, other cores - are doing the same. This chapter is the grammar of that conversation.

7.1 Registers are not memory

Behaviour	What it means	Consequence
Read-only	Writes are ignored	Harmless but hides bugs
Write-only	Reads return garbage or zero	Never read-modify-write these; keep a shadow copy in RAM
Read-to-clear (rc_r)	The act of reading clears the flag	A debugger's register view can consume your interrupt flag - a genuinely confusing bug
Write-1-to-clear (w1c)	Writing 1 clears; writing 0 does nothing	<code>SR &= ~FLAG;</code> is wrong - it clears every other pending flag too. Write <code>SR = FLAG;</code>
Set/clear register pairs	Separate addresses to set and clear bits	Atomic without a critical section - always prefer them
Reserved bits	Undefined on read; must be preserved on write	Read-modify-write, or use the vendor's documented reset value
Width-restricted	Some registers accept only 32-bit access	A byte write to a word-only register silently does nothing or faults

THE READ-MODIFY-WRITE RACE, AND ITS CURE

`GPIOA->ODR |= (1u << 5);` is three operations. If an interrupt toggles pin 3 between the read and the write, that change is lost. Three cures, in order of preference: (1) use the hardware's atomic set/reset register - on STM32, `GPIOA->BSRR = (1u << 5);` to set and `GPIOA->BSRR = (1u << 21);` to clear, one write, no race; (2) use bit-banding on Cortex-M3/M4, where each bit has its own word address; (3) wrap the sequence in a critical section (Chapter 9), which costs interrupt latency.

```
/* Atomic pin control - no read-modify-write anywhere. */
static inline void pin_set(GPIO_TypeDef *port, uint32_t pin)
{ port->BSRR = (1u << pin); }
static inline void pin_clear(GPIO_TypeDef *port, uint32_t pin)
{ port->BSRR = (1u << (pin + 16u)); }
static inline bool pin_read(GPIO_TypeDef *port, uint32_t pin)
{ return (port->IDR & (1u << pin)) != 0u; }
```

7.2 Ordering: the compiler, the write buffer and the bus

Three independent things can reorder or delay your accesses. `volatile` handles only the first.

Layer	Reorders?	Control
Compiler	Yes, freely, for non-volatile accesses	<code>volatile</code> , and a compiler barrier
Store buffer / bus	A write may complete after the next instruction	<code>__DSB()</code> - wait for the write to reach memory
Pipeline / caches	An instruction fetch may be stale after self-modifying or remapping	<code>__ISB()</code> after <code>VTOR</code> , MPU or control-register changes
Two masters (CPU + DMA)	Independent orders entirely	<code>__DMB()</code> between the data write and the flag write

```

/* The classic: clearing an interrupt flag at the end of an ISR. */
void TIM2_IRQHandler(void)
{
    TIM2->SR = ~TIM_SR_UIF;      /* w1c: clear the update flag */
    (void)TIM2->SR;              /* read back forces it to land */
    __DSB();                    /* ...or an explicit barrier */
} /* Without one of these, the ISR can be re-entered immediately */
/* because the flag was still set when the core returned. */

```

7.3 Two things to do before touching any peripheral

- **Enable its clock**, then read the enable register back. On many STM32 parts the clock takes a couple of cycles to arrive, and a register write in the instruction immediately after the enable is lost - a documented erratum that costs new engineers an afternoon each.
- **Release it from reset**, if the family gates peripherals through a reset register as well. Toggling reset is also the reliable way to return a wedged peripheral (a stuck I2C, a confused DMA channel) to a known state without rebooting.

7.4 How much abstraction to buy

Layer	Pros	Cons	When
Raw registers / CMSIS structs	Exact, small, fast, no surprises	Verbose; you own the datasheet reading	Interrupt paths, timing-critical code, and anything you will certify
Vendor LL (low-layer) drivers	Thin inline wrappers; still explicit	Vendor-specific naming	A good default for most peripherals
Vendor HAL	Fast to first light; handles errata	Large, blocking APIs, hidden state, hard to unit test	Prototyping, complex peripherals (USB, Ethernet) where writing your own is not justified
Portable framework (Zephyr, Arduino, mbed)	Portable across silicon; drivers and stacks included	Framework's model may not match your timing needs	Multi-vendor products and teams that value portability over control

THE SEAM THAT MAKES DRIVERS TESTABLE

Whatever layer you choose, put register access behind a handful of small inline functions in one file per peripheral. On the target they compile to a single store. On a host, that file is replaced by a fake that records the writes - which is what makes the unit tests of Chapter 26 possible without any hardware. The cost is zero; the benefit is that your driver logic becomes testable code rather than an untestable mixture of logic and addresses.

7.5 Bit-banding, shadow registers and other access patterns

Three patterns recur whenever register access gets awkward, and knowing them saves inventing something worse.

Bit-banding (Cortex-M3/M4 only): each bit gets its own word address.

$$\text{bit_word} = \text{bit_band_base} + (\text{byte_offset} * 32) + (\text{bit} * 4)$$

SRAM : base 0x22000000, region starts at 0x20000000

Peripheral: base 0x42000000, region starts at 0x40000000

Example: bit 5 of the byte at 0x20000300

$$0x22000000 + (0x300 * 32) + (5 * 4) = 0x22006014$$

Writing 1 to that word sets exactly that bit, atomically,
with no read-modify-write.

Bit-banding does not exist on Cortex-M0/M0+ or on most ARMv8-M parts, so treat it as an optimisation, not an architecture.

- **Shadow registers.** For a write-only register, keep the intended value in RAM, modify that, and write the whole word. This is the only correct way to do a partial update of a register you cannot read - and the shadow is also what lets you restore the peripheral after a low-power mode that loses its state.
- **Register maps as `const` tables.** Configuration that differs between product variants belongs in a table of (offset, value) pairs applied by a loop, not in variant-specific `#ifdef` code. It compresses well, is easy to review against the datasheet, and can be verified by reading back.
- **Read-back verification** at initialisation. After configuring a peripheral, read the registers back and compare with what you wrote. It costs microseconds once, and it catches a missing clock, a locked-out register, a write to a reserved bit and a wrong base address - four bugs that otherwise look identical from the application's point of view.

```
/* Applying and then verifying a register table. */
typedef struct { volatile uint32_t *reg; uint32_t val; } regcfg_t;

static const regcfg_t uart_cfg[] = {
    { &UART1->BRR, 0x0683u }, /* 115200 @ 16 MHz */
    { &UART1->CR2, 0x0000u }, /* 1 stop bit */
    { &UART1->CR1, 0x200Cu }, /* UE | TE | RE */
};

bool uart_apply(void)
{
    for (size_t i = 0; i < ARRAY_SIZE(uart_cfg); i++)
        *uart_cfg[i].reg = uart_cfg[i].val;
    for (size_t i = 0; i < ARRAY_SIZE(uart_cfg); i++)
        if (*uart_cfg[i].reg != uart_cfg[i].val) return false;
    return true; /* verified */
}
```

Note the verification loop is separate: some registers only take effect once a later one is written, so verify after the whole sequence rather than field by field.

7.6 Reading a datasheet register description

What the manual says	What it means for your code
'This bit is set by hardware and cleared by software by writing 1'	Write-1-to-clear: <code>SR = FLAG;</code> never <code>SR &= ~FLAG;</code>
'This bit must be written to 0'	A reserved bit: mask it out of any value you compute
'Writing this register while EN=1 has no effect'	Disable the peripheral, reconfigure, re-enable - and the disable may need a wait
'This register is protected; write the key sequence first'	A lock, common on watchdogs, RTCs and clock control; the sequence is exact

What the manual says	What it means for your code
'The bit is cleared by reading SR followed by reading DR'	The clear is a sequence ; a debugger reading SR can consume it
'Access must be 32-bit'	A <code>uint8_t</code> write to that address is silently dropped or faults
'Values are latched at the next update event'	The register is shadowed: your write applies at the next period, which is a feature (glitch-free updates), not a delay bug

Exercises

- 1 Find one `SR &= ~FLAG` in any codebase you have access to and work out which flags it destroys.
- 2 Toggle a pin with `ODR ^=` and with `BSRR` in a tight loop and compare both the disassembly and the maximum frequency achieved.
- 3 Remove the read-back after an interrupt-flag clear and observe the double-entry behaviour on a scope by toggling a second pin at ISR entry.
- 4 Write the minimal register-level GPIO driver for your part without using the vendor HAL, then diff its size against the HAL version.

8. Clocks, Power Domains and Pins

Two questions explain most 'the peripheral does nothing' tickets: is it clocked, and is the pin configured? This chapter answers both properly - the clock tree from oscillator to peripheral, the power domains underneath it, and the electrical reality of a pin.

8.1 Where clocks come from

Source	Typical accuracy	Start-up	Use
Internal RC (HSI)	+/-1% over temperature, +/-2% worst case	A few microseconds	Boot, and anything not timing-critical
Crystal (HSE)	10-50 ppm	1-5 ms	UART at high rates, USB, CAN, RF, real timekeeping
Ceramic resonator	0.1-0.5%	Under 1 ms	Cheaper than a crystal; adequate for UART, not for USB
TCXO	0.5-2 ppm	1-10 ms	Cellular, GNSS, precision timing
Low-speed RC (LSI)	+/-5-10%	Fast	Watchdog, coarse wake-up timing
32.768 kHz crystal (LSE)	20 ppm (about 1 min/month)	0.5-2 s	RTC, calendar, low-power tick

MATCH THE CLOCK TO THE PROTOCOL'S TOLERANCE

UART tolerates roughly +/-2% total error between the two ends, so an internal RC **can** work but leaves no margin over temperature. CAN needs about +/-0.5%, and USB full-speed needs +/-0.25% - neither is achievable with an internal RC unless the part has a crystal-less USB mode that trims the RC against the host's SOF packets. Deciding 'crystal or not' is therefore a firmware decision made at schematic review, and getting it wrong costs a board spin.

8.2 The clock tree, and the PLL arithmetic

```

HSE 8 MHz ---+
              |
              +-----+   +-----+   +-----+
HSI 16 MHz --+---> | /M | -> | xN | -+> | /P | --> SYSClk 168 MHz
              +-----+   +-----+   +-----+
              VCO in      VCO      | +-----+
              1-2 MHz    100-432  +>| /Q | --> 48 MHz USB/SDIO
                      +-----+
SYSClk -> [AHB /1 ] -> HCLK 168 MHz -> core, memory, DMA
                      -> [APB1 /4] -> PCLK1 42 MHz (timers x2 = 84)
                      -> [APB2 /2] -> PCLK2 84 MHz (timers x2 =168)

```

A representative Cortex-M4 clock tree. Every peripheral hangs off one of these buses, and its register description states which.

DERIVING A PLL CONFIGURATION

Target: 168 MHz system clock and exactly 48 MHz for USB, from an 8 MHz crystal. The PLL input must land in 1-2 MHz, so $M = 8$ gives 1 MHz. The VCO must land in 100-432 MHz and must be divisible for both outputs, so $N = 336$ gives a 336 MHz VCO. Then $P = 2$ gives $336/2 = \mathbf{168\text{ MHz}}$ for the core, and $Q = 7$ gives $336/7 = \mathbf{48.0\text{ MHz}}$ exactly for USB. Now the buses: AHB = 168 MHz; APB1 has a 42 MHz limit so the prescaler must be 4 (42 MHz); APB2's limit is 84 MHz so its prescaler is 2. **The timer trap:** when an APB prescaler is not 1, the timers on that bus are clocked at twice the bus clock - so APB1 timers run at 84 MHz, not 42 MHz. Getting this wrong makes every period you compute exactly half or double what you intended, which is the single most common timer bug.

The safe switching sequence is fixed, and each step exists because skipping it hangs the part:

- Set the voltage scale / power mode for the target frequency.

- **Raise the flash wait states first** and read the register back.
- Enable HSE, poll **HSERDY with a timeout**, and have a fallback to HSI - a cracked crystal must not produce a dead product.
- Configure the PLL only while it is off; enable it; poll **PLLRDY**.
- Set the AHB/APB prescalers **before** switching the source, so no bus is ever briefly overclocked.
- Switch **SW** to PLL and poll **SWS** until it reports PLL. Then update **SystemCoreClock** - every delay and baud calculation depends on it.

PROVE THE CLOCK INSTEAD OF ASSUMING IT

Route the system clock to the MCO pin and measure it, or toggle a GPIO in a known loop and check the frequency. Ten minutes here prevents a week of chasing 'the UART is garbage and the timer is half speed', which is one root cause with two symptoms.

8.3 Power domains and what stays alive

Domain	Contents	Notes
Core (VDD)	CPU, flash, most peripherals	Off in standby; RAM may or may not be retained
Backup (VBAT)	RTC, backup registers, LSE	Runs from a coin cell when VDD is gone - the place to keep a few bytes across a power cut
Analogue (VDDA)	ADC, DAC, reference	Deserves its own filtering; ADC noise is usually a VDDA problem
I/O	Pin drivers	Some parts keep pin states latched in standby, which matters for external circuits

Regulators matter to firmware in one specific way: an SMPS is efficient but noisy, an LDO is quiet but wastes the voltage difference as heat, and some parts let firmware choose. If your ADC readings are noisy at exactly the switching frequency, that choice is the reason.

8.4 Configuring a pin, completely

Setting	Options	Consequence of getting it wrong
Mode	Input / output / alternate function / analogue	The peripheral is connected internally but the pin is not
Alternate function number	AF0-AF15, per pin, from a table	The pin is muxed to the wrong peripheral - silent failure
Output type	Push-pull or open-drain	I2C must be open-drain; a push-pull output fighting another driver damages both
Pull resistors	Up / down / none (typically 30-50 kOhm)	A floating input oscillates and burns power; a bus without a pull-up never idles high
Speed / slew rate	Low to very high	Too slow corrupts fast signals; too fast radiates EMI and rings
Drive strength	A few mA per pin, with a total per port	Driving an LED directly at 20 mA can exceed the port limit

THE PIN CHECKLIST THAT SOLVES 'MY PERIPHERAL IS DEAD'

(1) Peripheral clock enabled? (2) GPIO port clock enabled? (3) Mode set to alternate function, not output? (4) **Correct AF number for that specific pin** - the table differs per pin, not per port. (5) Open-drain where the bus requires it, with pull-ups fitted? (6) Is the pin also a boot pin, an oscillator pin, or a debug pin (PA13/PA14 on SWD) that you have just disabled your own debugger with? That last one has cost more evenings than any other item on the list.

8.5 The electrical facts firmware must respect

- **Logic thresholds.** A 3.3 V part typically reads above about 2.0 V as high and below 0.8 V as low; between them the input is undefined and may oscillate. Slow-changing signals need a Schmitt-trigger input or hardware hysteresis.
- **5 V tolerance is per-pin**, and often lost when the pin is configured as analogue. Check the datasheet's pin table, not the family's marketing.
- **Series resistors** (22-100 ohm) on fast outputs tame ringing and EMI at no firmware cost.
- **Contact bounce** is real: a mechanical switch chatters for 1-20 ms. Debounce in firmware (sample every 5 ms and require N consistent samples) or in hardware with an RC filter.
- **Inrush and brownout.** Turning on a motor or a radio can dip the supply enough to reset the MCU. If the device resets whenever a load switches, the bug is in the power design, and firmware's job is to stagger the loads and log the brownout.

SIZING AN RC DEBOUNCE FILTER

A 10 kOhm resistor with a 100 nF capacitor gives a time constant of $\tau = RC = 10,000 \times 100e-9 = 1 \text{ ms}$. The input crosses the $\sim 2.0 \text{ V}$ threshold from 0 V after $t = -\tau \ln(1 - 2.0/3.3) = 1 \text{ ms} \times 0.96 = \text{about } 0.96 \text{ ms}$, so bounce shorter than a millisecond is absorbed. For a 20 ms bouncy switch, either scale the filter to $\tau = 10 \text{ ms}$ (which slows the response noticeably) or - better - filter in firmware: sample at 5 ms and require three identical samples, giving a 15 ms debounce that costs nothing and is adjustable in a field update.

Exercises

- 1 Derive a PLL configuration for your board's crystal that gives the maximum core frequency and an exact 48 MHz, and verify both on the MCO pin.
- 2 Compute your timers' actual input clock, including the APB doubling rule, and confirm it by generating a 1 Hz square wave.
- 3 Configure one pin in all four modes in turn and observe each with a meter and a scope, including a floating input.
- 4 Implement software debouncing for a real button and measure the bounce with a logic analyser before and after.
- 5 Deliberately raise the core clock without changing flash wait states and observe the failure, then fix the ordering.

9. Interrupts and Exceptions

Interrupts are what make a microcontroller responsive and what make firmware concurrent. Everything difficult about firmware - races, priority inversion, jitter, corrupted state - enters through this chapter. It is the most important chapter in Part II.

9.1 The exception model

An exception is any event that diverts the core from the current instruction stream. Faults, the SysTick timer, the RTOS's PendSV, and every peripheral interrupt are all exceptions, numbered and dispatched through the vector table of Chapter 6.

Exception	Number	Priority	Purpose
Reset	1	-3, fixed	Boot
NMI	2	-2, fixed	Non-maskable: clock failure, watchdog pre-warning
HardFault	3	-1, fixed	Catch-all, and escalation target
MemManage / BusFault / UsageFault	4-6	Configurable	MPU violation, bus error, illegal instruction or state
SVCall	11	Configurable	Supervisor call - the RTOS's entry to privileged code
PendSV	14	Usually lowest	Deferred context switch (Chapter 21)
SysTick	15	Configurable	The periodic tick
IRQ0..IRQn	16+	Configurable	Peripherals: timers, UART, DMA, EXTI

9.2 Priorities: the number that means the opposite

LOWER NUMBER MEANS HIGHER PRIORITY

Priority 0 preempts priority 1. Every engineer inverts this at least once, and the resulting bug - a critical interrupt that never preempts - is invisible in code review. Write a comment next to every `NVIC_SetPriority` call stating the intent in words, not numbers.

Cortex-M implements only the **upper** bits of the 8-bit priority field; a typical part implements 4, giving 16 levels. Those bits are then split by the priority grouping setting into preemption priority and sub-priority.

4 implemented bits, PRIGROUP = 3 -> 4 preempt bits, 0 sub bits

16 preemption levels: any lower number preempts any higher

4 implemented bits, PRIGROUP = 5 -> 2 preempt bits, 2 sub bits

4 preemption levels; within a level, sub-priority only decides

which of two **SIMULTANEOUSLY PENDING** interrupts runs first -

it never causes preemption.

Encoded value written to `NVIC_IPR` = `priority << (8 - implemented)`

If you use an RTOS, it will require a specific grouping (FreeRTOS expects all bits to be preemption bits) - set it once, at boot, and never change it.

Interrupt	Suggested priority	Reason
Motor control / safety shutdown	0-1	Must preempt everything; must never call RTOS APIs
High-rate DMA completion, timer capture	2-4	Short, frequent, latency-sensitive
UART/SPI/I2C transfer complete	5-8	Short work, deferred processing
RTOS SysTick and PendSV	Lowest configurable	The scheduler must not preempt real-time work
Buttons, housekeeping, LEDs	Lowest	Nothing depends on them

9.3 What the hardware does on entry and exit

On exception entry the core AUTOMATICALLY pushes 8 words:

```

SP+0x1C  xPSR      <- highest address
SP+0x18  PC        (the instruction that was about to run)
SP+0x14  LR
SP+0x10  R12
SP+0x0C  R3
SP+0x08  R2
SP+0x04  R1
SP+0x00  R0        <- SP points here inside the handler

```

(+ 18 more words if the FPU context is stacked - see lazy stacking)
 LR is set to EXC_RETURN, e.g. 0xFFFFFFF0 = return to thread mode, using the PSP, no FPU state. Handlers are ordinary C functions because the hardware saved exactly the caller-saved registers.

This stack frame is also the evidence in a crash: Chapter 25 decodes a real one to find the faulting instruction.

Mechanism	What it does	Why you care
Tail-chaining	Two back-to-back interrupts skip the unstack/restack (saves ~6 cycles)	Back-to-back ISRs are cheaper than they look
Late arrival	A higher-priority interrupt arriving during entry is serviced first	Priority is honoured even mid-entry
Lazy FPU stacking	FPU registers are reserved but only saved if the handler uses the FPU	Using a float in an ISR adds 17 words and dozens of cycles - avoid it in fast handlers
Automatic stacking	8 words always	Every nested interrupt costs at least 32 bytes of stack - the input to Chapter 24's budget

AN INTERRUPT LATENCY BUDGET

Requirement: respond to an external edge within 5 us at 168 MHz (5.95 ns per cycle, so 5 us = 840 cycles). Costs: up to 12 cycles of entry latency; up to the longest critical section during which interrupts are disabled - say a 200-cycle section elsewhere in the code; plus, if a higher-priority ISR is running, its full duration - say 300 cycles. Worst case so far: 12 + 200 + 300 = 512 cycles = **3.05 us**, leaving 328 cycles (1.95 us) for the handler itself. The lesson is arithmetic, not opinion: **the length of your longest interrupt-disabled region is part of every other interrupt's deadline**, which is why critical sections are measured in instructions, not lines.

9.4 Masking and critical sections

```

/* Correct nesting: save and restore, never blind enable. */
static inline uint32_t irq_lock(void)
{
    uint32_t primask = __get_PRIMASK();
    __disable_irq();
    return primask;
}
static inline void irq_unlock(uint32_t primask)
{
    __set_PRIMASK(primask);    /* restores, does not force enable */
}

uint32_t key = irq_lock();
shared_counter++;             /* the critical section */
irq_unlock(key);

```

A blind `__enable_irq()` at the end of a critical section re-enables interrupts that the caller had deliberately disabled - a bug that appears only when the function is called from inside another critical section.

BASEPRI is the better tool when some interrupts must never be delayed: writing a priority level to it blocks everything at or below that level while leaving higher-priority interrupts - a motor fault, a safety input - fully responsive. This is exactly how an RTOS protects its own data structures without adding latency to the interrupts it does not manage.

9.5 Writing a handler that will not haunt you

ISR rules

- ☐ Short: tens of microseconds, not milliseconds. Move work to the main loop or a task.
- ☐ Never block: no busy-wait on a peripheral flag, no mutex, no delay.
- ☐ No `printf`, no `malloc`, no floating point in fast handlers.
- ☐ Clear the interrupt flag - and read it back or use a barrier before returning.
- ☐ Touch shared state only through an agreed mechanism (Chapter 18).
- ☐ Keep the stack cost in mind: every nesting level costs at least 32 bytes.
- ☐ If using an RTOS, call only the `...FromISR` variants, and only from interrupts at or below the RTOS's maximum syscall priority.

```

/* Deferred processing: the ISR captures, the task decides. */
void USART2_IRQHandler(void)
{
    if (USART2->SR & USART_SR_RXNE) {
        uint8_t byte = (uint8_t)USART2->DR;    /* read clears RXNE */
        (void)ring_push(&rx_ring, byte);      /* lock-free, Ch 18 */
    }
    if (USART2->SR & USART_SR_ORE) {            /* overrun          */
        (void)USART2->DR;                        /* documented clear */
        rx_overruns++;                          /* count, do not hide */
    }
}

```

Note the overrun branch. Silently discarding an overrun turns a diagnosable 'we are too slow' into an unexplained data corruption months later; counting it makes the problem visible in telemetry.

9.6 Faults: the interrupts you did not ask for

Fault	Typical cause
HardFault	Escalation from any other fault, or a fault in a handler; also a bad vector table entry
MemManage	MPU violation - including a stack guard region doing its job

Fault	Typical cause
BusFault	Access to an unmapped address, or a peripheral whose clock is off - the single most common cause
UsageFault	Undefined instruction, illegal state (PC bit 0 clear), divide by zero if enabled, unaligned access if enforced
Imprecise BusFault	A buffered write failed after the pipeline moved on; set <code>SCB->ACTLR DISDEFWBUF</code> to make it precise while debugging

INSTALL A REAL FAULT HANDLER ON DAY ONE

A `while(1)` fault handler tells you nothing. A handler that captures the stacked PC and LR, the CFSR/HFSR fault status registers and MMFAR/BFAR into `.noinit` memory, then resets, turns every future crash - including one from the field - into an address you can feed to `addr2line`. Chapter 25 decodes such a frame by hand; Chapter 28 turns it into a field crash report.

Exercises

- 1 Set two interrupts to different priorities and prove preemption with a GPIO toggled at entry and exit of each, viewed on an analyser.
- 2 Measure your own interrupt latency: toggle a pin in hardware, capture it in the ISR, and read the difference on a scope.
- 3 Write the `irq_lock/irq_unlock` pair and audit your codebase for blind `__enable_irq()` calls.
- 4 Deliberately cause each of the four fault types and confirm your fault handler distinguishes them.
- 5 Find the longest interrupt-disabled region in your code by instrumenting `irq_lock/irq_unlock` with the DWT cycle counter and recording the maximum.

10. Timers, PWM and Keeping Time

A timer is a counter with opinions: it counts a clock, wraps at a value you choose, and can raise an interrupt, drive a pin, or capture the moment an input changed. Almost every periodic behaviour in firmware - control loops, PWM, timeouts, the RTOS tick, sampling - is a timer.

10.1 Why the delay loop in Chapter 1 was wrong

- It is **uncalibrated**: its duration depends on the compiler, the optimisation level, flash wait states and the core clock, all of which change.
- It is **blocking**: the CPU does nothing else and cannot sleep, which costs the entire power budget of Chapter 29.
- It is **fragile**: without `volatile` the optimiser deletes it entirely, and with `volatile` it still varies by a factor of two across builds.

The replacement is a hardware timebase: a free-running counter you compare against, so that waiting costs nothing and timing does not depend on code generation.

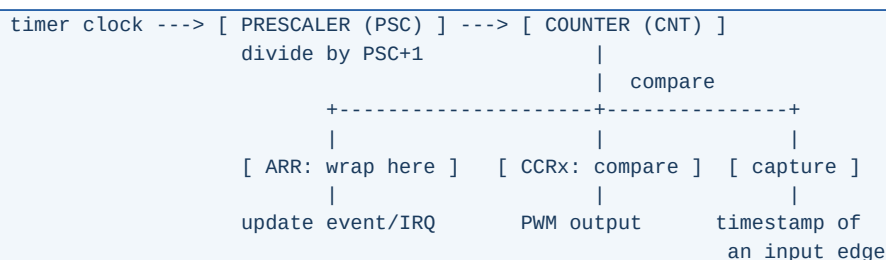
```
volatile uint32_t g_ms;           /* incremented by SysTick */
void SysTick_Handler(void) { g_ms++; }

/* Overflow-safe: works across the 49.7-day wrap of a 32-bit ms */
/* counter, because the subtraction wraps consistently. */
static inline bool time_after(uint32_t a, uint32_t b)
{
    return (int32_t)(a - b) > 0;
}

uint32_t deadline = g_ms + 250u;
while (!time_after(g_ms, deadline)) {
    do_other_work();              /* or __WFI() to sleep */
}
```

`if (g_ms > deadline)` is the bug this idiom exists to prevent: it fails for 49.7 days every 49.7 days, which is exactly the kind of defect that survives testing and appears in the field.

10.2 Timer anatomy



$$\text{Period} = (\text{PSC} + 1) * (\text{ARR} + 1) / f_{\text{timer}} \quad \leftarrow \text{the +1s are real}$$

Both the prescaler and the auto-reload are 'minus one' values. Almost every timer that is 0.1% off is missing one of these +1 terms.

TWO CONFIGURATIONS, COMPUTED

Timer clocked at 84 MHz. **(a) 20 kHz PWM at maximum resolution:** with $\text{PSC} = 0$ the counter runs at 84 MHz, so $\text{ARR} + 1 = 84\text{e}6 / 20\text{e}3 = 4200$, giving $\text{ARR} = 4199$ and 4200 duty steps - $\log_2(4200) = 12.04$ bits of resolution. A duty cycle of 25% is $\text{CCR} = 1050$. **(b) A 1 Hz interrupt:** $84\text{e}6$ counts is more than a 16-bit ARR can hold, so prescale first: $\text{PSC} = 8399$ divides by 8400 to give 10 kHz, then $\text{ARR} = 9999$ gives exactly 1 Hz. **The trade is fixed:** frequency resolution and duty resolution both come out of the same counter, so raising the PWM frequency costs duty-cycle steps one for one.

10.3 What the modes are for

Mode	What it does	Typical use
Output compare / PWM	Drives a pin when CNT reaches CCRx	Motor drive, LED dimming, heaters, tone generation
Centre-aligned PWM with dead time	Symmetric counting plus a forced gap between complementary outputs	Half-bridge motor and power control, where both switches on means a destroyed board
Input capture	Records CNT when an edge arrives	Frequency, pulse width, ultrasonic time-of-flight, RC receivers
Encoder mode	Counts quadrature A/B edges in hardware	Position feedback with zero CPU cost
One-pulse mode	A single pulse of defined delay and width	Triggering a sensor, a camera flash, a sample-and-hold
Trigger / master-slave	One timer starts, gates or clocks another	Jitter-free ADC sampling, synchronised multi-phase PWM
DMA burst (DMAR)	Rewrites several timer registers per update	Waveform tables, addressable LED protocols

MEASURE A PULSE WIDTH CORRECTLY

Capture on both edges into a circular DMA buffer and subtract consecutive timestamps in software, handling counter wrap by unsigned subtraction. Doing the subtraction in the ISR at high rates is where measurements are lost; letting DMA capture and processing them in a task is what makes a 500 kHz edge rate survivable.

10.4 Software timers on top of hardware

You will have more timed events than the chip has timers. The standard solution is one hardware timebase plus a list of software timers, which is what every RTOS's timer service is.

```
typedef struct { uint32_t due; uint32_t period; void (*cb)(void);
                bool active; } swtimer_t;

void swtimer_poll(uint32_t now)          /* call from the main loop */
{
    for (size_t i = 0; i < N_TIMERS; i++) {
        swtimer_t *t = &timers[i];
        if (t->active && time_after(now, t->due)) {
            t->due = (t->period != 0u) ? t->due + t->period : t->due;
            t->active = (t->period != 0u);
            t->cb();                /* runs in task context */
        }
    }
}
```

Note `t->due += t->period` rather than `now + period`: advancing from the scheduled time rather than the actual time prevents drift accumulating when service is late.

10.5 Clocks, calendars and drift

Concept	Definition	Rule
Monotonic time	Ticks since boot; never jumps	Use this for every timeout, timer and interval
Wall-clock time	Calendar date and time from an RTC or a server	Use only for logging and display; it can jump backwards
Drift	Accumulated error from oscillator inaccuracy	20 ppm = 1.7 s/day; 100 ppm = 8.6 s/day

Concept	Definition	Rule
Jitter	Variation in the interval between events	Caused by interrupt latency; measure it, do not assume it
Synchronisation	Correcting the local clock from a reference	Slew (adjust the rate) rather than step, or timestamps go backwards mid-log

HOW ACCURATE MUST THE CRYSTAL BE?

A data logger must timestamp within 1 second per day. A day is 86,400 s, so the required accuracy is $1/86,400 = 11.6$ ppm - **a standard 20 ppm watch crystal is not good enough** without correction, and the internal RC at 1% (10,000 ppm, i.e. 14 minutes per day) is off by three orders of magnitude. The options are a better crystal, temperature compensation using the on-chip sensor, or a periodic sync from the network - and only the last one keeps working after five years of ageing.

10.6 Watchdogs

A watchdog is a timer that resets the device unless firmware periodically proves it is alive. Two kinds matter: the **independent** watchdog, on its own low-speed oscillator, which survives a main clock failure, and the **window** watchdog, which faults if you feed it too early as well as too late - catching a runaway loop that is feeding it in a tight cycle.

Watchdog discipline

- ☐ Exactly one place in the code feeds the watchdog.
- ☐ That place feeds it only after confirming every critical task has checked in since the last feed (a bitmask of task heartbeats).
- ☐ Never feed it from an interrupt: an ISR that keeps running while the main loop is dead defeats the entire mechanism.
- ☐ The timeout is longer than the slowest legitimate operation, including flash erase.
- ☐ Watchdog resets are counted and logged - Chapter 28 makes them visible.
- ☐ It is enabled in production builds. A watchdog disabled 'for debugging' and never re-enabled is a classic field failure.

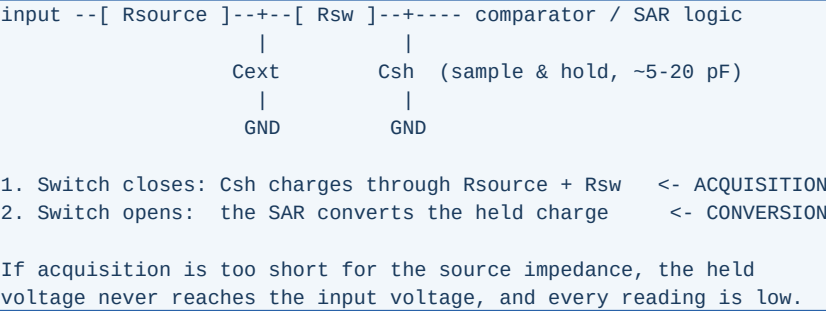
Exercises

- 1 Configure a timer for exactly 1 kHz and verify it on a scope; then compute what ARR value a 0.1% error corresponds to.
- 2 Generate 20 kHz PWM and sweep the duty from 0 to 100%, checking the extremes behave (0% and 100% are the cases that break).
- 3 Implement `time_after` and write a test that proves it works across the 32-bit wrap.
- 4 Use input capture to measure the frequency of a signal from another timer, and compare with the expected value.
- 5 Enable the independent watchdog, then deliberately hang in a loop and confirm the reset - and that your reset-cause logging reports it.

11. Analogue: ADC, DAC and Real Sensors

Analogue conversion is where firmware meets physics, and where plausible-looking numbers can be entirely wrong. The peripheral is easy to start and hard to trust: this chapter is about the difference.

11.1 How a SAR ADC actually samples



The single most common ADC bug is an acquisition time chosen from the example code rather than from the source impedance.

ACQUISITION TIME FROM SOURCE IMPEDANCE

To settle to within half an LSB of 12 bits, the RC network must settle to $1/2^{13}$, which needs $t = RC \times \ln(2^{13}) = 9.01 RC$. With a 10 kOhm source, 1 kOhm switch resistance and a 12 pF hold capacitor, $RC = 11,000 \times 12e-12 = 132$ ns, so $t = \mathbf{1.19\ us}$. At an ADC clock of 21 MHz (47.6 ns per cycle) that is 25 cycles, so a 28-cycle sampling setting is the smallest safe choice. **Halve the source impedance or double the sampling time** - and if the source is a 100 kOhm divider, buffer it with an op-amp, because 12 us of acquisition per channel destroys your sample rate.

11.2 Resolution, noise and what you really get

LSB

= V_ref / 2^N

12-bit at 3.300 V -> 0.806 mV

SNR_ideal

= 6.02 N + 1.76 dB

12-bit -> 74 dB

ENOB

= (SNR_measured - 1.76) / 6.02

Oversampling: averaging 4^k samples of a signal with white noise

gains k bits: 16x -> +2 bits, 256x -> +4 bits

(decimate by shifting right k, not by dividing)

The nominal resolution is a property of the datasheet; the effective resolution is a property of your board.

Oversampling only helps if there is at least one LSB of noise to dither the input - a perfectly quiet signal sitting between two codes reads the same code every time, and averaging 256 identical numbers yields no new information. In practice there is always enough noise, which is one of the few places where noise is a friend.

Error	Cause	Fix
Offset	Comparator and reference offsets	Measure at a known 0 and subtract
Gain	Reference voltage inaccuracy	Two-point calibration, or measure the internal reference
INL / DNL	Non-ideal internal capacitor matching	Rarely fixable; choose a better part if it matters
Temperature drift	Reference and offset move with temperature	Calibrate over temperature, or use a ratiometric measurement
Crosstalk between channels	Insufficient acquisition after the mux switches	Increase sampling time; interleave a dummy conversion

Error	Cause	Fix
Supply noise	Switching regulators, digital activity on VDDA	Filter VDDA, sample away from switching edges, average

RATIOMETRIC MEASUREMENT REMOVES THE REFERENCE FROM THE ANSWER

If a sensor is a resistive divider powered from the same rail that supplies the ADC reference, then the ratio of code to full scale depends only on the divider - the supply voltage cancels exactly. Wiring a potentiometer, a thermistor divider or a bridge this way makes reference drift irrelevant, and is free. Do it whenever the measurement is of a ratio rather than an absolute voltage.

11.3 Sampling correctly

- **Trigger conversions from a timer, not from software.** Software triggering inherits the jitter of your main loop; a timer trigger is exact, and it is a prerequisite for any signal processing that assumes a uniform sample interval.
- **Use DMA.** One interrupt per conversion at 100 kHz is 100,000 interrupts per second; one interrupt per 512-sample DMA half-buffer is 390. Chapter 15 covers the pattern.
- **Respect Nyquist.** Sample at more than twice the highest frequency present - **present**, not present-and-of-interest. Without an analogue anti-alias filter, a 60 Hz hum sampled at 100 Hz appears at 40 Hz and no amount of digital filtering can remove it afterwards.
- **Sample the internal reference and temperature channels** periodically: they let you detect supply drift and self-heating in the field.

11.4 Getting analogue out: DAC and PWM

Method	Resolution	Notes
True DAC	8-12 bits	Fast, clean, needs a buffer for any real load
PWM + RC filter	Set by PWM bits	Nearly free; the filter trades ripple against response time
Sigma-delta on a GPIO	High, at low bandwidth	Better spectral behaviour than plain PWM for audio
External DAC over SPI/I2C	16-24 bits	When precision matters more than latency

PWM AS A DAC: SIZING THE FILTER

A 20 kHz PWM through an RC filter with $R = 10 \text{ k}\Omega$ and $C = 1 \text{ }\mu\text{F}$ has a cutoff of $f_c = 1/(2 \pi RC) = \mathbf{15.9 \text{ Hz}}$, so the PWM carrier at 20 kHz is attenuated by roughly $20 \log_{10}(20000/15.9) = 62 \text{ dB}$ - the ripple on a 3.3 V swing falls to about 2.6 mV, under two LSBs of a 12-bit range. The price is the step response: 3 time constants is $3 \times 10 \text{ ms} = \mathbf{30 \text{ ms}}$ to settle. Ripple and speed trade directly; if you need both, raise the PWM frequency or use two filter stages.

11.5 Sensors, and the maths they need

Sensor	What firmware must do
NTC thermistor	Solve the beta or Steinhart-Hart equation, or use a lookup table with interpolation - the response is strongly non-linear
RTD (PT100/PT1000)	Linear to first order but needs the Callendar-Van Dusen correction for accuracy, plus lead-resistance compensation
Thermocouple	Microvolt-level signal needing amplification, plus cold-junction compensation from a second local sensor
Strain gauge bridge	Differential measurement in millivolts; ratiometric excitation and careful averaging
Current shunt	Watch the common-mode voltage; high-side sensing needs a dedicated amplifier

Sensor	What firmware must do
Digital sensor (I2C/SPI)	The conversion happens in the sensor - your job is the protocol, the units, and the datasheet's compensation formulae, which are often non-trivial

FILTER FOR THE RIGHT REASON

A moving average is a low-pass filter with a poor frequency response and a delay of half its window. In a control loop that delay is phase margin you no longer have, and the loop oscillates. Choose the filter from the requirement: a median filter removes impulsive spikes (a motor commutating), an exponential moving average smooths noise cheaply (one multiply-add of state), and a properly designed IIR or FIR is warranted when the frequency response actually matters. State the cutoff and the added delay in a comment - both are part of the control design.

Exercises

- 1 Compute the required acquisition time for your sensor's source impedance, then measure the error when you halve it.
- 2 Read a fixed voltage 1,000 times and plot the histogram. Compute the effective number of bits and compare with the datasheet.
- 3 Implement 16x oversampling with decimation and confirm you gain the predicted two bits on a noisy input.
- 4 Build a PWM DAC, measure its ripple and settling time, and compare with the arithmetic in the box.
- 5 Take an NTC thermistor, implement both a beta-equation conversion and a 16-point interpolated table, and compare accuracy and cycle cost.

12. Serial Buses I: UART, SPI and I2C

Three protocols carry the overwhelming majority of traffic between a microcontroller and everything around it. They are simple enough to bit-bang and subtle enough to lose a week to. This chapter is the detail that datasheets assume you know.

12.1 Choosing between them

	UART	SPI	I2C
Wires	2 (+2 for flow control)	4 + 1 chip select per device	2, shared
Clock	None - both ends must agree on the rate	Master supplies SCK	Master supplies SCL; slaves may stretch it
Speed	9.6 kbps - 3 Mbps typical	1-100 MHz	100 k / 400 k / 1 M / 3.4 M
Devices	Point to point (or a bus with RS-485)	Many, one CS each	Up to 127 addresses on one pair
Addressing	None	By chip select	7- or 10-bit address
Failure mode	Baud mismatch: consistent garbage	Mode mismatch: shifted bits	A stuck slave holds SDA low and kills the bus
Use it for	Consoles, GPS, modems, board-to-board over wires	Fast sensors, displays, external flash, radios	Slow sensors, EEPROMs, configuration

12.2 UART: framing and the baud-rate arithmetic

```

idle      start  D0 D1 D2 D3 D4 D5 D6 D7 parity stop idle
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
      |         |         |         |         |         |         |         |         |
      +-----+   LSB first                                     +--

```

8N1 = 8 data bits, no parity, 1 stop bit -> 10 bits per byte,
 so 115200 baud carries at most 11,520 bytes per second.
 The receiver samples in the MIDDLE of each bit, having found the
 start edge - which is why total clock error must stay under ~2%.

There is no clock on the wire: the receiver reconstructs bit timing from the start edge and its own oscillator. Every UART problem is ultimately a timing problem.

BAUD-RATE ERROR, AND WHY IT MATTERS

A UART with 16x oversampling from an 8 MHz clock, targeting 115200: the divider is $8e6 / (16 \times 115200) = 4.34$. Registers hold a fractional divider, here $4 + 6/16 = 4.375$, so the actual rate is $8e6 / (16 \times 4.375) = 114,286$ baud - an error of **-0.79%**. If the other end is also 0.79% off in the opposite direction, the total is 1.58%. By the tenth bit of a frame the sampling point has drifted 15.8% of a bit width - still inside the bit, so it works. At 3% total it does not, and the symptom is occasional framing errors that worsen with temperature. **Compute this number for your clock and baud before blaming the cable.**

Feature	Why you need it
RTS/CTS hardware flow control	The only reliable way to prevent overruns when the receiver is sometimes busy
IDLE line interrupt + DMA	The standard high-performance receive pattern: DMA into a circular buffer, and an interrupt when the line goes idle tells you a message ended - no per-byte interrupts
Break detection	A framing 'zero' longer than a character: used by LIN and by many bootloaders as an attention signal
Parity	One bit of error detection. Weak - prefer a CRC in the message
Overrun/framing/noise error flags	Count them and expose them; they diagnose cable, clock and speed problems

RS-232, RS-485 and TTL are **electrical** standards, not protocols: the same UART frame at different voltages. RS-485 is differential and multi-drop over hundreds of metres, half duplex, so firmware must drive a direction pin - and must not release it until the last stop bit has physically left the shift register. Releasing on 'transmit buffer empty' instead of 'transmission complete' truncates the final byte, which is the classic RS-485 bug.

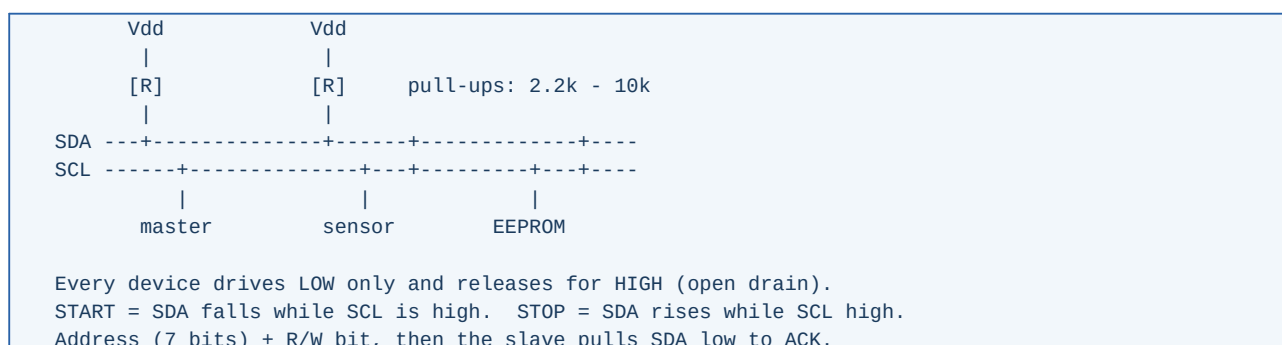
12.3 SPI: four modes and the details that bite

Mode	CPOL	CPHA	Meaning
0	0	0	Clock idles low; sample on the rising edge
1	0	1	Clock idles low; sample on the falling edge
2	1	0	Clock idles high; sample on the falling edge
3	1	1	Clock idles high; sample on the rising edge

Modes 0 and 3 cover most devices. A mode mismatch does not produce silence - it produces data shifted by one bit, which reads as plausible-but-wrong values. If a new SPI device returns 0xFF, 0x00, or every value doubled, check the mode before anything else.

- **SPI is always full duplex.** To read, you must clock out dummy bytes. `spi_transfer(tx, rx, len)` is the honest API; a `spi_read()` that hides the transmit buffer eventually confuses someone.
- **Chip select in software is usually better.** Hardware NSS has surprising timing rules; a GPIO gives you explicit control of setup and hold time, and lets you keep CS asserted across multiple transfers where the device requires it.
- **Respect the device's timing.** Many parts need a gap between CS falling and the first clock, or a maximum clock rate that drops when reading versus writing (external flash typically clocks slower for read commands without a dummy cycle).
- **Check the maximum frequency against the wiring.** A 50 MHz SPI over 20 cm of ribbon cable will not work no matter how correct the firmware is; reduce the clock, add series resistors, or shorten the cable.
- **Use DMA for anything over a few dozen bytes**, and for displays always - a 320x240 16-bit frame is 150 KB, which is 150,000 interrupts if you do it byte by byte.

12.4 I2C: the bus that fails in interesting ways



Open-drain wired-AND is what allows multi-master arbitration and clock stretching - and what makes a single stuck device disable the whole bus.

SIZING THE PULL-UP RESISTORS

The rise time is set by the pull-up and the total bus capacitance. Rising from 0.3 Vdd to 0.7 Vdd takes $t_r = 0.8473 \times R \times C$. The I2C specification allows 1000 ns at 100 kHz and 300 ns at 400 kHz. With a typical bus capacitance of 200 pF: at 100 kHz, $R \leq 1000\text{e-}9 / (0.8473 \times 200\text{e-}12) = \mathbf{5.9\text{ k}\Omega}$; at 400 kHz, $R \leq 300\text{e-}9 / (0.8473 \times 200\text{e-}12) = \mathbf{1.77\text{ k}\Omega}$. The lower bound comes from the 3 mA sink current limit: $R \geq 3.3 / 0.003 = 1.1\text{ k}\Omega$. So 4.7 kΩ is the classic choice for 100 kHz and 1.8-2.2 kΩ for 400 kHz - and if someone fitted 10 kΩ on a long bus, 400 kHz will fail while 100 kHz works, which is a confusing symptom until you put a scope on SCL and see the rounded edges.

Failure	Symptom	Handling
Slave holds SDA low	The whole bus is dead until power cycle	Bus recovery: send up to 9 clock pulses on SCL by bit-banging until SDA releases, then issue a STOP
Clock stretching too long	Master times out	Support stretching; give sensors that do slow conversions a generous timeout
Address collision	Two devices answer	Check address pins; many parts offer only 2-8 addresses, so use a mux or a second bus
Missing ACK	Transfer aborts	Distinguish 'device absent' from 'device busy' - many EEPROMs NACK while writing, which is the documented way to poll for completion
No pull-ups fitted	Everything reads 0x00 or nothing	The first thing to check on a new board
Peripheral lock-up erratum	The I2C block itself hangs	Follow the vendor's documented software reset sequence; keep a recovery path in the driver

```

/* Every bus API should be able to fail, and say how. */
typedef enum { I2C_OK, I2C_NACK_ADDR, I2C_NACK_DATA,
               I2C_TIMEOUT, I2C_ARB_LOST, I2C_BUS_ERROR } i2c_err_t;

i2c_err_t i2c_write_read(uint8_t addr,
                        const uint8_t *tx, size_t ntx,
                        uint8_t *rx, size_t nrx,
                        uint32_t timeout_ms);
/* One call performs write, repeated START, read - the pattern
   nearly every register-based sensor requires. Splitting it into
   separate write() and read() calls inserts a STOP that many
   devices interpret as 'abandon the transaction'. */

```

12.5 A driver structure that works for all three

Whatever the bus, the shape of a good driver is the same: a blocking API for configuration, a non-blocking or DMA API for streaming, a timeout on every wait, an explicit error enum, and no global state so that two instances of the same peripheral can coexist. Chapter 23 develops this into a layered design; the rule to adopt now is simply **every loop that waits for a hardware flag has a timeout**.

```

/* Never write this: */
while (!(SPI1->SR & SPI_SR_TXE)) { }          /* hangs forever */

/* Write this: */
uint32_t start = DWT->CYCCNT;
while (!(SPI1->SR & SPI_SR_TXE)) {
    if ((DWT->CYCCNT - start) > TIMEOUT_CYCLES) return DRV_TIMEOUT;
}

```

One unbounded wait loop is all it takes to convert a recoverable hardware glitch into a hung product. Chapter 28 makes this a rule with teeth.

Exercises

- 1 Compute the baud error for your clock at 9600, 115200 and 921600, and measure the actual bit time on a scope.
- 2 Implement UART receive with DMA plus the IDLE-line interrupt and compare CPU load with a per-byte interrupt version at 921600 baud.
- 3 Deliberately set the wrong SPI mode for a sensor and record exactly what the data looks like, so you recognise it next time.
- 4 Measure your I2C rise time with a scope and check it against the specification; then change the pull-ups and measure again.
- 5 Implement the 9-clock I2C bus recovery and prove it by holding SDA low with a jumper.

13. Serial Buses II: CAN, USB, Ethernet and the Rest

The buses in this chapter carry more structure than a byte stream: arbitration, enumeration, addressing, and stacks with their own state machines. You will rarely write these from scratch, but you must understand them to configure, debug and bound them.

13.1 CAN and CAN-FD

CAN was designed for cars: multi-master, differential, robust to noise, and with **non-destructive arbitration** - when two nodes transmit at once, the one sending the lower identifier wins and the other backs off without a collision or a retransmission. Priority is therefore a property of the message ID, decided at system design time.

Property	Classic CAN	CAN-FD
Payload	0-8 bytes	0-64 bytes
Bit rate	Up to 1 Mbps	Arbitration up to 1 Mbps, data phase up to 5-8 Mbps
Identifier	11-bit (standard) or 29-bit (extended)	Same
Error handling	Automatic retransmission, error counters, bus-off	Same, plus a stronger CRC

CAN BIT TIMING, COMPUTED

A CAN bit is divided into time quanta (tq): 1 sync segment + propagation + phase1 + phase2. For 500 kbps from a 40 MHz peripheral clock with 20 tq per bit: the tq rate must be $500e3 \times 20 = 10 \text{ MHz}$, so the prescaler is $40e6/10e6 = 4$. Splitting 20 tq as 1 sync + 13 + 6 puts the **sample point** at $(1+13)/20 = 70\%$ of the bit, which is the automotive convention (75-87.5% is the usual target; CANopen specifies 87.5%). Every node on the bus must agree on the sample point closely enough that propagation delay across the longest cable still lands inside the same bit - which is why the maximum bus length falls as the bit rate rises (about 40 m at 1 Mbps, 500 m at 125 kbps).

- **Error states matter.** Each node counts transmit and receive errors; at 128 it becomes error-passive, at 256 **bus-off** and stops transmitting entirely. Firmware must detect bus-off, log it, and implement a recovery policy - silently auto-recovering hides a physical fault, never recovering bricks the function.
- **Use hardware filters.** A node on a busy bus can see thousands of frames per second; acceptance filters let the peripheral discard everything not addressed to you, at zero CPU cost.
- **Higher layers:** CANopen (industrial), J1939 (trucks), UDS/ISO-TP (diagnostics and flashing over CAN), and AUTOSAR stacks. ISO-TP is the segmentation protocol that carries an OTA image over 8-byte frames - Chapter 31.

13.2 USB, in the amount a firmware engineer needs

Host	Device (your MCU)
1. detects pull-up on D+ (full speed)	
2. resets the bus	
3. GET_DESCRIPTOR(device) on endpoint 0	
4. SET_ADDRESS	
5. GET_DESCRIPTOR(config, interfaces, endpoints, strings)	
6. SET_CONFIGURATION -> device is ready	
then: class-specific traffic on endpoints 1..n	

Enumeration. Everything before step 6 happens on endpoint 0 with control transfers, and a device that answers those correctly is 90% of the way to working.

Transfer type	Guarantee	Used by
Control	Reliable, low bandwidth	Enumeration, configuration

Transfer type	Guarantee	Used by
Bulk	Reliable, no timing guarantee, uses spare bandwidth	Mass storage, CDC serial, DFU
Interrupt	Bounded latency, small payload per interval	HID: keyboards, mice, sensors
Isochronous	Guaranteed bandwidth, no retries	Audio and video, where late data is worse than lost data

- **Use a class that already has a host driver.** CDC-ACM (serial), HID (no driver needed anywhere), MSC (drag-and-drop firmware update), and DFU cover most needs without writing a host-side driver or signing one.
- **Descriptors are a data structure, not code** - and one wrong length byte causes a silent enumeration failure. Use a generator or a validated example, and read the host's log (`dmesg`, USB tree viewer) rather than guessing.
- **Timing is not negotiable:** the device must answer control transfers within milliseconds, so USB interrupt handling cannot be starved by a long critical section. This is a real constraint on the rest of your firmware.
- **A crystal is usually required** (+/-0.25%), unless the part has crystal-less USB that trims its RC oscillator against host frames.

13.3 Ethernet and IP on a microcontroller

An MCU with an Ethernet MAC connects to an external PHY over RMII or MII; firmware configures the PHY over MDIO, manages DMA descriptor rings for transmit and receive, and hands frames to a TCP/IP stack - lwIP being the near-universal choice on MCUs.

Concern	Guidance
Buffer strategy	Zero-copy where the stack supports it; size the descriptor rings for your burst rate, not your average
Checksum offload	Let the MAC compute IP/TCP checksums; it is free performance and removes a per-byte CPU cost
Cache coherency	On a cached part, descriptors and buffers must be in non-cacheable memory or maintained explicitly (Chapter 2)
Memory footprint	lwIP with TCP needs tens of KB of RAM; plan for it before choosing a part
TLS	mbedTLS or wolfSSL; a handshake needs 20-40 KB of RAM and hundreds of milliseconds without hardware acceleration
Time sync	IEEE 1588/PTP if the MAC supports hardware timestamping - sub-microsecond; NTP/SNTP otherwise

13.4 The rest of the bus zoo

Bus	What it is	Where you meet it
LIN	Single-wire, master-slave, 20 kbps, built on UART framing	Cheap automotive nodes: mirrors, seats, switches
Modbus RTU	A request-response protocol over RS-485	Industrial sensors, PLCs, energy meters
1-Wire	Data and (parasitic) power on one line, timing-critical bit-banging	Temperature sensors, ID chips
I2S / TDM	Synchronous audio streaming	Codecs, MEMS microphones, amplifiers
SDIO / SD	Command and data lines to SD cards and some radios	Bulk storage, logging
QSPI / OSPI	Quad or octal SPI to external flash, often memory-mapped	Code and asset storage, execute-in-place

Bus	What it is	Where you meet it
Parallel camera (DCMI) / MIPI CSI	Wide parallel or serial pixel streams into DMA	Imaging
SWD / JTAG	The debug transport itself	Chapter 25

INSTRUMENT THE BUS BEFORE YOU ARGUE ABOUT IT

A logic analyser with protocol decoders costs less than an hour of engineering time and answers, definitively: is the clock running, is the address right, did the slave ACK, is the mode correct, how long is the gap between transactions. Firmware engineers who own one resolve bus problems in minutes; those who do not, read code and guess.

Exercises

- 1 Compute a CAN bit timing configuration for 500 kbps on your part and verify the sample point matches the rest of the bus.
- 2 Force a CAN node into bus-off (short CANH to CANL briefly) and implement a logged, rate-limited recovery.
- 3 Bring up USB CDC on a board and read the host's enumeration log; then corrupt one descriptor length byte and observe the failure.
- 4 Measure the maximum sustained TCP throughput of your MCU's Ethernet and identify whether the CPU, the RAM or the stack is the limit.
- 5 Decode one transaction from each of UART, SPI and I2C on a logic analyser and annotate every field by hand.

14. Wireless: BLE, Wi-Fi, LoRa and Cellular

Adding a radio changes firmware architecture more than any other peripheral. The link is unreliable by nature, the stack is large and often not yours, timing is constrained by the protocol, and the power budget is now dominated by transmissions. This chapter covers what to expect from each technology and the design rules common to all.

14.1 Two architectures, and the choice between them

	Radio SoC (stack on the MCU)	Host + modem module
Example	nRF52/nRF53, ESP32, STM32WB/WL	An MCU plus a Wi-Fi or cellular module over UART/SPI
Pros	One chip, lowest cost and power, full control of timing	Certified module, simpler firmware, radio complexity is someone else's
Cons	Your firmware shares the CPU with strict radio timing; stack updates are your problem	A serial protocol (often AT commands) with its own failure modes; higher power
Impact on your code	Radio interrupts must be highest priority; your critical sections must stay short	Everything is asynchronous request-response with timeouts and retries

14.2 The technologies, compared honestly

Technology	Range / rate	Power	Use it when
Bluetooth LE	10-100 m, 0.1-1.4 Mbps effective	uA average when advertising slowly	A phone is the other end, or short-range sensor links
Wi-Fi	30-100 m, tens of Mbps	50-300 mA while associated	Mains power and IP connectivity are both available
Thread / Zigbee / Matter	Mesh, 250 kbps	Low; routers must stay awake	Many nodes in a building, local control without the cloud
LoRaWAN	2-15 km, 0.3-50 kbps	Very low; duty-cycle limited	Small messages, long range, battery years
NB-IoT / LTE-M	Cellular coverage, 20-1000 kbps	mA-level in connected mode, uA in PSM	No local gateway exists and coverage does
Sub-GHz proprietary	100 m - 5 km	Low	You control both ends and want no stack overhead
NFC	Centimetres	Powered by the reader	Commissioning, pairing, tap-to-configure

BLE THROUGHPUT AND BATTERY, FROM THE CONNECTION INTERVAL

BLE moves data in connection events. With a 7.5 ms connection interval, 6 packets per event and a 244-byte ATT payload (DLE enabled), the ceiling is $6 \times 244 / 0.0075 = 195 \text{ kB/s}$ = about **1.56 Mbps** at the application layer - but only if both ends agree to those parameters, and iOS and Android each impose their own limits. Now the other direction: advertising once per second at roughly 30 uA-seconds per advertising event plus 3 uA sleep gives about **33 uA average**, so a 220 mAh coin cell lasts $220,000/33 = 6,700 \text{ hours}$ = **9 months**. Halve the advertising rate and it lasts nearly twice as long. **The connection interval is the single parameter that trades latency, throughput and battery life** - decide it deliberately, not by copying an example.

14.3 Design rules that apply to every radio

- **Assume every transmission fails.** Design the protocol with idempotent messages, sequence numbers, acknowledgement and bounded retries with exponential backoff plus jitter. Without jitter, a thousand devices that lost power together will reconnect in lockstep and take down your server.
- **Buffer for the outage, not the average.** Store-and-forward with a ring buffer in flash means a two-hour network outage costs latency, not data.
- **Never block on the radio.** Every send is asynchronous with a timeout; a stack that stops responding must be recoverable by resetting the module without rebooting the product.
- **Budget power per message, not per second.** One transmission may cost more energy than an hour of sleep (Chapter 29); the protocol design *is* the power design.
- **Plan provisioning and keys** before the first prototype: how does a device learn its credentials, and how are they protected (Chapters 32 and 35)?
- **Keep the OTA path independent of the application.** If a bad release breaks the application's networking, the bootloader must still be able to recover the device (Chapter 31).

CERTIFICATION CONSTRAINS YOUR FIRMWARE

Radio modules are pre-certified only if you use them within their stated conditions: the same antenna, the same power settings, sometimes the same stack version. Changing transmit power or duty cycle in firmware can invalidate a certification and make the product illegal to sell. Regional limits differ (channel plans, duty-cycle limits in the EU sub-GHz bands, LBT requirements), so region configuration is a firmware feature with legal consequences, and it belongs in the provisioned configuration - not in a `#define`.

14.4 Enough RF to make good decisions

Link budget (dB):

$$\begin{aligned} \text{received power} &= \text{TX power} + \text{TX antenna gain} - \text{path loss} \\ &\quad + \text{RX antenna gain} \\ \text{link margin} &= \text{received power} - \text{receiver sensitivity} \end{aligned}$$

Free-space path loss:

$$\text{FSPL(dB)} = 20 \log_{10}(d_m) + 20 \log_{10}(f_{\text{Hz}}) - 147.55$$

Rules of thumb:

doubling the distance costs 6 dB
 +6 dB of margin roughly doubles the achievable range
 a human body or a metal enclosure can cost 10-20 dB

Everything in radio is decibels: 3 dB is a factor of two in power, 10 dB a factor of ten.

WILL THIS LINK WORK ACROSS A BUILDING?

BLE at 2.44 GHz, 0 dBm transmit, -2 dBi antennas at both ends, receiver sensitivity -95 dBm, over 30 m. $\text{FSPL} = 20 \log_{10}(30) + 20 \log_{10}(2.44\text{e}9) - 147.55 = 29.5 + 187.7 - 147.55 = \mathbf{69.7 \text{ dB}}$. Received power = $0 - 2 - 69.7 - 2 = \mathbf{-73.7 \text{ dBm}}$, giving a margin of 21 dB over sensitivity - comfortable in free space. Now add two plasterboard walls (about 4 dB each), a body between the devices (10 dB) and multipath fading (10 dB): 18 dB of the 21 is gone. **This is why a link that works on the bench fails in the installation**, and why the design levers are antenna placement and keeping the enclosure away from the antenna keep-out - both decided long before firmware.

- **Sensitivity improves as the data rate falls.** BLE coded PHY, LoRa spreading factors and lower-rate modulations buy range with time on air - and time on air is energy (Chapter 29). Range, rate and battery life are one trade, not three.

- **Retransmissions are not free.** A link at the edge of its margin retries constantly, so the average current rises sharply just as the throughput falls. Report the link margin (RSSI) in telemetry: it is the early warning for an installation that is about to become unreliable.
- **Certified modules constrain firmware** (see the warning below), and the antenna keep-out, ground plane and matching network are hardware decisions your firmware cannot compensate for.

Exercises

- 1 Measure the current profile of one BLE advertising event with a current probe or a power analyser, and compute the battery life at three advertising intervals.
- 2 Implement exponential backoff with jitter for reconnection and simulate 500 devices reconnecting simultaneously.
- 3 Build store-and-forward buffering and prove no data is lost across a deliberate 30-minute outage.
- 4 Take a module-based design and write the recovery path: detect an unresponsive module, reset it, re-provision, resume - with logging at each step.

15. DMA: Moving Data Without the CPU

Direct memory access is a second processor whose only skill is copying. Used well it removes entire interrupt loads and makes high data rates possible on modest parts. Used carelessly it produces the hardest bugs in firmware, because the corruption happens while your code is elsewhere.

15.1 What DMA buys, in numbers

THE INTERRUPT LOAD DMA REMOVES

Driving a 320x240 16-bit display over 40 MHz SPI means 153,600 bytes per frame. Byte-at-a-time interrupts at roughly 25 cycles each - entry, flag check, store, exit - cost $25 \times 153,600 = 3.84$ million cycles per frame; at 30 frames per second that is 115 million cycles, or **69% of a 168 MHz core** doing nothing but shovelling bytes. With DMA it is two interrupts per frame - transfer complete and error - and the CPU is free to render the next frame. The same arithmetic applies to a 100 kHz ADC (100,000 interrupts per second becomes 390 with a 512-sample double buffer) and to any UART above a few hundred kilobaud.

15.2 The mental model

```

peripheral request line
  |
  v
+-----+ reads +-----+ writes +-----+
| DMA chan. | <----- | source | -----> | dest   |
+-----+ +-----+ +-----+
          | arbitrates for the bus with the CPU
          v
interrupts: HALF transfer, FULL transfer, TRANSFER ERROR

Configure: source, destination, length, item width, increment
or not on each side, circular or one-shot, priority, request source.

```

Every DMA controller in every family is this, with different register names. The differences that matter are FIFO depth, burst support, and which requests are hard-wired to which channels.

Mode	Behaviour	Use
One-shot	Transfers N items then stops	A single SPI frame, one flash page
Circular	Wraps to the start automatically, forever	ADC sampling, UART receive, audio - the peripheral never stops
Double buffer / ping-pong	Two buffers alternate; the half- and full-transfer interrupts tell you which half is safe	The standard streaming pattern: process one half while the other fills
Memory to memory	No peripheral involved	Large copies in the background - but check it is actually faster than <code>memcpy</code> on your part
Linked list / scatter-gather	A descriptor chain the DMA walks itself	Ethernet frames, complex waveform sequences

```

/* The ping-pong idiom that underlies almost all streaming. */
static int16_t buf[2][512];      /* one DMA buffer, two halves */
volatile uint8_t ready_half = 0xFF;

void DMA1_Stream0_IRQHandler(void)
{
    if (half_transfer_flag()) { clear_ht(); ready_half = 0; }
    if (full_transfer_flag()) { clear_tc(); ready_half = 1; }
}

void main_loop(void)              /* or a task */
{
    if (ready_half != 0xFF) {
        uint8_t h = ready_half; ready_half = 0xFF;
        process(buf[h], 512);      /* must finish before the
                                   other half completes! */
    }
}

```

The deadline is explicit: at 100 kHz with 512 samples per half, `process()` has 5.12 ms. Exceeding it silently corrupts data, so measure it (Chapter 30) and assert on it.

15.3 The five DMA bugs

Bug	Symptom	Prevention
Buffer out of scope or reused	Corruption after a function returns - DMA writes into a dead stack frame	DMA buffers are <code>static</code> or long-lived, never automatic
Cache not maintained (Cortex-M7, Cortex-A)	Stale data in one direction only	Clean before a DMA read, invalidate after a DMA write, or use non-cacheable memory
Length or width mismatch	Half the data, or every other byte	Item width x count = bytes; check both sides' increment settings
Wrong request/channel mapping	Transfer never starts, or starts on the wrong event	Check the request-mapping table in the reference manual - it is device-specific
Race on the completion flag	Occasional lost or duplicated blocks	One writer, one reader, and a memory barrier between the data and the flag

ALWAYS ENABLE THE DMA ERROR INTERRUPT

A transfer error - a bad address, a bus fault, a FIFO overrun - is otherwise completely silent: the stream simply stops, and the symptom appears minutes later as 'the sensor stopped updating'. Handle the error, count it, log it, and restart the stream deliberately.

15.4 Circular reception: the pattern for an unknown message length

Receiving a stream whose message boundaries you do not know in advance - a UART console, a GPS, a modem - is the case people get wrong. The answer is a circular DMA buffer that never stops, plus a read pointer you advance yourself.

```

/* DMA fills rx[] forever; the consumer chases the DMA's position. */
static uint8_t rx[512];
static size_t tail; /* consumer position */

static inline size_t dma_head(void) /* where DMA will write */
{ return sizeof(rx) - DMA1_Stream5->NDTR; }

size_t uart_read(uint8_t *out, size_t max)
{
    size_t head = dma_head(), n = 0;
    while (tail != head && n < max) {
        out[n++] = rx[tail];
        tail = (tail + 1u) % sizeof(rx);
    }
    return n;
}
/* The IDLE-line interrupt tells you a message just ended, so the */
/* task can be woken exactly once per message instead of polling. */

```

The counter register (**NDTR** here) is the position: no interrupt is needed to know how much has arrived. Size the buffer for the longest gap between reads times the byte rate, and count overruns - if the consumer falls a whole lap behind, the data is already gone.

SIZING A CIRCULAR RECEIVE BUFFER

A 921,600 baud link delivers 92,160 bytes per second, or one byte every 10.9 us. If the consuming task can be delayed by up to 20 ms (a slow flash write, a long higher-priority burst), it may miss $20 \times 10^{-3} \times 92,160 = \mathbf{1,844 \text{ bytes}}$. A 512-byte buffer therefore overruns; 2,048 bytes is the smallest safe power of two, and 4,096 gives margin. **The buffer size is a function of the worst-case scheduling delay** from Chapter 20, not of the message size - which is why the two chapters belong together.

15.5 Arbitration, bursts and FIFOs

- **Channel priority** decides which stream wins when several are ready. Give the highest priority to the peripheral that cannot wait - an ADC at 1 MHz overruns in a microsecond, a UART at 115200 has 87 us of slack.
- **Burst transfers and the FIFO** let a stream fetch four or eight beats at once, cutting bus arbitration overhead substantially for memory-to-memory and high-rate peripheral transfers. The rule is that the burst must not cross a 1 KB address boundary, and the buffer alignment must match the burst size - misconfigure it and the controller raises a FIFO error rather than transferring wrongly.
- **Peripheral versus memory increment** is set independently: a peripheral register does not advance, memory does. Half the 'every other byte is wrong' reports are these two settings swapped.
- **Circular plus double-buffer mode** on some families gives two target addresses swapped automatically, so you can process one while the other fills without any pointer arithmetic at all.

DOES THE DMA HAVE ENOUGH BANDWIDTH?

An ADC at 1 Msps into memory, 16 bits per sample, is 2 MB/s. An SPI display at 40 MHz is 5 MB/s. Ethernet at 100 Mbps is 12.5 MB/s. Together, 19.5 MB/s. A 168 MHz AHB matrix moving 4 bytes per cycle has a theoretical 672 MB/s, so bandwidth is not the constraint - **contention is**. Each DMA beat steals a bus cycle from the CPU when they target the same SRAM bank, so the observable effect is that the CPU slows by roughly the fraction of cycles DMA takes. Splitting DMA buffers into a different SRAM bank from the stack and hot data typically recovers most of it, and costs one line in the linker script.

Exercises

- 1 Convert a per-byte interrupt driver to DMA and measure the CPU load before and after with a GPIO and a scope.

- 2 Implement the ping-pong ADC pattern and deliberately make `process()` too slow; observe and then detect the overrun.
- 3 On a cached part, reproduce and then fix a DMA coherency bug in both directions.
- 4 Put a DMA buffer on the stack, watch it corrupt, and explain exactly which bytes were destroyed and why.

16. Non-Volatile Memory and Storage

Every product keeps something across a power cycle: calibration, configuration, counters, logs, and the firmware image itself. Flash is not a disk - it wears out, it erases in large blocks, it cannot be read while it is being written, and a power cut in the wrong microsecond can leave a record half-written. This chapter is how to store data that survives all of that.

16.1 How flash behaves

Property	NOR (internal and QSPI)	NAND (eMMC, SD, raw)
Read	Random access, byte addressable, execute in place	Page at a time, no XIP
Write granularity	Word or page (8-256 bytes)	Page (2-16 KB)
Erase granularity	Sector: 2-128 KB	Block: 128 KB - 1 MB
Erase time	20-500 ms per sector - an eternity	1-3 ms per block
Endurance	10,000 - 100,000 cycles per sector	1,000 - 100,000, and needs ECC and bad-block management
Retention	10-20 years at room temperature, far less when hot and heavily cycled	Similar, with more variation

THE THREE RULES OF FLASH

(1) Writing only clears bits. A programmed cell goes from 1 to 0; only an erase restores 1s. This is why you can add a record to an erased area without erasing again, and why a 'valid' flag should be written **after** the data it validates. **(2) Erase is slow and blocking.** On many parts the whole flash bank cannot be read while one sector is erasing, so the code performing it must execute from RAM, and interrupts fetching from flash will fault. **(3) Every erase wears the sector.** The lifetime of a product can be set by one badly designed counter.

HOW LONG UNTIL YOU WEAR OUT THE FLASH?

A device logs a 32-byte record every 10 minutes: 144 records a day. If each write re-erases a single 2 KB sector holding the log, that is 144 erases a day against an endurance of 10,000 cycles, giving $10,000 / 144 = 69$ days before the sector is unreliable. Two changes fix it. First, **append** rather than rewrite: $2048/32 = 64$ records fit per sector, so the sector is erased only every 64 records, i.e. 2.25 times a day - 4,400 days. Second, **wear-level** across 16 sectors: $16 \times 4,400$ days = **190 years**. Same hardware, same data rate, three orders of magnitude of lifetime, decided by ten lines of storage design.

16.2 Storing configuration so it survives a power cut

The hazard is a reset in the middle of an erase or a write. The rule is: **never leave the only copy of the truth in a partially written state**. Two standard designs achieve that.

```
A/B (ping-pong) records:
  slot A: [magic][version][seq=7][payload][CRC32]    <- valid
  slot B: [magic][version][seq=8][payload][CRC32]    <- being written
On boot: read both, keep the one with a valid CRC and the highest
sequence number. A power cut during the write to B leaves A intact.

Append-only journal:
[rec][rec][rec][rec][partial.....erased 0xFF.....]
On boot: scan forward while records have valid CRCs; the last valid
record is the state. Compact into a fresh sector when full.
```

Both designs share one discipline: a record is valid only when its CRC matches, and the CRC (or a validity marker) is written last.

```
typedef struct {
    uint32_t magic;           /* keep it 4-byte aligned */
    uint16_t version;        /* 'CFG1' - identifies a record */
    uint16_t length;         /* schema version, for upgrades */
    uint32_t sequence;       /* monotonic; highest wins */
    uint8_t  payload[CFG_SIZE];
    uint32_t crc32;          /* over everything above */
} cfg_record_t;

/* Write order matters: payload first, CRC last, then verify by
   reading the record back and re-checking the CRC. A flash cell
   that reads correctly immediately after programming can still be
   marginal - the read-back is not paranoia. */
```

- **Version every stored structure.** Firmware updates change layouts; a **version** field plus a migration function is the difference between an update and a field failure.
- **Keep factory calibration separate from user settings**, in a different sector with different write frequency - and ideally write-protected after production (Chapter 35).
- **Do not store secrets in plain flash.** Readout protection is bypassable; use the part's secure storage or a secure element (Chapter 32).
- **Reserve space you have not designed yet.** Adding a field to a full sector after ten thousand units have shipped is painful.

16.3 File systems, and whether you need one

Option	Footprint	Power-fail safe	Use when
Hand-rolled records	A few hundred bytes of code	Yes, if you design it as above	Configuration and small logs - most products
LittleFS	~10-20 KB code, small RAM	Yes, by design (copy-on-write, wear levelling)	You want files and directories on raw flash with power-cut safety
FatFs (FAT12/16/32)	~10 KB	No - a cut during an update can corrupt the FAT	A PC or a phone must read the medium (SD card, USB mass storage)
SPIFFS	Small	Weak	Legacy projects only
eMMC/SD with a controller	Driver plus a file system	The controller handles wear and ECC, not power loss	Gigabytes: video, audio, dense logs

FAT ON AN SD CARD IN A BATTERY-POWERED PRODUCT

It is the most common storage choice and the most common cause of corrupted field data. FAT updates directory entries and the allocation table separately from the data, so an unexpected power loss can lose a file, a directory, or the whole card. If you must use FAT for host compatibility: keep files open for the shortest possible time, flush after each record, pre-allocate files, and detect the supply falling early enough (a brown-out interrupt plus a bulk capacitor) to close cleanly. Better: log to an append-only region and expose it as FAT only when a host is attached.

16.4 External flash and execute-in-place

QSPI or Octal-SPI flash adds megabytes for a few cents and can be memory-mapped so the CPU executes directly from it (XIP). Three consequences for firmware: reads are slower than internal flash so hot code should stay internal or be cached; writes suspend XIP, so the erase/program routine must run from RAM; and the external part is easy to desolder and read, so anything sensitive on it must be encrypted.

Exercises

- 1 Compute the erase-cycle lifetime of your product's current storage scheme at the real write rate. Most people are surprised.

- 2 Implement the A/B configuration record with CRC and sequence number, then power-cycle the device 500 times during writes and confirm no corruption.
- 3 Measure a sector erase time on your part and check whether interrupts still execute during it.
- 4 Fill a LittleFS volume to 95% and measure how write latency changes.
- 5 Deliberately corrupt one byte of a stored record and confirm your boot code detects it and falls back to defaults.

PART III

Firmware Architecture

How to organise a program that has no operating system, or one so small it fits in a few kilobytes: superloops and state machines, sharing data with interrupts, what an RTOS actually does, synchronisation without deadlock, proving deadlines are met, memory discipline, driver layering, and protocol stacks.

17. Architecture Without an Operating System

Most firmware ever written runs one loop. Done badly that loop becomes a tangle of flags, delays and special cases that nobody dares change. Done well it is a clear, testable, deterministic program that is easier to reason about than any RTOS. This chapter is how to do it well.

17.1 The superloop, and where it breaks

```
int main(void)
{
    hw_init();
    for (;;) {
        read_sensors();
        update_control();
        drive_outputs();
        service_comms();
        HAL_Delay(10);          /* <-- the seed of every later problem */
    }
}
```

It works until one of these happens, and one of them always does: a function starts taking 50 ms and the control loop jitters; two features need different rates; something must respond in 1 ms while a flash write takes 100 ms; or a new engineer adds another `HAL_Delay` and the loop period becomes an emergent property nobody can predict.

Symptom	Root cause	Fix in this chapter
Loop period varies wildly	Blocking calls inside the loop	Non-blocking state machines
Features interfere with each other's timing	One rate for everything	A cooperative scheduler with per-task periods
Response time is unbounded	Long work in the loop	Split work into run-to-completion steps
Flags everywhere, order matters	Implicit state	Explicit state machines and event queues
Cannot test anything	Logic entangled with hardware	Layering with injected dependencies (Chapter 23)

17.2 Step one: a time-triggered cooperative scheduler

```
typedef struct { uint32_t period, next; void (*fn)(void); } task_t;

static task_t tasks[] = {
    { .period = 1, .fn = control_step }, /* 1 kHz */
    { .period = 10, .fn = sensor_step }, /* 100 Hz */
    { .period = 100, .fn = comms_step }, /* 10 Hz */
    { .period = 500, .fn = ui_step }, /* 2 Hz */
};

for (;;) {
    uint32_t now = millis();
    for (size_t i = 0; i < ARRAY_SIZE(tasks); i++) {
        if (time_after(now, tasks[i].next)) {
            tasks[i].next += tasks[i].period; /* no drift */
            tasks[i].fn(); /* must NOT block */
        }
    }
    __WFI(); /* sleep until the next interrupt */
}
```

```
}

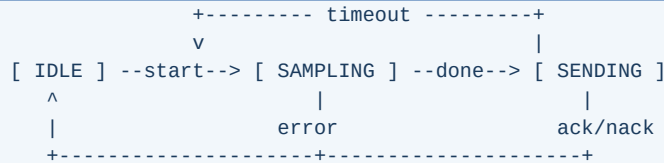
```

Fifteen lines that give periodic execution, a sleep-when-idle power story, and a single place to instrument every task's execution time. The one rule that makes it work: **no function called from here may block**.

RUN TO COMPLETION

Each task function runs from start to finish without waiting, then returns. Anything that would wait becomes a state machine that returns now and resumes at the next call. This single discipline gives you bounded latency for everything else, makes worst-case timing measurable (Chapter 20), and removes the need for a stack per task - which is what an RTOS spends most of its RAM on.

17.3 State machines: the core tool



Events: START, SAMPLE_DONE, TX_DONE, ACK, NACK, TIMEOUT, ERROR
Each (state, event) pair has exactly one defined action.

Drawing this before coding it takes ten minutes and eliminates the 'what happens if the timeout fires while we are already sending' class of bug - because the diagram forces you to answer it.

```
typedef enum { ST_IDLE, ST_SAMPLING, ST_SENDING, ST_N } state_t;
typedef enum { EV_START, EV_DONE, EV_TIMEOUT, EV_ERR, EV_N } event_t;

/* Table-driven: the transition table IS the specification. */
static const state_t next[ST_N][EV_N] = {
    /*          START      DONE      TIMEOUT      ERR      */
    /* IDLE    */ {ST_SAMPLING, ST_IDLE, ST_IDLE, ST_IDLE },
    /* SAMPLING*/ {ST_SAMPLING, ST_SENDING, ST_IDLE, ST_IDLE },
    /* SENDING */ {ST_SENDING, ST_IDLE, ST_IDLE, ST_IDLE },
};

void fsm_dispatch(event_t e)
{
    state_t from = st, to = next[st][e];
    if (to != from) { on_exit(from); st = to; on_entry(to); }
}
```

A **switch** statement is fine for three states; a table is better past five, because it makes the empty cells - the transitions nobody thought about - visible. For deeply nested behaviour, hierarchical state machines (UML statecharts, the QP framework) let a parent state handle events that all its children share.

17.4 Events, queues and decoupling

The next step up is an event-driven architecture: interrupts and tasks post small events into a queue, and one dispatcher pulls them out and feeds the relevant state machine. It costs one queue and gives three things: ISRs become trivially short, modules stop calling each other directly, and the whole system becomes replayable - log the event stream and you can reproduce a field failure on the bench.

Pattern	When to use	Cost
Direct call	Fast, simple, one caller	Tight coupling; the caller's stack and timing become the callee's
Flag plus poll	An ISR signals the main loop	Latency of one loop period; needs care with shared data
Event queue	Many producers, one consumer, ordering matters	A few hundred bytes; the standard choice

Pattern	When to use	Cost
Publish-subscribe	Several unrelated modules care about the same event	Indirection makes control flow harder to follow - use sparingly and document the topics
Callback with context	A driver notifying its owner	Runs in the caller's context - document whether that is an ISR

17.5 Bare metal or RTOS?

Choose bare metal / cooperative when	Choose an RTOS when
The work decomposes into short periodic steps	Some work is long and must be preempted by short work
RAM is scarce (a stack per task is 0.5-4 KB each)	You have RAM to spare and many independent activities
Determinism and certification matter, and you want to argue about every context switch	A protocol stack (BLE, TCP/IP, USB host) already assumes threads
The team is small and the product is focused	Several people work on independent subsystems
Power management is aggressive and hand-tuned	Blocking APIs would make the code much clearer

YOU CAN HAVE BOTH, AND USUALLY SHOULD

Even in an RTOS design, write each task's body as a run-to-completion state machine over an event queue. You then get preemption where it helps and determinism where it matters, and the same logic can be compiled and unit-tested on a host with no RTOS at all - which is what makes Chapter 26 possible.

Exercises

- 1 Take an existing superloop and measure each function's execution time with a GPIO. Plot the loop period distribution over a minute.
- 2 Convert one blocking sequence (for example: assert CS, wait, send, wait, release) into a state machine and confirm the loop period becomes stable.
- 3 Implement the cooperative scheduler above and add per-task worst-case execution time tracking.
- 4 Draw the state diagram for a feature you have already written. Count the transitions that are undefined in the code.
- 5 Replace a chain of three module-to-module calls with an event queue and compare the resulting stack depth.

18. Concurrency, Shared Data and Lock-Free Patterns

The moment you enable one interrupt, your program is concurrent. Two flows of control now touch the same variables at times you do not choose, and the resulting bugs are rare, non-reproducible, and expensive. This chapter is the small set of techniques that make shared data safe, and the discipline of knowing which one you are using.

18.1 Where the concurrency comes from

- **Interrupts** preempt your code between any two instructions - including in the middle of a `++`.
- **DMA** writes memory while the CPU runs, with no synchronisation at all except the flags you check.
- **A second core**, on dual-core parts, executes truly in parallel, with its own cache.
- **RTOS tasks** preempt each other at tick boundaries and at every blocking call.

18.2 What is atomic, and what only looks atomic

THE LOST UPDATE, IN THREE INSTRUCTIONS

`counter++` on a 32-bit value compiles to `LDR r0,[counter] / ADDS r0,#1 / STR r0,[counter]`. Main loop reads 41. An interrupt fires, reads 41, writes 42. The main loop resumes with its stale 41 in a register, adds one, and writes **42**. One increment vanished. The vulnerable window is two instructions wide - about 12 ns at 168 MHz. If the main loop touches the counter once per millisecond, that window covers $12\text{e-}9/1\text{e-}3 = 1.2\text{e-}5$ of the time, so with 1,000 interrupts per second a count is lost about **once every 83 seconds** - often enough to ruin a total over a day, rare enough that it never happens while you are watching it in the debugger.

Operation on Cortex-M	Atomic?
Aligned 32-bit load or store	Yes - a single bus transaction
Aligned 8- or 16-bit load or store	Yes
Unaligned access	No - may be split into several transactions
<code>x++, x = b, x &= ~b</code>	No - read, modify, write
64-bit values, structs, arrays	No
Bitfield assignment	No - the compiler emits a read-modify-write over the whole storage unit, so writing one bitfield can clobber a neighbouring one updated by an ISR
<code>LDREX/STREX</code> pair	Yes, by construction - the basis of C11 atomics and of RTOS-free lock-free code

18.3 The three safe mechanisms

```
/* 1. Critical section - simple, correct, costs interrupt latency */
uint32_t key = irq_lock();
shared.count++; shared.total += value;      /* multi-word update */
irq_unlock(key);

/* 2. Atomic - no latency cost, single word only */
#include <stdatomic.h>
atomic_uint_least32_t counter;
atomic_fetch_add_explicit(&counter, 1u, memory_order_relaxed);

/* 3. Ownership - no sharing at all: the ISR owns the buffer while */
/*   filling it, then hands it over through one atomic word. */
if (buffer_ready) { process(buf); buffer_ready = false; }
```

PREFER OWNERSHIP TO PROTECTION

The cheapest concurrency bug is the one that cannot happen. Design so that each piece of data has exactly one writer at any time and ownership is transferred through a single atomic word - a flag, an index, a queue head. Locks are the fallback for when that is genuinely impossible, not the default.

18.4 The single-producer single-consumer ring buffer

This is the most useful data structure in firmware: an ISR pushes, the main loop or a task pops, and no critical section is needed anywhere - provided there is exactly one producer and exactly one consumer.

```
#define RB_SIZE 256u                                /* MUST be a power of two */
typedef struct {
    uint8_t buf[RB_SIZE];
    volatile uint32_t head;        /* written by producer */
    volatile uint32_t tail;        /* written by consumer */
} ring_t;

bool rb_push(ring_t *r, uint8_t v)    /* ISR context */
{
    uint32_t h = r->head, t = r->tail;    /* one read each */
    if ((uint32_t)(h - t) >= RB_SIZE) return false; /* full */
    r->buf[h & (RB_SIZE - 1u)] = v;
    __DMB();                            /* data before index */
    r->head = h + 1u;                    /* publish */
    return true;
}

bool rb_pop(ring_t *r, uint8_t *v)    /* task context */
{
    uint32_t h = r->head, t = r->tail;
    if (h == t) return false;          /* empty */
    *v = r->buf[t & (RB_SIZE - 1u)];
    __DMB();
    r->tail = t + 1u;
    return true;
}
```

Why it is safe: the producer only ever writes **head**, the consumer only ever writes **tail**, both are 32-bit aligned so each read and write is atomic, and free-running indices with power-of-two masking make the full/empty test a single subtraction that is correct across wrap. The `__DMB()` ensures the data is visible before the index that advertises it.

IT STOPS BEING SAFE THE MOMENT THERE ARE TWO PRODUCERS

Two ISRs pushing into the same ring will both read the same **head** and write the same slot. If you need multiple producers, either give each one its own ring, or protect the push with a critical section, or use a proper multi-producer structure built on **LDREX/STREX**. **Write the number of producers and consumers in a comment above every ring buffer you create** - it is the invariant that keeps the code correct as it is edited by other people.

18.5 Publishing multi-word data without a lock

```

/* Sequence lock: reader retries if the writer was mid-update. */
volatile uint32_t seq;      /* even = stable, odd = being written */
measurement_t data;

void writer(const measurement_t *m) /* ISR */
{ seq++; __DMB(); data = *m; __DMB(); seq++; }

bool reader(measurement_t *out) /* task */
{
    uint32_t s1, s2;
    do {
        s1 = seq; if (s1 & 1u) continue; /* writer is active */
        __DMB(); *out = data; __DMB();
        s2 = seq;
    } while (s1 != s2); /* it changed: retry */
    return true;
}

```

The reader never blocks the writer - essential when the writer is a high-rate ISR that must not be delayed. The cost is that the reader may retry, so it must not be used where the reader has a hard deadline and the writer is very frequent.

18.6 Reentrancy

A function is **reentrant** if it can be called again before the first call returns - from an interrupt, or from another task - and still behave correctly. Firmware breaks this constantly and by accident.

Makes a function non-reentrant	Fix
static local variables holding state	Pass state in through a context pointer
Writing to globals without protection	Ownership, atomics or a critical section
Calling a non-reentrant library function (<code>strtok</code>, <code>rand</code>, the non-<code>_r</code> newlib functions, <code>printf</code> on a shared buffer)	Use the <code>_r</code> variants, or serialise access
Using a shared hardware peripheral	One owner per peripheral, or a mutex around the driver
Shared <code>errno</code> in a multi-task build	Build the library thread-safe, or return errors explicitly

18.7 Multi-core, briefly

Dual-core microcontrollers (Cortex-M4 plus M0+, M7 plus M4, or an M-class core beside an A-class one) add true parallelism, and with it: cache coherency is not automatic, so shared buffers live in non-cacheable memory or are maintained by hand; hardware semaphores or mailboxes are provided for mutual exclusion because disabling interrupts on one core does nothing to the other; and startup is asymmetric, with one core releasing the other. Treat the interface between cores as a **message-passing protocol with a versioned structure**, not as shared globals - the debugging cost of getting that wrong is out of proportion to the effort of doing it properly.

18.8 Finding races before they find you

Concurrency review checklist

- ☐ Every shared variable has a documented owner and access mechanism.
- ☐ Every ring buffer states its producer and consumer count.
- ☐ No multi-word update happens outside a critical section, an atomic handover, or a seqlock.
- ☐ The longest critical section is known, measured, and short.
- ☐ No blocking call happens in an ISR.
- ☐ Every `static` local in a shared function is justified in a comment.

- ☐ Stress tests run with interrupts at their real production rates, not at debug rates.

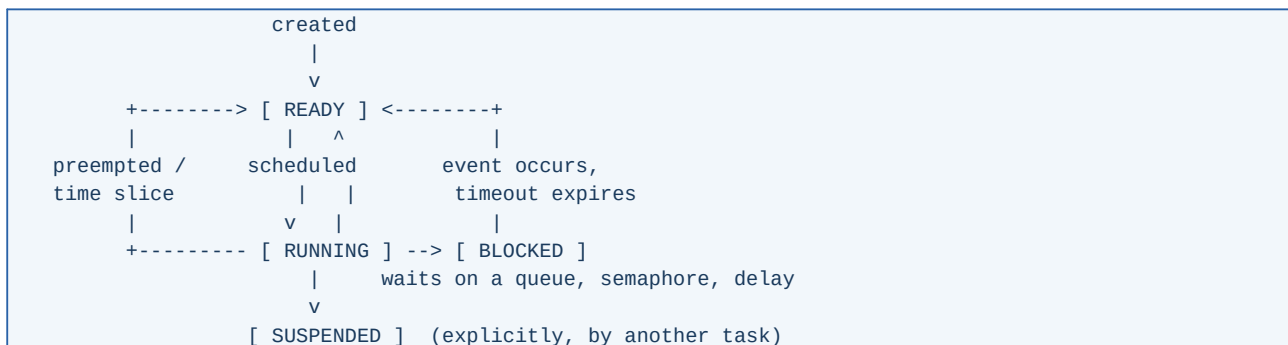
Exercises

- 1 Reproduce the lost-update bug: increment a counter from an ISR at 10 kHz and from the main loop, and compare with the expected total.
- 2 Implement the ring buffer above and prove it never loses or duplicates a byte across a million transfers at full UART rate.
- 3 Instrument `irq_lock/irq_unlock` with the cycle counter and find your longest critical section.
- 4 Write a seqlock for a multi-field measurement struct and verify the reader never sees a mixed pair of old and new fields.
- 5 Take any function in your codebase and determine, by inspection, whether it is reentrant. Most teams find at least one surprise.

19. RTOS Fundamentals: Tasks, Scheduling and Synchronisation

An RTOS gives you the illusion that several sequential programs run at once, each with its own stack, each able to block while waiting. That illusion is worth real RAM and real complexity, and it is the right trade whenever some work is long and some work is urgent.

19.1 Tasks and the scheduler



Only one task runs at a time on one core. A blocked task consumes no CPU - which is the entire point, and the difference from a polling loop.

The scheduling policy in every common embedded RTOS is **fixed-priority preemptive**: the highest-priority ready task runs, immediately, and keeps running until it blocks or a higher-priority task becomes ready. Equal-priority tasks share the CPU by time slicing if that is enabled - and relying on time slicing is usually a sign that two tasks should have been one.

Design decision	Guidance
How many tasks?	As few as possible. Each costs a stack, a TCB, and a new set of interactions. Five to fifteen is typical for a product; forty is a smell
What is a task?	One activity with one rate or one blocking source: a sensor pipeline, a comms link, a control loop. Not 'one per feature'
Priority assignment	Shorter deadline gets higher priority (rate-monotonic, Chapter 20). Never assign by importance - importance is not a timing property
Stack size	Measure, do not guess: <code>-fstack-usage</code> , high-water marks, and a margin (Chapter 24)
Static or dynamic creation	Create everything statically at startup. A system that can fail to create a task at run time has a failure mode with no good handling

19.2 The synchronisation primitives, and what each is for

Primitive	Purpose	Use it for	Never
Binary semaphore	Signalling: one event, one waiter	ISR tells a task 'data arrived'	Mutual exclusion - it has no priority inheritance
Counting semaphore	Counting resources or events	A pool of N buffers; counting interrupts	As a queue with payload
Mutex	Mutual exclusion with priority inheritance	Protecting a shared peripheral or data structure	From an ISR - an interrupt has no priority to inherit
Queue / message queue	Transferring data with ownership	Producer to consumer, the default IPC	Passing pointers to stack data

Primitive	Purpose	Use it for	Never
Event flags / event groups	Waiting for a combination of conditions	'Wi-Fi up AND time synced'	As a substitute for a queue of work
Task notification	The fastest one-to-one signal (a word in the TCB)	High-rate ISR-to-task signalling - several times faster than a semaphore	When several tasks must wait on the same event
Stream / message buffer	Byte or message streams between one writer and one reader	UART and network data paths	With multiple writers, without external locking

```

/* The canonical ISR-to-task handover in FreeRTOS. */
void DMA1_Stream0_IRQHandler(void)
{
    BaseType_t woken = pdFALSE;
    clear_dma_flags();
    vTaskNotifyGiveFromISR(h_adc_task, &woken);
    portYIELD_FROM_ISR(woken);    /* switch NOW if that task is */
}                                /* higher priority than this */
                                /* task - otherwise the wake */
void adc_task(void *arg)        /* is delayed by up to a tick */
{
    for (;;) {
        if (ulTaskNotifyTake(pdTRUE, pdMS_TO_TICKS(100)) == 0u) {
            handle_timeout();    /* every wait has a timeout */
            continue;
        }
        process_block();
    }
}

```

Two details carry most of the value: `portYIELD_FROM_ISR` turns a 'within one tick' latency into 'immediately', and the timeout on the wait converts a silent hang into a handled, loggable event.

19.3 Priority inversion, and why mutexes are special

```

H (high)    .....#####deadline missed#####
M (medium)  .....XXXXXXXXXXXXXXXXX
L (low)     ##LOCK##.....UNLOCK
            ^         ^             ^
            |         |             +-- H finally runs
            |         +-- H needs the lock L holds, so H blocks;
            |         then M (which needs no lock) preempts L
            +-- L takes a mutex H will also need

```

Unbounded inversion: H waits for L, but L cannot run because M is running - and M may run for an arbitrarily long time.

This is the failure that famously reset the Mars Pathfinder lander repeatedly on the surface of Mars, and it was fixed by enabling priority inheritance on one mutex.

Priority inheritance solves it: while a high-priority task waits on a mutex, the owner is temporarily raised to that priority, so M cannot preempt it. This is why a mutex is not merely a binary semaphore, and why you must not use a semaphore for mutual exclusion. **Priority ceiling** is the stronger alternative used in certified systems: the owner is raised immediately to the highest priority of any task that can take the lock.

Hazard	Condition	Prevention
Deadlock	Two tasks each hold one lock and want the other	A global lock ordering, obeyed everywhere; or never hold two locks
Livelock	Tasks keep retrying and never progress	Backoff, or a queue instead of retry
Starvation	A low-priority task never runs	Bound the CPU use of high-priority work; check with the analysis of Chapter 20

Hazard	Condition	Prevention
Unbounded inversion	Mutual exclusion without inheritance	Use mutexes, not semaphores, for locks
Death by tick	Everything wakes on the same tick	Offset periodic tasks deliberately

19.4 RTOS rules that prevent most defects

RTOS discipline

- ☐ Every blocking call has a timeout, and the timeout path is implemented, not a `TODD`.
- ☐ ISRs use only the `FromISR` API, and only at or below the RTOS's maximum syscall priority.
- ☐ No mutex is taken in an ISR; no `malloc` is called after startup.
- ☐ Locks are held for microseconds, never across an I/O operation.
- ☐ There is a documented lock order if more than one lock exists.
- ☐ Stack high-water marks are checked in production builds and reported.
- ☐ The idle hook sleeps the core; nothing busy-waits.
- ☐ Priorities are assigned by deadline and written down with their justification.

FREERTOS AND ZEPHYR, IN ONE PARAGRAPH EACH

FreeRTOS is a scheduler and primitives in a handful of C files; you keep your own build, drivers and structure. Set `configASSERT`, enable stack-overflow checking and `configUSE_TRACE_FACILITY` on day one, prefer static allocation (`heap_1`/static creation), and understand `configMAX_SYSCALL_INTERRUPT_PRIORITY` before assigning any interrupt priority. **Zephyr** is a whole platform: devicetree-driven drivers, a device model, subsystems for networking, storage, Bluetooth and settings, plus its own build system. It gives far more out of the box and asks you to work its way; it is the stronger choice for multi-vendor products and connected devices, and the heavier one for a tiny single-purpose board.

Exercises

- 1 Build a two-task producer-consumer with a queue and measure the end-to-end latency from ISR to consumer with a GPIO.
- 2 Replace a binary semaphore handover with a task notification and measure the difference in cycles.
- 3 Construct a priority inversion deliberately (three tasks, one mutex, a semaphore in place of the mutex) and observe the missed deadline. Then switch to a mutex and observe the fix.
- 4 Create a deadlock with two mutexes taken in opposite orders, then impose a lock order and prove it cannot recur.
- 5 Print every task's stack high-water mark after an hour of stress testing and adjust the allocations.

20. Real-Time Analysis: Proving the Deadlines Are Met

'It seems fast enough' is not an engineering claim. This chapter turns timing into arithmetic: measure the worst-case execution time of each activity, add the interference it suffers from everything of higher priority, and compare the result with the deadline. The technique is standard, takes an afternoon, and is the difference between a system that works and one that works so far.

20.1 Vocabulary, precisely

Term	Symbol	Definition
Period	T	How often the activity is released
Deadline	D	By when it must be finished (often $D = T$)
Worst-case execution time	C	Longest CPU time it can need, alone, with no interference
Response time	R	From release to completion, including interference and blocking
Blocking	B	Time spent waiting for a lower-priority task that holds a resource
Jitter	J	Variation in release or completion time
Utilisation	U	Sum of C/T over all activities: the fraction of the CPU consumed

20.2 Measuring C: what to trust

- **Toggle a pin** at the start and end of the activity and measure the maximum pulse width over a long, adversarial run. This is the most trusted number in practice because it includes everything - cache misses, flash wait states, the lot.
- **Read the DWT cycle counter** at entry and exit and keep a high-water mark in a variable you can inspect. Costs a few cycles, works in the field, and turns 'worst case' into a number your telemetry reports.
- **Stress the worst path deliberately.** The worst case is rarely the typical one: full buffers, maximum message length, the error branch, the first call after a cache flush, the sensor returning its longest response.
- **Beware measurement bias.** Measured maxima are a lower bound on the true worst case. Add margin - 1.5x to 2x is common practice - or use static WCET analysis if you are certifying (Chapter 33).

20.3 The quick check: utilisation bounds

$$\text{Utilisation } U = \sum_i C_i / T_i$$

Rate-monotonic bound (Liu and Layland), n tasks:

$$U \leq n (2^{1/n} - 1) \quad n=2: 0.828 \quad n=3: 0.780$$

$$n=5: 0.743 \quad n \rightarrow \infty: 0.693$$

If U is below the bound, the task set IS schedulable under rate-monotonic priorities (shortest period = highest priority).

If U is above it, the set MAY still be schedulable - use response time analysis. Only $U > 1.0$ is definitely impossible.

The famous 69% figure is a sufficient condition, not a limit: real systems routinely run above it and meet every deadline - they just need the exact analysis below rather than the quick one.

20.4 Response time analysis, worked end to end

A real system: two interrupt handlers and three tasks, on one core. Interrupts always preempt tasks, so they enter the analysis as the highest-priority activities.

Activity	C (ms)	T (ms)	D (ms)	Priority
UART receive ISR	0.005	0.1	0.1	Interrupt (highest)
DMA complete ISR	0.010	1.0	1.0	Interrupt
Control task	0.2	1.0	1.0	1 (highest task)
Sensor task	2.0	10.0	10.0	2
Comms task	10.0	50.0	50.0	3 (lowest)

$$U = 0.005/0.1 + 0.010/1 + 0.2/1 + 2/10 + 10/50$$

$$= 0.05 + 0.01 + 0.20 + 0.20 + 0.20 = 0.66$$

$$\text{RMS bound for } n = 5 \text{ activities: } 5(2^{(1/5)} - 1) = 0.743$$

$$0.66 < 0.743 \rightarrow \text{schedulable, no further work needed.}$$

But suppose a driver holds a mutex for 0.5 ms ($B = 0.5$) - the bound no longer applies, so compute the response time exactly:

$$R = C + B + \text{SUM}_{(\text{higher priority } j)} \text{CEIL}(R / T_j) * C_j$$

solved by iteration, starting from $R = C + B$.

Each **ceil** term counts how many times a higher-priority activity can be released inside the window R - the exact interference, not an average.

ITERATING FOR THE COMMS TASK

Start: $R = 10.0 + 0.5 = 10.5$. **Step 1:** $R = 10.5 + \text{ceil}(10.5/0.1)(0.005) + \text{ceil}(10.5/1)(0.010) + \text{ceil}(10.5/1)(0.2) + \text{ceil}(10.5/10)(2) = 10.5 + 0.525 + 0.11 + 2.2 + 4 = 17.335$. **Step 2:** with $R = 17.335$ the ceilings grow: $0.87 + 0.18 + 3.6 + 4$, giving **19.150**. **Step 3: 19.660. Step 4: 19.685. Step 5: 19.685** - converged. The comms task completes at worst 19.7 ms after release, against a 50 ms deadline: **schedulable with 61% margin**. Repeat for each task; the analysis fails only if R exceeds D , or if R grows past the deadline without converging - which is the arithmetic signature of an overloaded system.

WHAT THE ANALYSIS TEACHES EVEN WHEN IT PASSES

Look at where the comms task's 19.7 ms went: 10 ms of its own work, 0.5 ms of blocking, and **9.2 ms of interference** from things it does not control. Halving the control task's execution time would help it more than halving its own. That is the value of the calculation - it tells you which optimisation is worth doing, before you do any of them.

20.5 Jitter, and where it comes from

Source	Typical size	Reduction
Interrupt latency and nesting	Microseconds	Shorten critical sections; raise the priority of the jittery activity
Tick-based scheduling	Up to one tick (1 ms typical)	Trigger from a hardware timer instead of a tick
Software-triggered sampling	Whole loop periods	Timer-triggered ADC with DMA (Chapter 11)
Flash wait states and cache misses	Hundreds of cycles	Place the hot path in RAM/TCM (Chapter 30)
DMA contention on the bus	Cycles to microseconds	Separate the SRAM banks used by CPU and DMA
Garbage in the design: HAL_Delay inside a periodic path	Milliseconds	Remove it (Chapter 17)

20.6 Designing for overload

Every real system eventually receives more work than it can do. The question is not whether that happens but what it does when it does. Decide the policy explicitly:

- **Shed load:** drop the oldest samples, coalesce messages, reduce the log level. Always better than an unbounded queue.
- **Bound every queue** and count the drops. An unbounded queue turns a transient overload into an out-of-memory crash minutes later.
- **Degrade in a defined order:** telemetry before control, display before safety.
- **Detect and report:** a deadline miss counter in telemetry turns an invisible problem into a fixable one.
- **Keep the watchdog as the backstop**, not as the design (Chapter 28).

Exercises

- 1 Build the table of C, T and D for your own system - measured, not estimated. Most teams have never written it down.
- 2 Compute utilisation and check it against the rate-monotonic bound.
- 3 Run the response-time iteration for your lowest-priority task and report where its time goes.
- 4 Measure the jitter of your control loop over an hour with a scope in persistence mode, and compare it with your analysis.
- 5 Deliberately overload the system (double the message rate) and confirm the behaviour matches your intended degradation policy.

21. Inside an RTOS: Context Switching and Porting

Knowing what the scheduler does to your registers is what lets you debug a system that stops switching, size a stack correctly, and understand why an interrupt priority setting can break the whole kernel. This chapter opens the box.

21.1 What a context switch actually is

Task A running on PSP	Task B ready

1. Something makes B ready (ISR gives a semaphore, tick expires)	
2. Kernel sets PendSV pending (lowest priority exception)	
3. All higher-priority interrupts finish first	
4. PendSV entry: hardware pushes R0-R3,R12,LR,PC,xPSR onto A's stack	
5. PendSV handler pushes R4-R11 (and S16-S31 if FPU used)	
6. Saves A's PSP into A's TCB; loads B's PSP from B's TCB	
7. Pops R4-R11 for B	
8. Returns via EXC_RETURN: hardware pops B's 8-word frame	
9. B resumes exactly where it was preempted	

The hardware saves the caller-saved registers; the handler saves the callee-saved ones. That split is why a context switch on Cortex-M is roughly 80-200 cycles and why PendSV must be the lowest-priority exception - so it runs only when no real work is pending.

```
/* The core of every Cortex-M port, in ~15 instructions. */
__attribute__((naked)) void PendSV_Handler(void)
{
    __asm volatile (
        " mrs   r0, psp                \n" /* current task stack */
        " stmdb r0!, {r4-r11}          \n" /* save callee-saved */
        " ldr   r1, =pxCurrentTCB      \n"
        " ldr   r2, [r1]                \n"
        " str   r0, [r2]                \n" /* TCB->top_of_stack=r0 */
        " bl    vTaskSwitchContext      \n" /* pick the next task */
        " ldr   r2, [r1]                \n" /* new TCB */
        " ldr   r0, [r2]                \n"
        " ldmbia r0!, {r4-r11}          \n" /* restore */
        " msr   psp, r0                 \n"
        " bx    lr                      \n" /* EXC_RETURN -> task */
    );
}
```

Simplified - a real port also handles the FPU context and the BASEPRI critical section - but the shape is exactly this. A task's initial stack is a hand-built fake exception frame, so the very first switch into it is indistinguishable from a resume.

21.2 The tick, and doing without it

A periodic tick (typically 1 ms) drives timeouts, delays and time slicing. It is also, in a low-power design, a thousand pointless wake-ups per second. **Tickless idle** solves this: before sleeping, the kernel computes the time until the next timer expiry, programs a hardware timer for exactly that, sleeps deeply, and on waking corrects the tick count for the elapsed time.

Setting	Effect
Tick rate 1000 Hz	1 ms timing granularity, ~1-2% CPU overhead, worst power
Tick rate 100 Hz	10 ms granularity - too coarse for many timeouts; use a hardware timer for those
Tickless idle	Microamps instead of milliamps in idle; requires a low-power timer that keeps running in sleep (Chapter 29)
Time slicing on	Equal-priority tasks share the CPU; adds switches and non-determinism
Idle hook	Where you put <code>__WFI()</code> , stack checks and CPU-load measurement

21.3 The interrupt priority rule that breaks kernels

CONFIGMAX_SYSCALL_INTERRUPT_PRIORITY, EXPLAINED ONCE

The kernel protects its own data structures with `BASEPRI`, which masks interrupts **at or below** a chosen priority. An interrupt above that level therefore runs while kernel data is inconsistent - so it must never call any RTOS function, not even a `FromISR` one. The rules: (1) any ISR that calls a `FromISR` API must have a numerically **greater or equal** priority value - i.e. lower urgency - than `configMAX_SYSCALL_INTERRUPT_PRIORITY`; (2) any ISR above it gets no kernel calls at all, and in exchange gets latency the kernel cannot delay; (3) set the priority grouping so all bits are preemption bits, once, at boot. Enable `configASSERT` and FreeRTOS will catch violations for you - most teams discover several the first time they turn it on.

21.4 Debugging a running RTOS

Tool	What it shows
Thread-aware debugger view	Every task's state, priority, stack usage and call stack at a breakpoint - the first thing to configure
Runtime stats (<code>vTaskGetRunTimeStats</code>)	CPU share per task, from a high-resolution timer; finds the task that quietly consumes 40%
Stack high-water marks	Remaining headroom per task; the input to Chapter 24
Trace tools (SEGGER SystemView, Percepio Tracealyzer)	A timeline of every switch, ISR, semaphore and queue operation - the only practical way to see a priority inversion or an unexpected wake-up pattern
ITM/SWO printf	Low-overhead logging from tasks and ISRs (Chapter 25)
<code>configASSERT</code> plus a fault handler	Catches misuse at the moment it happens rather than three seconds later

Symptom	Usual cause
Everything stops after a while, no fault	A task deleted or blocked forever with no timeout; or a stack overflow that corrupted a TCB
A task never runs	A higher-priority task never blocks - check for a polling loop without a delay
Random corruption near a task's data	Stack overflow: raise the size, enable overflow checking, check the high-water mark
Works with the debugger attached, fails without	A timing race, or semihosting left enabled - it blocks forever with no debugger
Kernel asserts on an ISR call	Interrupt priority above <code>configMAX_SYSCALL_INTERRUPT_PRIORITY</code>
Latency spikes of exactly one tick	A wake-up missing <code>portYIELD_FROM_ISR</code>

21.5 Building a task's first stack

A newly created task has never run, so there is nothing to restore. The port solves this by **forging** the stack frame the hardware and the handler would have pushed, so the first context switch into it is indistinguishable from a resume.

```

top of the task's stack buffer
xPSR      = 0x01000000 (Thumb bit set - mandatory)
PC        = task_entry (bit 0 cleared here; xPSR carries Thumb)
LR        = task_exit  (what happens if the task returns)
R12, R3, R2, R1
R0        = the task's argument
R11..R4   = zeros (or a recognisable pattern for debugging)
<- the TCB's saved stack pointer points here

```

Two details cause real bugs: forgetting the Thumb bit in xPSR gives an immediate UsageFault on the task's first instruction, and leaving LR as garbage means a task that accidentally returns jumps somewhere random instead of into a defined handler.

MAKE A RETURNING TASK LOUD

Point that LR at a function that logs the task name and either deletes the task or resets. A task that falls off the end of its function is a defect, and the default behaviour in several RTOS ports is undefined behaviour - which presents as a corrupt system minutes later rather than as the obvious error it is.

21.6 What a port actually contains

Port element	Job
<code>portSAVE_CONTEXT / PendSV_Handler</code>	The register save and restore shown above
Critical section macros	<code>BASEPRI</code> set and restore, with nesting counts
Tick source setup	SysTick (or a low-power timer for tickless operation)
<code>portYIELD</code>	Set the PendSV pending bit, then <code>__DSB() / __ISB()</code>
Stack initialisation	The forged frame above
First-task start	Usually an SVC that switches to PSP and returns into the first task
FPU handling	Lazy stacking configuration and the extra 16 registers on switch
Optional MPU support	A per-task region set, reprogrammed on every switch

When an RTOS 'does not run at all' on a new part, the fault is almost always in this layer: the SysTick and PendSV priorities, the priority grouping, the FPU enable, or a `configCPU_CLOCK_HZ` that does not match reality. Check those four before suspecting the kernel.

21.7 Measuring the kernel's own cost

```

Context switch      80-200 cycles (about 0.5-1.2 us at 168 MHz)
Semaphore give/take 100-300 cycles each
Task notification   about half the cost of a semaphore
Queue send/receive  200-600 cycles, plus the copy
Tick handler        ~100 cycles, times the tick rate

At a 1 kHz tick, the tick alone costs 100,000 cycles/second
= 0.06% of a 168 MHz core - negligible on mains power,
and the dominant wake-up source on a battery (Chapter 29).

```

Measure these on your own part with the cycle counter rather than trusting the table: they vary by core, compiler and configuration, and the numbers feed directly into Chapter 20's analysis.

Exercises

- 1 Set a breakpoint in `PendSV_Handler` and single-step one full context switch, watching PSP and the stacked registers change.

- 2 Enable runtime statistics and find your system's real CPU load and its distribution across tasks.
- 3 Turn on tickless idle and measure the change in idle current.
- 4 Deliberately give an ISR a priority above the syscall limit, call a `FromISR` API from it, and observe what `configASSERT` reports.
- 5 Capture a trace with a trace tool during a burst of activity and identify the longest period during which your highest-priority task was ready but not running.

22. Protocols and Middleware: Framing, Parsing and Reliability

Between two devices there is always a protocol, and if you do not design it deliberately you will get one by accident - usually 'send the struct and hope'. This chapter covers the layers a device protocol needs, the framing that makes a byte stream into messages, and the reliability machinery that makes it survive a real link.

22.1 Layer the protocol, even a small one

```
application  commands, telemetry, versioned payloads
-----
transport    sequence numbers, ACK/NACK, retries, windowing
-----
framing      start marker, length, payload, CRC, end marker
-----
physical     UART / SPI / CAN / BLE characteristic / TCP socket
```

Each layer is testable on its own. A protocol written as one function that reads bytes and calls handlers cannot be tested, cannot be reused on a second link, and cannot be fuzzed.

The layering costs perhaps a hundred lines and repays it the first time the product gains a second interface.

22.2 Framing: finding message boundaries in a byte stream

Scheme	How	Trade-off
Length prefix	<code>[STX][len][payload][CRC16]</code>	Simple and compact; a corrupted length byte desynchronises the parser until a timeout
Delimiter with escaping (SLIP)	A reserved END byte; escape it inside the payload	Self-synchronising; payload can grow by up to 2x in the worst case
COBS	Removes all zero bytes from the payload, then uses 0x00 as an unambiguous delimiter	Best of both: self-synchronising with bounded overhead of one byte per 254
Fixed-size frames	Every message is the same length	Trivial parsing; wasteful unless messages really are uniform
Idle-time framing	A gap on the line ends the message (Modbus RTU)	No overhead; needs accurate timing and hardware support

COBS, ENCODED BY HAND

Payload `11 22 00 33`. COBS splits it at zero bytes and replaces each zero with the distance to the next one. The first run before a zero is `11 22` - two bytes, so the code byte is 3 (two bytes plus itself). The second run is `33`, one byte, code 2. The encoding is `03 11 22 02 33`, followed by a single `00` as the frame delimiter. The receiver can resynchronise at any time simply by scanning for `00` - no escape state, no ambiguity. Overhead is one byte per 254 bytes of payload plus the delimiter: **0.4% worst case**, against up to 100% for byte stuffing.

```
/* A parser that cannot be desynchronised for long: byte at a time, */
/* explicit states, a length guard, and a timeout that resets it. */
typedef enum { W_SYNC, W_LEN, W_PAYLOAD, W_CRC } pstate_t;

void parse_byte(uint8_t b, uint32_t now_ms)
{
    if (now_ms - last_byte_ms > FRAME_GAP_MS) st = W_SYNC; /* resync */
    last_byte_ms = now_ms;

    switch (st) {
        case W_SYNC:    if (b == STX) { st = W_LEN; } break;
```

```

case W_LEN:      if (b == 0u || b > MAX_PAYLOAD) { st = W_SYNC; }
                  else { len = b; idx = 0u; st = W_PAYLOAD; } break;
case W_PAYLOAD:  buf[idx++] = b;
                  if (idx == len) { st = W_CRC; crc_idx = 0u; } break;
case W_CRC:      /* accumulate, verify, dispatch or count an error */
                  break;
}
}

```

Three defences in one function: bound the length before trusting it, reset on an inter-byte gap, and never write past the buffer.

Most protocol vulnerabilities and most 'the link locks up' bugs are the absence of one of these.

22.3 Reliability on top of framing

Mechanism	Purpose	Design note
Sequence number	Detect loss, duplication and reordering	One counter per direction; 8 bits is enough for a small window
ACK / NACK	Confirm receipt	ACK the sequence number, not 'the last message' - they differ after a loss
Retransmission with backoff	Recover from loss	Timeout ~2-3x the measured round trip; add jitter; bound the attempts
Idempotent commands	Make a duplicate harmless	'Set output to 5' is idempotent; 'increment output' is not - prefer the first form
Keep-alive / heartbeat	Detect a silently dead peer	Cheap, and the only way to notice a half-open TCP connection
Flow control	Stop the sender overrunning the receiver	Credit-based windows, or a simple stop-and-wait for slow links

CHOOSING THE RETRANSMISSION TIMEOUT

A link whose round trip is 40 ms typical and 150 ms at the 99th percentile: a 100 ms timeout retransmits on 1% of messages unnecessarily, doubling load exactly when the link is already struggling. A 300 ms timeout (2x the tail) is the better starting point, with exponential backoff 300, 600, 1200 ms and a total budget of 5 attempts before declaring the peer unreachable - about 4.5 seconds. **Measure the round trip in the field and set the timeout from the tail, not from the average;** a fixed timeout copied from an example is one of the most common causes of self-inflicted congestion.

22.4 Message payloads that survive version skew

- **Version every message.** A one-byte protocol version in the header and a schema version in the payload cost nothing and make a phased rollout possible.
- **Prefer TLV or a schema encoding** (protobuf via nanopb, CBOR) over packed structs: new fields can be added without breaking old receivers, and unknown fields can be skipped.
- **Be explicit about endianness and width** in the wire format, and never rely on struct layout (Chapter 3).
- **Design the unknown-message rule now:** ignore and count, or reject with an error. Both are fine; unspecified is not.
- **Keep units in the field name** - `temp_mdegC`, `curr_uA`. Unit confusion between two teams is a real and expensive failure mode.

22.5 When to use a standard stack instead

Protocol	Cost on an MCU	Use when
MQTT over TCP/TLS	TLS handshake 20-40 KB RAM, ~1 s; small steady-state overhead	IP connectivity, cloud telemetry, many-to-one

Protocol	Cost on an MCU	Use when
CoAP over UDP/DTLS	Much lighter than TCP+TLS	Constrained links, sleepy nodes, request-response
HTTP(S) REST	Heaviest; simple to integrate	Occasional transactions where the device has power and RAM
Modbus RTU	Tiny	Industrial equipment expects it
Custom binary	Smallest and fastest	You own both ends and the link is not IP

Exercises

- 1 Implement COBS encode and decode and verify the worked example, then fuzz the decoder with random bytes and confirm it never writes out of bounds.
- 2 Write the byte-at-a-time parser and prove it resynchronises after you inject a corrupted length byte.
- 3 Measure your link's round-trip distribution and choose a retransmission timeout from the tail.
- 4 Add a version byte to an existing protocol and implement graceful handling of both older and newer peers.
- 5 Replay a captured byte stream into your parser as a regression test - the cheapest protocol test that exists.

23. Drivers, HALs and Portable Firmware

A driver is where hardware knowledge is concentrated so that the rest of the firmware does not need it. Good drivers make a codebase portable, testable and readable; bad ones spread register writes and blocking delays through the application until nothing can be changed or tested.

23.1 The three layers

```
+-----+
| service layer:  sensor_read_temperature_mdegC() |
| units, calibration, retries, state - no registers |
+-----+
| driver layer:  lis3dh_read_reg(), lis3dh_configure() |
| device protocol and semantics - no bus registers |
+-----+
| bus/HAL layer: i2c_write_read(bus, addr, ...) |
| peripheral registers, DMA, interrupts |
+-----+
```

Each layer depends only on the one below, through an interface it could not distinguish from a fake.

The device driver knows nothing about which I2C peripheral it uses; the bus layer knows nothing about accelerometers. That separation is what lets the same sensor driver run on a different MCU, and on a host PC under test.

23.2 API design rules

```
/* A driver instance is a struct the caller owns. No globals. */
typedef struct {
    const i2c_bus_t *bus;      /* injected dependency */
    uint8_t         addr;
    int32_t          offset_mg; /* per-unit calibration */
    drv_state_t      state;
} accel_t;

drv_err_t accel_init(accel_t *d, const accel_cfg_t *cfg);
drv_err_t accel_read (accel_t *d, vec3_t *out);          /* block */
drv_err_t accel_start(accel_t *d, accel_cb_t cb, void *ctx); /* async */
void      accel_deinit(accel_t *d); /* releases clocks and pins */

/* Document the context of every callback, in the header:
   cb is called from the DMA completion ISR: keep it short,
   do not block, do not call accel_* functions from inside it. */
```

A driver worth reusing

- ☐ No global state: everything lives in a caller-owned handle.
- ☐ Dependencies are injected (bus, clock, GPIO), not hard-coded.
- ☐ Every function returns an explicit error code; none can hang.
- ☐ There is a blocking API and a non-blocking or DMA API, and the docs say which contexts each may be called from.
- ☐ It has a `deinit` that truly releases everything - the low-power story depends on it (Chapter 29).
- ☐ It compiles for the host with a fake bus, and has unit tests (Chapter 26).
- ☐ The datasheet revision it was written against is named in a comment.

23.3 The vendor HAL question

Use the vendor HAL for	Write your own for
Complex peripherals where errata handling is embedded: USB, Ethernet, SDMMC, USB host	Peripherals you use in a timing-critical path: GPIO, timers, SPI/I2C/UART in production data paths
Bring-up and prototyping - get to first light fast	Anything you must certify, audit or unit-test thoroughly
Parts of the chip you touch once at boot	Anything called at kHz rates, where HAL overhead and blocking APIs dominate

THREE HAL HABITS THAT CAUSE REAL PROBLEMS

`HAL\_Delay()` in application code - a blocking busy-wait that silently depends on the SysTick interrupt still running, and hangs forever if called from an ISR. **Blocking transfer APIs** in a data path, which serialise everything behind the slowest device. **Weak callback overriding**, where a misspelled function name means your callback is never called and nothing warns you. Wrap the HAL behind your own interface, and these become one-line problems instead of codebase-wide ones.

23.4 Portability without ifdef sprawl

`#ifdef` chains inside driver logic are the reason firmware cannot be ported. The alternative is a single interface header plus one implementation file per platform, chosen by the build system:

```
/* port/gpio.h - the interface, no platform types anywhere */
typedef struct gpio_pin gpio_pin_t; /* opaque, per-platform */
void gpio_write(const gpio_pin_t *p, bool level);
bool gpio_read (const gpio_pin_t *p);

/* port/stm32/gpio.c, port/nrf/gpio.c, port/host/gpio.c */
/* The host version records writes into an array so tests can */
/* assert on them - the same trick that makes CI possible. */
```

Zephyr formalises exactly this with its device model and devicetree: drivers implement an API struct, and the board's hardware description selects the instances. If portability across silicon matters to your product, adopting that structure - with or without Zephyr - is the decision that makes it possible.

23.5 A complete driver, end to end

The abstract rules become concrete quickly. Here is the full shape of a production driver for a register-based I2C sensor, with everything the checklist demands.

```
/* --- sensor.h : the interface, no vendor types leak out ----- */
typedef struct {
    const i2c_bus_t *bus; /* injected: real or fake */
    uint8_t addr;
    int32_t gain_ppm; /* per-unit calibration from flash */
    int32_t offset;
    uint32_t err_count;
} sensor_t;

drv_err_t sensor_probe(sensor_t *d); /* check WHO_AM_I */
drv_err_t sensor_configure(sensor_t *d, sensor_rate_t r);
drv_err_t sensor_read_mdegC(sensor_t *d, int32_t *out);
drv_err_t sensor_sleep(sensor_t *d);
```

```
/* --- sensor.c : device semantics only, no peripheral registers */
#define REG_WHO_AM_I 0x0Fu
#define WHO_AM_I_VAL 0x33u
#define REG_TEMP_L 0x20u

drv_err_t sensor_probe(sensor_t *d)
{
```

```

uint8_t id = 0u;
drv_err_t e = i2c_read_reg(d->bus, d->addr, REG_WHO_AM_I, &id, 1u);
if (e != DRV_OK) { d->err_count++; return e; }
if (id != WHO_AM_I_VAL) { return DRV_NO_DEVICE; }
return DRV_OK;
}

drv_err_t sensor_read_mdegC(sensor_t *d, int32_t *out)
{
    uint8_t raw[2];
    drv_err_t e = i2c_read_reg(d->bus, d->addr, REG_TEMP_L, raw, 2u);
    if (e != DRV_OK) { d->err_count++; return e; }
    int16_t code = (int16_t)((uint16_t)raw[1] << 8 | raw[0]);
    *out = sensor_code_to_mdegC(code, d->gain_ppm, d->offset);
    return DRV_OK;
}

/* Pure, total, and unit-tested to death on the host: */
int32_t sensor_code_to_mdegC(int16_t code, int32_t gain_ppm,
                             int32_t offset)
{
    int64_t v = (int64_t)code * 1000 / 256;          /* LSB = 1/256 */
    v += (v * gain_ppm) / 1000000;                  /* calibration */
    return (int32_t)(v + offset);
}

```

Everything device-specific is here; nothing peripheral-specific is. The register numbers appear once. The conversion is a pure function that a host test can drive across its whole input range in microseconds, including the extremes where the sign and the rounding behave differently.

Design choice	Why
<code>i2c_read_reg</code> takes the bus as its first argument	Two sensors on two buses, or a fake bus in a test, need no code change
Errors increment a counter in the handle	Telemetry can report per-device health without global state (Chapter 28)
A separate <code>probe</code> step	Distinguishes 'wiring is wrong' from 'device is misbehaving' at bring-up
Calibration lives in the handle	Loaded from flash at init (Chapter 16); the driver has no idea where it came from
<code>sleep()</code> exists	The low-power story of Chapter 29 requires every device to have an off switch
No <code>printf</code> , no delays, no globals	It can run in any context and be tested anywhere

Exercises

- 1 Take one existing driver and remove every global from it, passing a handle instead. Note how the tests become possible.
- 2 Write a fake bus implementation and run your sensor driver on a host, asserting on the exact byte sequence it emits.
- 3 Find every `HAL_Delay` in your codebase and replace them with non-blocking state machines or RTOS delays.
- 4 Port one driver to a second MCU family by writing only a new port file, and measure how long it takes.
- 5 Document, for each driver, which functions may be called from an ISR. Most codebases have never written this down.

24. Memory: Budgets, Stacks, Heaps and Protection

A microcontroller's RAM is a fixed, small, unforgiving resource, and it is consumed by three things that are easy to underestimate: static data, stacks, and whatever the heap fragments into. Running out is not a graceful failure - it is silent corruption. This chapter is how to know, before it happens, that you will not.

24.1 Write the budget down

Region	Sized by	How to measure
<code>.text + .rodata</code>	Code and constants	<code>size</code> and the map file
<code>.data + .bss</code>	Globals and static buffers	<code>size</code> ; then <code>nm --size-sort</code> for the offenders
Stacks	Deepest call chain plus interrupt nesting	<code>-fstack-usage</code> , high-water marks, MPU guard
Heap	Dynamic allocation - ideally zero after init	Allocator statistics; peak, not current
DMA buffers	Transfer size x number in flight	Explicit, in the design
Reserve	The margin you keep for the next two years of features	Aim for 20-30% of RAM free at first release

24.2 Static allocation, and pools when you need flexibility

The rule in serious firmware is simple: **allocate everything at startup and never again**. When objects genuinely come and go, use a fixed-size block pool rather than a general heap - allocation becomes O(1), deterministic, and immune to fragmentation.

```
/* A pool allocator: 30 lines, deterministic, no fragmentation. */
typedef struct blk { struct blk *next; } blk_t;
static uint8_t pool_mem[N_BLOCKS][BLOCK_SIZE] __attribute__((aligned(8)));
static blk_t *free_list;

void pool_init(void)
{
    free_list = NULL;
    for (size_t i = 0; i < N_BLOCKS; i++) {
        blk_t *b = (blk_t *)pool_mem[i];
        b->next = free_list; free_list = b;
    }
}

void *pool_alloc(void) /* O(1), fails predictably */
{
    uint32_t k = irq_lock();
    blk_t *b = free_list;
    if (b != NULL) free_list = b->next;
    irq_unlock(k);
    return b; /* NULL when exhausted - handle it */
}
```

Give each subsystem its own pool: a burst of network packets then cannot starve the sensor pipeline, and exhaustion is diagnosable because you know which pool ran out.

WHY A HEAP FAILS EVEN WITH MEMORY LEFT

An 8 KB heap. The program alternately allocates 500-byte buffers and 100-byte records, then frees all the 500-byte buffers. The 100-byte records remain scattered every 600 bytes. Thirteen such pairs fit ($13 \times 600 = 7,800$ bytes), so after freeing the buffers the heap holds **6.9 KB free** - but in thirteen fragments of 500 bytes plus a 892-byte tail. A request for a 1 KB buffer **fails**, with 84% of the heap unused. This is not a leak and no code is wrong; it is the arithmetic of variable-size allocation with mixed lifetimes. Pools eliminate it by construction, which is why they are the standard answer in firmware.

24.3 Sizing stacks properly

Stack needed = deepest call chain (locals + saved regs + args)

+ interrupt nesting levels x per-level cost

+ margin

Per interrupt level on Cortex-M:

8 words hardware frame = 32 bytes

+ 17 words FPU frame if the handler uses floats = 68 bytes

+ the handler's own locals = varies

-fstack-usage emits a .su file per translation unit giving each function's frame size; a small script sums the deepest path through the call graph.

A WORKED STACK BUDGET

A task's deepest chain measures 1,024 bytes from the .su files. Three interrupt priority levels can nest; each costs 32 bytes of hardware frame plus about 96 bytes of handler locals, and one handler uses floats, adding 68 bytes: $3(32 + 96) + 68 = 452$ bytes. Subtotal 1,476 bytes. Add 30% margin for the compiler version changing and for the call chain you have not measured: 1,919 bytes, so allocate **2,048 bytes**. Then verify empirically: paint the stack with a pattern at boot, run the worst-case workload for an hour, and read the high-water mark. If it shows 1,900 used, your margin is gone and the number was optimistic somewhere.

```
/* Stack painting: fill at boot, scan later for the high-water mark */
#define PAINT 0xA5A5A5A5u
void stack_paint(uint32_t *from, uint32_t *to)
{ while (from < to) *from++ = PAINT; }

size_t stack_unused(const uint32_t *base, size_t words)
{
    size_t n = 0;
    while (n < words && base[n] == PAINT) n++; /* untouched words */
    return n * sizeof(uint32_t);
}
```

Report this in telemetry. A device whose stack headroom is shrinking release by release is telling you something before it crashes.

24.4 Catching overflow instead of debugging corruption

Technique	Detects	Cost
MPU guard region below each stack	The write that overflows, at the instant it happens - a MemManage fault with the faulting address	One MPU region; the best option when available
ARMv8-M stack limit registers (MSPLIM/PSPLIM)	Overflow in hardware, precisely	Free on Cortex-M23/M33/M55
Canary word below the stack, checked periodically	Overflow, but late	A few cycles per check

Technique	Detects	Cost
RTOS stack-overflow hook	Overflow at context-switch time	Small; enable it in every build
Painting plus high-water marks	How close you are - before the crash	One boot-time fill

THE SIGNATURE OF STACK OVERFLOW

Symptoms that look unrelated: a variable in another module changes value by itself; a task's local becomes garbage after a function call; the system faults in a place that has never changed; behaviour differs between debug and release builds because frame sizes differ. Before spending a day on any of those, **check the stacks**. It is the single most common cause of inexplicable corruption in firmware.

24.5 Placing memory deliberately

Region	Property	Put this there
TCM / CCM RAM	Zero wait states, no DMA access on some parts	Hot ISR code and data, the stack
Normal SRAM bank A	Shared with DMA	DMA buffers, network descriptors
Normal SRAM bank B	Contended less	Task stacks, to avoid competing with DMA
Non-cacheable region (MPU)	No cache maintenance needed	DMA buffers on Cortex-M7
.noinit	Not cleared by startup	Crash records, reset reason, boot counters
External SDRAM	Large and slow	Frame buffers, big datasets - never the stack

Exercises

- 1 Produce your project's full RAM budget from the map file and account for every kilobyte.
- 2 Paint the stacks, run your worst-case workload, and report the high-water mark for each task.
- 3 Set up an MPU guard region under one stack and deliberately overflow it; confirm you get a MemManage fault with a useful address.
- 4 Replace one `malloc/free` usage with a pool and compare worst-case allocation latency.
- 5 Reproduce the fragmentation example numerically with your allocator and record the largest allocatable block over time.

PART IV

Making It Work

Debugging with a probe, a pin and a fault handler; testing firmware without hardware and with it; building reliability in through watchdogs and defined failure behaviour; engineering the power budget; and making it fast enough with measurements rather than opinions.

25. Debugging: Probes, Traces and Post-Mortems

Firmware debugging is different because you cannot print your way out of every problem: printing changes the timing that caused the bug. The toolkit is a debug probe, a non-intrusive trace channel, an oscilloscope or analyser, and a fault handler that captures evidence when nobody is watching.

25.1 The debug hardware

Interface	Pins	Gives you
SWD	SWCLK, SWDIO (+ GND, reset)	Halt, step, breakpoints, memory access - the baseline
JTAG	5 pins	The same, plus boundary scan and daisy chaining
SWO / ITM	+1 pin (SWO)	printf-class output at MHz with minimal CPU cost, plus event timestamps
ETM / TPIU trace	+4-5 pins	Instruction trace: exactly which instructions ran before the crash. Priceless for rare bugs, needs a trace-capable probe
RTT (SEGGER)	None - uses SWD	Bidirectional console through a RAM ring buffer at ~1 us per line

KEEP THE DEBUG PORT ON THE PRODUCTION BOARD

Bring SWD out to test points or a small connector, even on the final product, and decide deliberately (Chapter 32) whether to lock it in the field. Boards with no debug access cost days at every bring-up, and a returned unit with no debug port is a unit you cannot diagnose.

25.2 Halting versus not halting

A breakpoint stops the core - but not the world. Timers keep counting, DMA keeps transferring, a motor keeps spinning, a watchdog keeps counting down, and the peer at the other end of the link times out. Most parts have debug-freeze bits that stop selected peripherals when the core halts: set them for the watchdog and the timers you care about, and know that the rest of the system does not pause.

Situation	Use
Reproducible bug, timing not critical	Breakpoints and stepping
Bug disappears when you stop	RTT/ITM logging, or a GPIO toggle plus a scope
Corruption of a specific variable	A data watchpoint - halt on write to that address, and read the call stack. This finds in minutes what code reading takes days to find
Rare crash, cause unknown	Instruction trace (ETM), or a post-mortem crash record
Field failure	Crash record in <code>.noinit</code> plus telemetry (Chapter 28)
Timing anomaly	Two GPIOs and a logic analyser

25.3 Logging that does not distort what it measures

```

/* Deferred formatting: the hot path stores IDs and arguments, */
/* the host does the printf. ~20 cycles instead of ~2,000. */
#define LOG_ID_RX_OVERRUN 0x0412u
log_binary(LOG_ID_RX_OVERRUN, count, DWT->CYCCNT);

/* Levels, rate limiting, and a module tag - all compile-time */
/* removable so a release build pays nothing for debug logging. */
LOG_DBG(MOD_I2C, "addr=%02x nack", addr);
LOG_WRN_RATE(1000, MOD_RF, "link margin low");

```

Three rules: never format inside an ISR, never log inside a critical section, and rate-limit anything that can repeat at kHz rates - a logger that floods its own buffer during a fault storm destroys the evidence you need.

25.4 Decoding a hard fault by hand

A fault handler that captures the stacked frame turns a crash into an address. The frame is the eight words the hardware pushed (Chapter 9), on MSP or PSP depending on bit 2 of `EXC_RETURN`.

```

void HardFault_Handler(void)
{
    __asm volatile (
        "tst lr, #4      \n" /* which stack was in use? */
        "ite eq          \n"
        "mrseq r0, msp   \n"
        "mrsne r0, psp   \n"
        "b    fault_report \n");
}

void fault_report(uint32_t *frame) /* frame[6] = PC */
{
    crash.pc    = frame[6]; crash.lr  = frame[5];
    crash.psr   = frame[7]; crash.r0  = frame[0];
    crash.cfsr  = SCB->CFSR; crash.hfsr = SCB->HFSR;
    crash.mmfar = SCB->MMFAR; crash.bfar = SCB->BFAR;
    crash.magic = CRASH_MAGIC; /* in .noinit */
    NVIC_SystemReset();
}

```

READING ONE REAL CRASH RECORD

The captured record says PC = 0x08001234, LR = 0x08000ABD, CFSR = 0x00000082, MMFAR = 0x00000000. Decode CFSR from the bottom byte (the MemManage status): bit 1 = **DACCVIOL**, a data access violation, and bit 7 = **MMARVALID**, so MMFAR holds the faulting address - which is **0x00000000**. That is a write through a null pointer, caught by the MPU or by the unmapped region. Now `arm-none-eabi-addr2line -e firmware.elf 0x08001234` names the source line, and LR (with bit 0 masked, so 0x08000ABC) names the caller. **Two commands and four numbers turn 'it resets sometimes' into a line number.** Note that a BusFault may be *imprecise* (CFSR bit 10, IMPRECISERR), meaning the PC has moved on - set `SCB->ACTLR DISDEFWBUF` while debugging to make it precise.

25.5 A method, not a ritual

- **Reproduce it reliably first.** A bug you can trigger in ten seconds is a bug you will fix; one that appears twice a week is a research project. Invest in the reproduction before investigating.
- **Bisect.** `git bisect` over commits, or bisect the system: remove half the functionality, does it still happen?
- **Change one thing at a time**, and write down what you changed and what happened. Debugging without notes repeats itself.
- **Question the hardware.** Roughly half of bring-up bugs are electrical: a solder bridge, a missing pull-up, a brownout, an erratum. Measure before concluding.
- **Prefer evidence to reasoning.** A scope trace beats an argument about what the compiler emitted; the disassembly beats an argument about what the C means.

Symptom	Look here first
Resets randomly	Watchdog (why?), brownout, stack overflow, hard fault handler that resets silently
Works in debug, fails in release	Undefined behaviour, missing <code>volatile</code> , a timing race that <code>-O0</code> hid
Works with the debugger attached only	Semihosting enabled, timing changed, or a peripheral frozen by the debugger
Hangs after hours or days	Counter wrap (Chapter 10), a leak, queue exhaustion, a missing timeout
Corruption of unrelated data	Stack overflow, DMA into a stale buffer, an out-of-bounds write
Communication works then stops	Overrun flag never cleared, buffer never drained, ISR flag not acknowledged
Fails only in the cold or the heat	Crystal margin, ADC drift, flash timing, a marginal supply

Exercises

- 1 Set a data watchpoint on a variable and catch the code that writes it. Time how long that took compared with reading code.
- 2 Implement the fault handler above, cause a null-pointer write, and decode the record by hand as in the box.
- 3 Set up RTT or ITM logging and measure the cost per log line with the cycle counter.
- 4 Toggle a GPIO around your busiest ISR and photograph the analyser trace showing entry, exit and nesting.
- 5 Take a bug from your issue tracker and write down the five-step method you used. Most teams find they skipped 'reproduce reliably'.

26. Testing Firmware

The usual objection - 'you cannot unit-test firmware, it needs the hardware' - is true only of code that was written without testing in mind. Most firmware logic is protocol parsing, state machines, unit conversion, scheduling and error handling, none of which needs silicon. This chapter is how to get those under test on a host, and what to do about the part that genuinely needs hardware.

26.1 The firmware test pyramid

/\	manual / field trials	rare, slow, priceless
/ \	system tests on real HW	hours, few, end-to-end
/----\	hardware-in-the-loop	minutes, per feature
/ \	on-target integration	seconds, per driver
/-----\	HOST UNIT TESTS	milliseconds, hundreds

The base is where the volume must be: fast enough to run on every save, and running the same source that ships.

If the base is empty, every defect is found by the slowest, most expensive layer - which is the normal state of firmware projects and the reason schedules slip late.

26.2 Making firmware testable

```
/* Untestable: hardware, timing and logic in one function. */
void sensor_task(void) {
    HAL_I2C_Master_Transmit(&hi2c1, 0x38, cmd, 2, 100);
    HAL_Delay(50);
    HAL_I2C_Master_Receive(&hi2c1, 0x38, raw, 6, 100);
    temp = ((raw[0] << 8 | raw[1]) * 175) / 65536 - 45;
}

/* Testable: the arithmetic is a pure function, the I/O is behind */
/* an interface, and time is injected. */
int32_t sht_raw_to_mdegC(uint16_t raw) /* pure - test this */
{ return ((int32_t)raw * 175000) / 65535 - 45000; }

drv_err_t sht_read(const i2c_bus_t *bus, clock_t *clk, int32_t *out);
```

Splitting one function into a pure part and an I/O part is the single highest-value refactoring in firmware testing. The pure part gets exhaustive tests in microseconds; the I/O part gets a fake bus.

```
/* A fake bus: records what the driver did, replays what you want. */
static uint8_t canned[] = { 0x64, 0x8C, 0x00, 0x5A, 0x3C, 0x00 };

TEST(sht, reads_and_converts)
{
    fake_i2c_t fake; fake_i2c_init(&fake, canned, sizeof canned);
    int32_t mdeg;
    TEST_ASSERT_EQUAL(DRV_OK, sht_read(&fake.bus, &fake.clk, &mdeg));
    TEST_ASSERT_EQUAL(0x38, fake.last_addr); /* right address */
    TEST_ASSERT_EQUAL(2, fake.tx_len); /* right command */
    TEST_ASSERT_INT_WITHIN(50, 24000, mdeg); /* right answer */
}
```

This test runs on your laptop in a millisecond, catches a wrong register address, a wrong byte order and a wrong conversion, and needs no board. Unity, CppUTest, GoogleTest and Ceedling all work well; the framework matters far less than the seam.

26.3 The layers above

Layer	What it catches	How to build it
Host unit tests	Logic, parsing, conversions, state machines, error paths	The same C compiled for the host with fakes; run in CI on every commit
Sanitisers and fuzzing on host	Buffer overruns, undefined behaviour, parser crashes	Build the parser with ASan/UBSan and libFuzzer; feed random and mutated frames
On-target integration	Register-level driver behaviour, DMA, interrupts, timing	A test runner in the firmware, driven over a serial console; report results in a machine-readable form
Hardware in the loop	Real sensors and actuators, power cycling, signal injection	A fixture with relays, a programmable supply, a signal generator, a logic analyser, and a script
Soak and stress	Leaks, wrap-around, drift, wear, thermal	Days of continuous operation with telemetry; deliberately hostile input rates
Field trials	Everything you did not imagine	Staged rollout with crash reporting (Chapters 28, 31)

THREE TESTS WORTH WRITING BEFORE ANY OTHERS

(1) The ring buffer, exhaustively across the wrap boundary - it underpins every data path. **(2) The protocol parser**, fed with truncated, oversized, corrupted and interleaved frames - it is your attack surface and your resync path. **(3) The unit conversion and calibration maths**, against values computed by hand - it is what the customer actually sees. Between them these three catch a disproportionate share of real defects.

26.4 Static analysis and build hygiene

Tool	Finds
Compiler warnings at maximum	Most type, conversion and initialisation defects - and it is free
clang-tidy / cppcheck	Dead code, suspicious casts, null-pointer paths, resource leaks
ASan / UBSan (host builds)	Out-of-bounds, use-after-free, undefined behaviour - the ones that appear only at -O2 on target
MISRA/CERT checkers	Rule violations required by safety standards (Chapter 33)
Stack analysis (-fstack-usage, worst-case tools)	Overflow before it happens (Chapter 24)
Size regression tracking in CI	The commit that added 8 KB of printf float support

26.5 Continuous integration for firmware

A CI pipeline that pays for itself

- ☐ Builds every target and every configuration on every commit, with warnings as errors.
- ☐ Runs the host unit tests plus sanitisers, with coverage reported.
- ☐ Records flash and RAM usage per build and fails on an unexplained jump.
- ☐ Archives the ELF, the map file and the binary for every build - forever. You will need them to decode a field crash.
- ☐ Runs static analysis and reports new findings only, so the signal is not drowned.
- ☐ Flashes at least one real board and runs a smoke test (boot, self-test, one message).
- ☐ Produces a versioned, signed artefact ready for the update system (Chapter 31).

Exercises

- 1 Extract one pure function from a driver and write ten tests for it, including the boundary values.
- 2 Build a fake bus for one device and assert the exact transaction sequence.

- 3 Fuzz your protocol parser on the host for ten minutes with a sanitiser enabled. Most parsers fail within seconds the first time.
- 4 Add flash and RAM size tracking to CI and plot the last fifty commits.
- 5 Take the last three bugs you fixed and write a test that would have caught each. If a bug cannot be tested, that is a design finding.

27. Board Bring-Up

Bring-up is the day the schematic meets reality. Done systematically it takes an afternoon; done by plugging in and hoping, it can take weeks and end with a damaged board and no idea which of forty changes helped. This chapter is the systematic version.

27.1 Before power is applied

Firmware's schematic review

- ☐ Debug port (SWD/JTAG) is brought out, with ground, and not shared with a function that disables it.
- ☐ Boot mode pins are strapped to the mode you want by default.
- ☐ The reset line has a pull-up and is accessible to the probe.
- ☐ Every crystal has its correct load capacitors, and the part supports that crystal's drive level.
- ☐ Decoupling capacitors exist on every supply pin, and VDDA is filtered separately.
- ☐ I2C buses have pull-ups fitted, of a value matched to the intended speed.
- ☐ Test points exist on the signals you will need to probe - at least power rails, reset, clock output, and one spare GPIO per subsystem.
- ☐ Every pin's alternate function is checked against the pin-mux table, not assumed.
- ☐ Power sequencing requirements of every rail and every peripheral are satisfied.
- ☐ There is a way to measure current: a sense resistor or a jumper in the supply.

THE THREE SCHEMATIC ERRORS FIRMWARE DISCOVERS

A peripheral wired to a pin whose alternate-function table does not offer that peripheral - no firmware can fix it. An I2C device at an address that collides with another on the same bus. And a signal that needs a pull-up in hardware because the internal one is too weak for the bus capacitance. Catching these at review costs minutes; catching them at bring-up costs a board spin.

27.2 The bring-up sequence

Step	What to do	Stop if
1. Power only	Apply current-limited power with no firmware. Measure every rail and the total current	Any rail is wrong, or current is far from expected - do not continue
2. Probe contact	Connect the debug probe; read the device ID and erase the flash	The probe cannot see the target: check SWDIO/SWCLK, reset, ground, and boot pins
3. Blink	Flash the simplest possible program on the internal oscillator	Nothing runs: check the vector table, the linker script and the reset handler (Chapter 6)
4. Clocks	Bring up the crystal and PLL; output the clock on MCO and measure it	The frequency is wrong - fix it before anything timing-dependent
5. Console	Get UART or RTT logging working - your eyes for everything after this	Garbage on the line: recheck the baud arithmetic (Chapter 12)
6. One peripheral at a time	Power domain, clock, pins, then a minimal transaction with a scope on the bus	Never bring up two new things at once
7. Integration	Run the real firmware; then soak it	Record everything in the bring-up log

KEEP A BRING-UP LOG PER BOARD

A dated, numbered log per physical board - what you measured, what you changed, which rework is fitted - is worth more than any tool. Boards diverge: board 3 has a bodge wire, board 5 has the old regulator. Without a log, a result from one board gets applied to another and the resulting confusion can consume days. Write the board number on the board.

27.3 Reading hardware symptoms

Symptom	Likely hardware cause	How to confirm
Debugger cannot connect	Wrong boot pins, no reset, low supply, or firmware that disabled SWD pins	Hold reset low while connecting ('connect under reset')
Runs, then resets when a load switches	Supply dip / brownout	Scope the rail with the load switching; check the BOR flag
Bus works at low speed, fails at high	Rise time, capacitance, termination	Scope the edges; compute the RC (Chapter 12)
ADC reads noisy or offset	VDDA filtering, ground loops, source impedance	Short the input to ground and measure; check the reference
One board of ten misbehaves	Assembly defect, solder bridge, wrong part fitted	Compare against a known-good board; measure resistance
Works cold, fails warm	Crystal margin, thermal shutdown, marginal timing	Freeze spray and a heat gun - crude and decisive

When you hand a problem to a hardware engineer, bring three things: the measurement, the conditions that reproduce it, and what you have already ruled out. 'The I2C does not work' starts an argument; 'SCL rises in 900 ns with 4.7k pull-ups and 300 pF of bus capacitance, which exceeds the 300 ns limit at 400 kHz, and it works at 100 kHz' starts a fix.

27.4 First-light firmware: what to write before the drivers

Before any application code, build a small diagnostic image whose only job is to answer questions about the board. It pays for itself on the first day and stays useful for the product's whole life.

What first-light firmware should do

- ☐ Print the chip's unique ID, flash size, revision and the firmware build hash over the console.
- ☐ Report the measured system clock (by counting a known external reference, or by driving MCO) rather than the configured one.
- ☐ Offer a command to read and write any memory address - the fastest way to poke a peripheral by hand.
- ☐ Scan the I2C bus and list every address that acknowledges.
- ☐ Toggle any named pin, and read any named pin, from a console command.
- ☐ Report the reset cause and the crash record from the last run.
- ☐ Provide a 1 Hz heartbeat on an LED so 'is it running?' is answerable from across the room.

THE I2C SCANNER IS WORTH TEN MINUTES OF YOUR LIFE

Twenty lines that address every 7-bit address in turn and print the ones that ACK. It answers, immediately and without ambiguity: is the bus wired, are the pull-ups fitted, is the device powered, is its address strap what you think, and did you get the 7-bit versus 8-bit address convention right - which is the single most common I2C mistake, since datasheets quote both.

27.5 Measuring what you cannot see

Question	Cheapest reliable measurement
Is my code reaching this line?	Toggle a pin, look at the LED or the analyser - faster than any breakpoint
How long does this take?	Set a pin high on entry, low on exit; read the pulse width
Is the clock what I think?	MCO out to a frequency counter or a scope
Is the supply dipping?	Scope on the rail, AC-coupled, while the load switches
Is the bus signal clean?	Scope the edges; a logic analyser hides rise time
How much current, right now?	A sense resistor and a differential probe, or a current probe
Which task is running?	Two GPIOs encoding the task ID, set on every switch - a poor man's trace, and it works

Exercises

- 1 Write the bring-up checklist for your own board before it arrives, and time how long the sequence takes.
- 2 Measure the current of a bare board before and after your firmware starts; explain the difference.
- 3 Verify your system clock on MCO and compare with what `SystemCoreClock` claims.
- 4 Deliberately mis-strap a boot pin and document the exact symptom, so you recognise it later.
- 5 Bring up one peripheral end to end using only the reference manual - no vendor examples.

28. Reliability: Failing Well

Firmware runs unattended for years, on hardware that ages, with inputs nobody anticipated. It will encounter conditions you did not design for. Reliability is not the absence of failure - it is having decided, in advance, what happens when each kind of failure occurs.

28.1 A strategy for errors, not a habit

Error class	Example	Correct response
Transient, retryable	I2C NACK, dropped packet, ADC out of range once	Retry a bounded number of times with backoff; count it
Persistent, recoverable	Sensor not responding, radio module wedged	Reinitialise the subsystem; degrade the feature; report
Configuration or data corruption	Bad CRC on stored settings	Fall back to defaults, log loudly, keep running
Programming defect	Failed assertion, impossible state, fault exception	Capture evidence, reset - do not attempt to continue in an unknown state
Resource exhaustion	Queue full, pool empty, stack near limit	Shed load by policy (Chapter 20); count it; never block indefinitely
Environmental	Brownout, over-temperature, out-of-range supply	Enter the defined safe state; log; recover when conditions return

DEFINE THE SAFE STATE FIRST

Before writing error handling, answer one question: **what must be true of the outputs when the firmware does not know what is going on?** Motors stopped, heater off, valve closed, radio silent, brake engaged. Then make that state the one the hardware assumes on reset - pull-downs on enable lines, a watchdog that removes drive - so it is reached even if firmware never runs at all. Every other reliability mechanism in this chapter is a way of getting to that state promptly.

28.2 Assertions, and what they should do in the field

```
/* An assert that is useful in production, not just on the bench. */
#define ASSERT(cond) do { \
    if (!(cond)) { fault_record(__FILE__, __LINE__); reset(); } \
} while (0)

ASSERT(idx < ARRAY_SIZE(buf)); /* invariant: must be true */
if (i2c_read(...) != DRV_OK) { ... } /* expected error: handle it */
```

The distinction matters: **assert on invariants** - things that are impossible unless the code is wrong - and **handle expected errors** with code. Asserting on a NACK is wrong (buses are noisy); returning an error code for an out-of-range array index is also wrong (the program is already broken). Never compile assertions out of the release build unless a standard forces you to; instead make them cheap and make the failure path capture evidence.

28.3 Watchdogs as an architecture, not a line of code

```

task A --heartbeat--
task B --heartbeat--> supervisor: all bits set since last feed?
task C --heartbeat--
            |
            +--yes--> feed the watchdog, clear bits
            +--no---> do NOT feed: reset in <T>

```

Plus: window watchdog catches feeding too fast (a runaway loop), and an external supervisor IC catches a clock failure the internal watchdog would share.

A watchdog fed unconditionally from a timer interrupt proves only that interrupts still work - which is exactly the state a hung system is in.

28.4 Layered recovery

Level	Action	When
0	Retry the operation	Transient bus or link error
1	Reinitialise the peripheral or module	Repeated failures on one interface
2	Restart the subsystem (task, stack, module state)	The module cannot recover itself
3	Reset the device, preserving a crash record	Fault, failed assertion, watchdog
4	Boot the previous firmware image	The new image resets repeatedly (Chapter 31)
5	Enter a minimal recovery mode that can only be updated	Everything else failed - the device must remain reachable

BOOT-LOOP PROTECTION, CONCRETELY

Keep a boot counter in `.noinit` (or in the backup domain). It is incremented on every start and cleared only after the application has run successfully for, say, 60 seconds. If the counter reaches 3, the bootloader stops trying the current image and reverts to the previous one; at 5, it enters recovery mode. **Without this, one bad update bricks every unit that receives it**; with it, the failure costs one minute of downtime and reports itself. The 60-second criterion is the part people forget - an image that crashes after 40 seconds must not be treated as healthy.

28.5 Evidence from the field

What every shipped device should record

- ☐ Reset cause on every boot, from the hardware register.
- ☐ Boot count and uptime.
- ☐ A crash record: PC, LR, fault status registers, task name, and the firmware version (Chapter 25).
- ☐ Error counters per subsystem - bus errors, retries, CRC failures, queue drops, overruns.
- ☐ Stack high-water marks and free heap or pool minima.
- ☐ The last N log lines in a RAM ring buffer, preserved across a warm reset.
- ☐ A way to retrieve all of it: over the link, and locally through the debug port.

TWO ANTI-PATTERNS THAT HIDE FAILURES

The silent catch: `if (err) { /* ignore */ }`, which converts a diagnosable fault into a mysterious behaviour months later. **The infinite retry:** a loop that reinitialises a dead sensor forever, consuming the CPU and the battery while reporting nothing. Both come from the same instinct - keep going - and both destroy the information you need. Count, bound, report, and degrade.

28.6 Data integrity over years

- **CRC the firmware image at boot** and refuse to run a corrupted one (and see Chapter 32 for why a CRC is not a security measure).
- **CRC stored configuration** and keep an A/B copy (Chapter 16).
- **Consider a periodic RAM scrub** on long-running devices in harsh environments: read-verify critical structures, or keep a checksum over the configuration held in RAM.
- **Bound the age of state**: a cached sensor value must carry a timestamp, and consumers must reject stale data rather than acting on a reading from an hour ago.

Exercises

- 1 Write down your product's safe state, then check that the hardware reaches it with no firmware running.
- 2 Implement multi-task heartbeat watchdog feeding and prove that hanging any one task causes a reset.
- 3 Add a crash record in `.noinit` and verify it survives a reset and reports the right address.
- 4 Implement boot-loop protection with a 60-second health criterion and test it with a deliberately crashing image.
- 5 Audit your codebase for ignored return values - most projects find dozens.

29. Low-Power Firmware

On a battery-powered product, firmware **is** the power budget. The hardware sets the floor, but the decisions that determine whether a device runs for a week or five years - how often it wakes, how long it stays awake, what it leaves enabled, how often it transmits - are all firmware decisions.

29.1 The only equation that matters

$$\begin{aligned} \text{Energy} &= \text{SUM over states of } I_{\text{state}} \times V \times t_{\text{state}} \\ \text{Average current } I_{\text{avg}} &= \text{SUM} (I_{\text{state}} \times t_{\text{state}}) / T_{\text{cycle}} \\ \text{Battery life} &= \text{capacity_mAh} / I_{\text{avg_mA}} \quad (\text{then derate}) \\ \text{Dynamic CPU power} &\sim C \times V^2 \times f \quad (\text{scales with frequency}) \\ \text{Leakage} &\sim \text{constant with } f \quad (\text{dominates in deep sleep}) \end{aligned}$$

Because dynamic energy per instruction is roughly independent of frequency while leakage is not, the usual optimum is **race to sleep**: run fast, finish, and switch off - not run slowly to 'save power'.

Mode	Typical current	Retained	Wake sources	Wake time
Run	5-50 mA	Everything	-	-
Sleep (WFI)	1-5 mA	Everything; core clock stopped	Any interrupt	Instant
Stop / deep sleep	2-50 uA	RAM and register state	RTC, external pin, low-power UART/timer	5-100 us
Standby	0.3-3 uA	Backup domain only	RTC, wake pin, reset	Full reboot, ms
Shutdown / off	10-100 nA	Nothing	Wake pin, reset	Full reboot

A SENSOR NODE'S BATTERY LIFE, COMPUTED

Design: sleep at **3 uA**; wake once a minute for **20 ms at 5 mA** to sample; transmit once every ten minutes for **100 ms at 25 mA**. Per hour: sampling costs $60 \times 0.020 \text{ s} \times 5 \text{ mA} = 6 \text{ mA-s}$; transmitting costs $6 \times 0.100 \text{ s} \times 25 \text{ mA} = 15 \text{ mA-s}$; sleeping costs $0.003 \text{ mA} \times 3,600 \text{ s} = 10.8 \text{ mA-s}$. Total 31.8 mA-s per hour, so $I_{\text{avg}} = 31.8/3600 = \mathbf{8.8 \text{ uA}}$. On two AA cells (2,000 mAh) that is 226,000 hours - **26 years**, which means the limit is now the battery's own self-discharge and the datasheet's shelf life, not your firmware. On a 220 mAh coin cell it is **2.8 years**. Notice the shape of the budget: **the radio is 47% of it while being on for 0.017% of the time**. Halving the transmission rate buys more than any code optimisation ever will.

29.2 Where the microamps actually go

Low-power audit

- ☐ Every unused peripheral's clock is disabled - clock gating is the cheapest saving there is.
- ☐ Every GPIO is driven or pulled: a floating input oscillates and can cost hundreds of microamps.
- ☐ No pull-up fights an external driver (a pull-up to 3.3 V through a device pulling low costs V/R continuously).
- ☐ External devices are actually asleep, not merely idle - check their datasheets and their enable pins.
- ☐ Debug interfaces are disabled in the shipped build (they can hold the core awake and cost milliamps).
- ☐ The RTOS uses tickless idle; nothing polls in a loop.
- ☐ LEDs and indicators are off, dimmed, or duty-cycled.
- ☐ The regulator is in its low-power mode when the load is small.
- ☐ Brown-out detection is configured (it costs a little current, and prevents corrupt writes).

MEASURE WITH THE RIGHT INSTRUMENT

A handheld multimeter averages, cannot see a 100 us pulse of 25 mA, and its burden voltage can brown out the device. Use a current probe, a source-measure unit or a dedicated power analyser that samples at kHz-to-MHz rates and can span microamps to milliamps in one capture. **Then look at the waveform, not the number:** the surprises are always a peripheral that stayed on, a wake-up that lasted 10x longer than intended, or a retry storm - all invisible in an average.

29.3 Design patterns for low power

- **Batch and coalesce.** Ten messages sent together cost far less than ten separate wake-and-connect cycles, because the fixed cost of waking the radio dominates.
- **Let hardware work while the CPU sleeps.** DMA, timer-triggered ADC and hardware protocol engines run in stop mode on many parts, so the core wakes once per buffer instead of once per sample.
- **Wake on events, not on polls.** An interrupt-capable sensor with a threshold configured is worth more than any firmware optimisation of a polling loop.
- **Keep the wake-up short and deterministic.** Measure the time from wake to sleep and treat regressions in it as bugs - it is the number that multiplies by every wake-up for the product's life.
- **Size the capacitors for the pulses.** A coin cell has high internal resistance; a 25 mA transmit pulse can drop its voltage below the brown-out level. A bulk capacitor next to the radio is a firmware problem when it is missing.
- **Make the power profile visible:** a GPIO raised during each active phase turns the current trace into an annotated timeline.

29.4 Entering and leaving deep sleep, correctly

```
static void enter_stop_mode(void)
{
    /* 1. Put every external device to sleep first.          */
    sensor_sleep(&accel); sensor_sleep(&temp);

    /* 2. Park the pins: no floating inputs, no pull-up fighting */
    /*    an external driver, no output holding an LED on.      */
    pins_low_power();

    /* 3. Configure the wake sources you actually want.        */
    lptim_set_wakeup_ms(next_event_ms());
    exti_enable(ACCEL_INT_PIN);

    /* 4. Flush anything that must not be lost, then sleep.    */
    log_flush();
    __DSB();
    HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);

    /* 5. On wake, the clock is back on the internal RC: restore */
    /*    the PLL BEFORE any timing-dependent work (Chapter 8). */
    clock_restore();
    pins_restore();
}
```

Steps 2 and 5 are the ones that get omitted. Pin parking is often the largest single saving in a first design, and forgetting the clock restore produces a device that works but runs every timing calculation at the wrong frequency after its first sleep.

Wake source	Available in	Cost	Note
GPIO / EXTI line	All modes including standby	Zero	The cheapest wake; use sensor interrupt outputs
RTC alarm or wake-up timer	All modes	~0.3-1 uA	The standard periodic wake for long intervals

Wake source	Available in	Cost	Note
Low-power timer (LPTIM)	Stop modes	~1 uA	Millisecond-scale periodic wake; also the tickless RTOS timebase
Low-power UART (LPUART)	Stop modes	~1-2 uA	Wake on a received character without running the main clock
Analogue comparator / window watchdog on ADC	Stop modes	1-5 uA	Wake only when a signal crosses a threshold - no polling
Radio event	Stop modes on radio SoCs	Varies	The stack manages its own timing; do not fight it

29.5 Energy per operation, and what it implies

Charge for an activity $Q = I \times t$ (uA-seconds, or uC)

CPU awake 1 ms at 6 mA = 6 uA-s

ADC conversion burst, 1 ms = 1 uA-s

Flash page write, 5 ms at 8 mA = 40 uA-s

BLE advertising event = 20-40 uA-s

BLE connection + 1 notification = 100-300 uA-s

LTE-M attach and send = 5,000-50,000 uA-s

One year at 10 uA average = 87.6 mAh of budget in total.

Put the fixed costs of your own device into this table once, by measurement. Every subsequent design argument - how often to sample, whether to log, whether to send now or batch - becomes arithmetic.

SHOULD I LOG TO FLASH OR SEND IMMEDIATELY?

Suppose a reading is 8 bytes. Sending each one over BLE costs about 200 uA-s. Buffering 32 readings in RAM and writing them to flash once costs one page write, 40 uA-s, then a single connection to send the batch, 300 uA-s: **340 uA-s for 32 readings, or 10.6 uA-s each - nearly 20x cheaper** than 200 uA-s each. The cost is up to 32 readings of latency and the risk of losing the buffer on reset. That trade - latency and durability against energy - is the central design decision of a battery-powered connected product, and it is decided with this arithmetic, not by preference.

Exercises

- 1 Build the energy budget table for your product and compute the predicted battery life. Then measure it over 24 hours and compare.
- 2 Capture the current waveform of one complete wake cycle and annotate each phase.
- 3 Find one peripheral or pin that is costing you current when idle - almost every first design has one.
- 4 Enable tickless idle and measure the idle current before and after.
- 5 Halve the transmission rate and recompute the budget, then decide whether the product requirement really needs the original rate.

30. Performance and Optimisation

Firmware optimisation is unusual in that the answers are almost always structural - move the work to hardware, move the data closer, or do it less often - rather than clever code. The prerequisite in every case is measurement, because intuition about where cycles go is reliably wrong.

30.1 Measure first, and measure the right thing

Tool	Gives	Cost
DWT cycle counter around a region	Exact cycles, including stalls	About 10 cycles
GPIO toggle plus analyser	Timing and overlap of concurrent activities	2 cycles, no instrumentation of memory
Sampling profiler (SWO/ITM PC sampling)	Where the time goes across the whole program	Small, needs a trace-capable probe
Per-function high-water marks	Worst case, not average - which is what Chapter 20 needs	A few cycles and a word of RAM per function
Instruction trace (ETM)	Exactly what ran, in order	None on the target; needs the hardware

OPTIMISE THE WORST CASE, NOT THE AVERAGE

Application performance work targets throughput and averages. Firmware usually needs the **maximum**: the longest ISR, the slowest path through the parser, the worst-case control loop. A change that improves the average by 20% while doubling the worst case has made the system less schedulable, not faster.

30.2 Where the cycles actually go

Cost	Typical	What to do
Flash wait states on a cache miss	3-7 cycles per fetch	Enable the prefetch/cache; move hot code to RAM or TCM
Integer divide	2-12 cycles (Cortex-M4), or a library call on M0	Multiply by a reciprocal and shift; use powers of two
Float without an FPU	50-100+ cycles	Fixed point (Chapter 3), or enable the FPU you already have
double on a single-precision FPU	Hundreds of cycles	float literals with the <code>f</code> suffix; <code>-Wdouble-promotion</code>
Unaligned or byte-wise <code>memcpy</code>	3-8x the aligned cost	Align buffers to 4 bytes; copy words
Interrupt entry/exit	12-25 cycles each	Batch with DMA instead of per-item interrupts
A <code>printf</code> call	1,000-10,000 cycles	Deferred binary logging (Chapter 25)
Bus contention with DMA	Unpredictable stalls	Separate SRAM banks; schedule bulk transfers off the critical path

REPLACING A DIVISION

Converting an ADC count to millivolts: $mv = (code * 3300) / 4095$. The multiply is 1 cycle; the divide costs up to 12 on a Cortex-M4 and far more on an M0. Since $3300/4095 = 0.80586$, use a fixed-point reciprocal: $0.80586 \times 2^{16} = 52,812$, so $mv = (code * 52812) \gg 16$. The multiply-and-shift costs **2 cycles**. For code = 2048 the exact answer is 1650.2 and the approximation gives $(2048 \times 52812) \gg 16 = 108,158,976 \gg 16 = \mathbf{1650}$ - within one millivolt, at a sixth of the cost. Check the error at the extremes before shipping any such substitution, and add a comment giving the scale factor.

30.3 The order to try things

- 1. **Do it less often.** Halving a sample rate or a poll rate halves the cost exactly, and usually nothing notices.
- 2. **Move it to hardware.** DMA, a CRC unit, a hardware crypto accelerator, a timer doing waveform generation - typically a 10-100x saving, not a percentage.
- 3. **Fix the memory story.** Hot code and data into RAM/TCM, buffers aligned, caches enabled and maintained.
- 4. **Fix the algorithm.** A lookup table instead of a transcendental function; an incremental update instead of a full recomputation; a better data structure.
- 5. **Use the ISA.** SIMD/DSP instructions and CMSIS-DSP for filters, matrix maths and transforms - typically 2-4x on a Cortex-M4F.
- 6. **Only then, tune the code.** Inlining, unrolling, strength reduction, `restrict`, and letting the compiler do more with `-O2` and LTO.

TWO COMPILER SETTINGS WORTH MEASURING TODAY

Rebuild at `-O2` and at `-Os` and compare both size and your worst-case timings - the difference is often 20-40% either way, and which wins is program-specific. Then try `-flto`: it commonly saves another 5-15% of flash by removing cross-module dead code. Neither costs a line of source. Do this before hand-optimising anything, because it may make the hand optimisation unnecessary.

30.4 Optimising for size

Consumer	Typical	Remedy
<code>printf</code> with float support	8-15 KB	Integer-only formatting, or binary logging
Unused vendor HAL modules	5-30 KB	<code>-ffunction-sections -fdata-sections -WL,--gc-sections</code>
Lookup tables not marked <code>const</code>	RAM as well as flash	Add <code>const</code> - check they land in <code>.rodata</code>
C++ exceptions and RTTI	10-40 KB	<code>-fno-exceptions -fno-rtti</code>
Duplicated string literals and error text	1-5 KB	Error codes plus a host-side table; enable string merging
Debug logging in release	Varies	Compile-time level filtering so the strings are not linked at all

30.5 When to stop

Stop when the measured worst case meets the deadline with the margin you decided on in Chapter 20, and not before or after. Optimised firmware that nobody can read is a liability that outlives the performance problem it solved; leave a comment stating what was measured, what changed, and what the number became, so the next engineer knows whether the complexity is still earning its place.

Exercises

- 1 Instrument your five busiest functions with the cycle counter and rank them by worst-case time. Predict the ranking first.
- 2 Move one hot ISR into RAM/TCM and measure the change in its jitter, not just its average.
- 3 Replace one division in a hot path with a multiply-shift and verify the error bound across the full input range.
- 4 Build your project at `-O0`, `-Og`, `-Os`, `-O2` and `-O2 -flto`, and tabulate flash, RAM and worst-case loop time for each.
- 5 Find the three largest functions in your image and decide, with the map file open, whether each earns its size.

PART V

Shipping and Trusting It

Updating firmware in the field without bricking it, making the device resistant to attack, satisfying a safety standard, choosing embedded Linux when a microcontroller is not enough, and manufacturing units that leave the factory correct.

31. Bootloaders and Firmware Update

Any product that will exist for more than a year needs a way to change its firmware in the field. Getting this wrong is uniquely expensive: a failed update on a fleet of devices is not a bug report, it is a recall. This chapter is how to build an update path that cannot brick a device.

31.1 Memory layout is the design

A/B (dual-slot) layout on 512 KB of flash:

```

0x08000000 +-----+ bootloader    32 KB
            | immutable, signed, tiny | (never updated in field,
0x08008000 +-----+ or updated only via probe)
            | slot A: application    | 224 KB
0x08040000 +-----+
            | slot B: application    | 224 KB
0x08078000 +-----+
            | metadata / state       | 16 KB (2 sectors, A/B)
0x0807C000 +-----+
            | config / calibration   | 16 KB
0x08080000 +-----+
```

The cost of A/B is half your flash for the application. The benefit is that a failed update never leaves the device without a working image - which is why it is the default for any device you cannot physically reach.

Strategy	Flash cost	Risk	Use when
Dual slot A/B, boot either	2x app size	Lowest - rollback is instant	Connected products, the default
Single slot + external staging flash	App + external part	Low, but the copy step is a window of vulnerability	Internal flash is tight and an external QSPI part exists
Single slot, in-place update	1x	High - a power cut during the copy bricks the unit	Only with a recovery mode reachable by a technician
Delta / patch update	Adds patch-apply code	Medium - needs an exact source image match	Bandwidth is expensive: cellular, LoRa, large images

31.2 The update sequence, step by step

1. DOWNLOAD receive image into the inactive slot, in chunks, resumable, with a per-chunk checksum
2. VERIFY whole-image hash + SIGNATURE check (Chapter 32), version and hardware-compatibility check
3. STAGE write 'pending' state to metadata, atomically
4. RESET
5. BOOTLOADER re-verify the pending image; if bad -> keep the old
6. BOOT jump to the new image (Chapter 6)
7. CONFIRM the application proves health for N seconds, then writes 'confirmed'. Otherwise the next boot reverts.

Power can be lost between ANY two steps. At every point, exactly one image must be bootable, and the metadata must be atomic.

Step 7 is the one most often omitted, and it is the one that turns a bad release from a fleet-wide brick into a one-minute outage.

HOW LONG DOES THE DOWNLOAD TAKE?

A 200 KB image over a BLE link achieving 40 kB/s of application throughput takes $200/40 = 5$ seconds - fine. The same image over LoRaWAN at spreading factor 9, with a 115-byte payload every 10 seconds under a 1% duty cycle, needs $200,000/115 = 1,740$ messages = **4.8 hours**, which makes a full-image update impractical and delta updates or a firmware architecture that separates rarely-changed code essential. **Compute this number before choosing the radio**, because it constrains the product's whole maintenance strategy.

31.3 Rules for the bootloader itself

Bootloader design rules

- ☐ Small and simple enough to review completely - it is the code that can never fail.
- ☐ No dynamic memory, no RTOS, minimal peripheral use.
- ☐ Verifies the image's signature before every boot, not only after an update.
- ☐ Has its own watchdog, enabled before any long operation.
- ☐ Keeps a boot counter and reverts after N failed attempts (Chapter 28).
- ☐ Offers a recovery path that does not depend on the application - a button held at reset, a magic sequence on the UART, a DFU mode.
- ☐ Never trusts the length or the address in a downloaded header without bounds-checking them against the slot.
- ☐ Deinitialises everything before jumping (Chapter 6).
- ☐ Is itself updatable only through a mechanism you have tested to destruction - or not at all in the field.

THE FAILURE MODES THAT ACTUALLY HAPPEN

The new image boots but cannot connect to the network, so no further update can be delivered - fixed only by the confirm-and-revert step. The metadata sector is erased when power is lost mid-write - fixed by A/B metadata with sequence numbers and CRCs (Chapter 16). The image is compiled for a different hardware revision - fixed by a hardware compatibility field checked before staging. A rolled-back image cannot read the newer configuration format - fixed by making configuration migrations forward- **and** backward-compatible, which must be a rule from the first release.

31.4 Rolling out to a fleet

- **Stage the rollout:** 1% of devices, then 10%, then everyone, with an automatic halt on a rise in crash reports or a fall in check-ins.
- **Group by hardware revision and region** - a change that is fine on revision C may not be on revision A.
- **Randomise download times** so that ten thousand devices do not fetch a 200 KB image in the same second.
- **Keep the previous image available** for as long as any device might roll back to it, and keep every ELF and map file forever (Chapter 26).
- **Measure the outcome:** version distribution across the fleet, update success rate, and time-to-converge are the metrics that tell you the mechanism works.

Exercises

- 1 Draw your product's flash layout with real addresses and sizes, and check the slots fit with room for growth.
- 2 Implement the confirm-and-revert step and test it with an image that crashes after 30 seconds.
- 3 Power-cycle a device randomly during 100 updates and confirm it always boots something valid.
- 4 Compute the download time for your image over your slowest supported link.
- 5 Write the recovery procedure a field technician would follow, then have someone else follow it without your help.

32. Embedded Security

A shipped device is hardware in someone else's hands, with a debug port, a flash chip that can be desoldered, and a radio link anyone can listen to. Security for firmware is therefore not a feature but a set of properties: that only your code runs, that secrets stay secret, and that the device cannot be turned against its user or its network.

32.1 Threat model first

Attacker	Capability	Typical goal
Remote	Sends packets to your device or its cloud service	Take control, extract data, pivot into the network
Local network / radio	Sniffs and injects on the link	Eavesdrop, replay commands, impersonate a device
Owner with the device in hand	Debug port, flash reader, logic analyser	Unlock paid features, clone the product, extract keys
Supply chain	Access during manufacture or service	Insert modified firmware, steal keys, overbuild units
Well-funded laboratory	Decapsulation, fault injection, side-channel analysis	Extract keys from silicon

DECIDE WHAT YOU ARE DEFENDING, AND FROM WHOM

You cannot defend against every attacker, and pretending otherwise wastes the budget. Write down: what an attacker gains, what it costs them, and what a breach costs you. A connected door lock and a temperature logger deserve very different answers - but **both** need authenticated firmware updates, because a device that will run any image is a device that belongs to whoever reaches it first.

32.2 Secure boot: the root of trust

```

immutable ROM / locked bootloader  <- root of trust, in silicon
    | verifies signature of
    v
second-stage bootloader             <- your code, signed
    | verifies signature of
    v
application image                   <- signed, version-counted
    | verifies
    v
configuration, ML models, assets    <- signed or MAC'd

```

Each stage verifies the next BEFORE transferring control.
 A CRC proves the image is not corrupt. Only a SIGNATURE proves
 it is yours - the two are not substitutes.

Public key in the device, private key in an HSM you control. The device can verify but never sign, which is what makes it safe to ship the public key in every unit.

Element	Practice
Signature algorithm	ECDSA P-256 or Ed25519: 64-byte signatures, verification in milliseconds even without acceleration
Key storage on device	Public key in write-protected flash, or in OTP fuses; never in an updatable region
Private key	In an HSM or a cloud KMS with audit logging. It never exists on a build machine or in a repository

Element	Practice
Anti-rollback	A monotonic version counter in OTP or a protected region, so an old signed image with a known vulnerability cannot be reinstalled
Debug port	Locked in production via option bytes or fuses, with an authenticated debug unlock if you need field diagnostics
Readout protection	Enable it - but treat it as a speed bump: published attacks exist for many parts
TrustZone-M (ARMv8-M)	Separates secure and non-secure worlds on one core: keys and crypto in the secure world, application in the non-secure one

32.3 Cryptography that fits on a microcontroller

Need	Primitive	Cost on a Cortex-M4 at 100 MHz
Integrity	SHA-256	~10-20 cycles/byte in software; free with a hash accelerator
Confidentiality	AES-128/256 GCM or CCM	Software ~20-40 cycles/byte; hardware AES is common and much faster
Authenticity of firmware	ECDSA P-256 verify	10-100 ms in software; a few ms with acceleration - fine at boot
Key agreement	ECDH P-256 / X25519	Similar to ECDSA
Message authentication	HMAC-SHA256 or AES-CMAC	Cheap; the right tool for per-message authenticity
Randomness	Hardware TRNG	Essential - <code>rand()</code> seeded from a timer is not a key source

THE FIVE WAYS EMBEDDED CRYPTO IS GOT WRONG

(1) Rolling your own algorithm or protocol - use mbedTLS, wolfSSL, tinycrypt or the vendor's validated library. **(2) A predictable random source** - a device that generates its key from the boot-time counter generates the same key on every unit. **(3) The same key in every device**, so one extraction compromises the fleet. **(4) No replay protection** - encrypted commands recorded and resent still open the door; include a nonce or a counter. **(5) Verifying after acting** - checking a signature after the image has already been copied over the running one.

32.4 Attack surface in ordinary firmware

Hardening checklist

- ☐ Every parser validates length before use and cannot write past a buffer (Chapter 22).
- ☐ All external input - radio, USB, serial, files, sensors - is treated as hostile.
- ☐ No debug backdoors, hard-coded passwords, or 'temporary' unlock commands. They are always found.
- ☐ Secrets are not in the repository, not in logs, and not in the shipped image where `strings` can find them.
- ☐ The device authenticates the server and the server authenticates the device - mutual, not one-sided.
- ☐ Certificates and keys can be rotated in the field before they expire.
- ☐ Update images are signed and version-counted; downgrade is refused.
- ☐ Unused interfaces (JTAG, test pins, factory commands) are disabled in production builds.
- ☐ Third-party components are inventoried (an SBOM) and monitored for published vulnerabilities.
- ☐ There is a documented process for receiving a vulnerability report and shipping a fix.

Physical and side-channel attacks are a real category rather than a theoretical one: voltage and clock glitching can skip a comparison instruction - which is why a secure boot check should be written to resist a single skipped branch (double checks, random delays, non-trivial success values) - and power analysis can recover a key from a naive AES implementation. If the threat model includes a determined owner or a laboratory, use a part with hardened crypto or an external secure element, and do not attempt to invent the countermeasures.

32.5 Regulation is now part of the job

Security expectations for connected products have moved from good practice to legal requirement in several markets: baseline standards such as ETSI EN 303 645 (no universal default passwords, a vulnerability disclosure policy, secure update, secure storage of credentials), sector rules such as IEC 62443 for industrial systems and UNECE R155/R156 for vehicles, and horizontal legislation covering products with digital elements, which brings obligations for support periods, vulnerability handling and an SBOM. The engineering consequence is concrete: **plan for signed updates, a support window, a component inventory, and a way to receive and act on vulnerability reports, from the first design review** - retrofitting them to a shipped fleet is far more expensive.

Exercises

- 1 Write your product's threat model in one page: assets, attackers, and what you will and will not defend.
- 2 Implement signature verification in your bootloader and measure the boot-time cost.
- 3 Run `strings` over your shipped binary and see what it reveals.
- 4 Attempt to read your own device's flash with a debug probe after enabling readout protection, and document how far you get.
- 5 Generate an SBOM for your firmware and check every component against a vulnerability database.

33. Functional Safety and Certification

When firmware can hurt someone, an external standard decides how it must be developed, and an auditor asks for evidence. This chapter is what those standards require in practice, and which of their habits are worth adopting even when nobody is auditing you.

33.1 The landscape

Standard	Domain	Levels
IEC 61508	Generic industrial - the parent standard	SIL 1-4
ISO 26262	Road vehicles	ASIL A-D
IEC 62304	Medical device software	Class A/B/C
DO-178C	Airborne systems	DAL A-E
EN 50128 / EN 50657	Railway	SIL 0-4
IEC 60730	Household appliance controls	Class A/B/C
IEC 62443	Industrial cyber-security (often alongside safety)	SL 1-4

The level is determined by the **risk**, not by the technology: severity of harm, frequency of exposure, and the possibility of controlling the hazard by other means. That assessment happens before any code exists and determines everything that follows, including how much of the process below applies.

33.2 What certification actually demands

Requirement	In practice
Requirements, traceable	Every requirement has an identifier, links to design, code and tests, and the trace is maintained automatically rather than in a spreadsheet
Development process evidence	Reviews with records, a change process, configuration management, and a tool-qualification argument for your compiler and test tools
Coding standard	MISRA C or CERT C, enforced by a checker, with every deviation justified and approved
Structural coverage	Statement and branch coverage at lower levels; MC/DC at the highest - which drives how you write conditions
Hazard analysis	FMEA, fault-tree analysis: what can fail, what it causes, and what detects it
Diagnostics	Self-tests with a stated diagnostic coverage: RAM and flash tests, CPU register tests, watchdogs, plausibility checks
Freedom from interference	Memory protection between components of different criticality; timing partitioning
Documented safe state	What the system does when a fault is detected, and the time within which it gets there

THE FAULT TOLERANT TIME INTERVAL

Safety standards ask a question ordinary firmware never does: **how long may a fault persist before it becomes dangerous?** That interval sets the watchdog timeout, the self-test period, the sampling rate of plausibility checks, and the deadline analysis of Chapter 20. It turns 'we check periodically' into a number that can be verified.

33.3 Techniques worth borrowing even without an auditor

- **Plausibility checks between independent sources:** two sensors, or a sensor and a model, that must agree within a tolerance. This catches failures no single-channel check can.
- **Power-on self-test:** a RAM pattern test, a flash CRC, a check of each output driver. Costs milliseconds at boot and detects real hardware degradation.

- **Redundant storage of critical variables** - store a value and its inverse, and verify on read. Cheap protection against corruption.
- **Control-flow monitoring**: a checksum of the sequence of stages executed in a cycle, verified before acting on the result.
- **Memory partitioning with the MPU** so a defect in a non-critical module cannot corrupt the critical one.
- **A defined safe state and a measured time to reach it** - useful in every product, mandatory in a certified one.

THE COST OF CERTIFICATION IS PROCESS, NOT CODE

Teams routinely underestimate certification by assuming it is about writing safer C. In reality most of the effort is evidence: requirements traceability, review records, tool qualification, coverage analysis, and the argument tying it all together. Retrofitting that evidence to an existing codebase typically costs more than writing the code did. **If certification is possible in your product's future, adopt requirement IDs, traceability and a coding standard from the start** - they are cheap at the beginning and brutally expensive later.

33.4 Self-tests and diagnostic coverage in practice

Standards ask not only 'do you have a self-test' but 'what fraction of dangerous failures does it detect' - the **diagnostic coverage**. That reframes the work usefully: each test is claimed against a specific failure mode.

Test	Failure detected	When to run
RAM pattern test (march test)	Stuck-at and coupling faults in SRAM	At power-on; a partial, incremental version periodically
Flash CRC over the image	Corruption or degradation of stored code	At boot, and periodically in slices for long-running devices
CPU register and ALU test	Stuck-at faults in the core	At power-on; required at higher integrity levels
Clock plausibility (cross-check two sources)	Oscillator drift or failure	Periodically - a wrong clock makes every timing wrong at once
Watchdog with a window	Runaway or stalled execution	Continuous (Chapter 28)
Program-flow monitoring	Skipped or repeated stages	Every control cycle
Output read-back / loopback	A driver stage that fails open or short	Every cycle where the output matters
Sensor plausibility and rate-of-change limits	A stuck or drifting sensor	Every sample
Stack and MPU integrity	Corruption from a defect elsewhere	Continuous, by construction (Chapter 24)

TURNING THE FAULT TOLERANT TIME INTERVAL INTO SETTINGS

Suppose the hazard analysis says a stuck output must not persist for more than 500 ms. The detection chain is: the output read-back runs every control cycle (10 ms), the fault must be confirmed over three consecutive cycles to reject noise (30 ms), the safe-state transition takes 20 ms, and the watchdog - the backstop if the software itself has stopped - must therefore fire well inside the remainder: a 200 ms timeout leaves 250 ms of margin. **Every one of those numbers is now traceable to the hazard**, which is exactly what an auditor asks for and, independently, exactly how you would want it designed.

Exercises

- 1 Write a one-page hazard analysis for your product: what can fail, what harm follows, and what detects it.
- 2 Define the safe state and measure how long your firmware takes to reach it from a detected fault.
- 3 Add a power-on RAM test and flash CRC and measure the boot-time cost.
- 4 Run a MISRA or CERT-C checker over your codebase and triage the top twenty findings - not to comply, but to see what they are.

- 5 Implement a plausibility check between two independent measurements and test it by disconnecting one sensor.

34. When a Microcontroller Is Not Enough: Embedded Linux

At some point a product needs a filesystem, a network stack with TLS, a graphical display, a camera pipeline, or simply more software than one team can write. That is the moment to consider an application processor running Linux - and to understand what you gain and what you give up.

34.1 The trade, stated plainly

	MCU (bare metal / RTOS)	MPU (Linux)
Boot time	Milliseconds	1-30 seconds (optimisable to ~1 s)
Timing determinism	Microseconds, provable	Tens to hundreds of microseconds with PREEMPT_RT; not provable in the same sense
Memory	KB of RAM	Hundreds of MB, with an MMU
Power	uA sleep	mW to W; deep sleep means a full reboot
Software available	You write most of it	Enormous ecosystem: TLS, databases, Python, GUI toolkits
Update	Your bootloader	A/B rootfs with RAUC/SWUpdate/Mender
Cost and complexity	Low; one engineer can hold it all	Higher BOM, DDR layout, a build system, and a kernel to maintain

THE COMMON ANSWER IS BOTH

Heterogeneous designs - a Cortex-A running Linux beside a Cortex-M running the real-time control - are now standard, on a single SoC or on two chips. Linux does connectivity, UI and storage; the M core does the deterministic work; they communicate over shared memory with a mailbox (OpenAMP / rpmsg). This gets you both properties, at the price of two build systems and an interface to design carefully (Chapter 18's multi-core rules apply exactly).

34.2 The boot chain

```

Boot ROM (in silicon)
  | loads from eMMC/SD/SPI according to boot pins
  v
SPL / TF-A           initialises DDR, minimal drivers
  v
U-Boot              environment, device tree, boot script,
  |                  and where A/B rootfs selection happens
  v
Linux kernel + DTB   probes drivers described by the device tree
  v
init (systemd/BusyBox) mounts the rootfs, starts services
  v
your application

```

Each stage can be signed and verified by the previous one, giving the same chain of trust as Chapter 32 - here it is called verified boot, and dm-verity extends it to the whole root filesystem.

34.3 Device tree, in one page

```
/* The hardware description the kernel reads at boot - not code. */
&spi1 {
    status = "okay";
    pinctrl-0 = <&spi1_pins>;
    accel@0 {
        compatible = "st,lis3dh"; /* selects the driver */
        reg = <0>; /* chip select 0 */
        spi-max-frequency = <5000000>;
        interrupt-parent = <&gpio2>;
        interrupts = <5 IRQ_TYPE_EDGE_RISING>;
    };
};
```

The **compatible** string binds this node to a driver. Adding a sensor to a Linux board is therefore usually a device-tree edit rather than new code - and when a peripheral 'does not appear', the first check is **dmesg** plus the device tree, not the driver source.

Task	Interface
GPIO from user space	libgpiod (/dev/gpiochipN) - the sysfs interface is deprecated
Sensors, ADC	The IIO subsystem: /sys/bus/iio/..., with buffered and triggered modes
SPI / I2C from user space	spidev , /dev/i2c-N - fine for prototyping, a kernel driver for production
Serial	/dev/ttySx with termios - remember to disable the console on a port you want to use
Real-time threads	SCHED_FIFO with a priority, mlockall() to avoid page faults, and CPU isolation
Writing a kernel driver	Platform driver + device tree binding; worth it when timing, power or an interrupt path demands it

34.4 Building the system

	Buildroot	Yocto / OpenEmbedded
Model	A single kernel config plus a package list; builds a whole image	Layered recipes, package feeds, SDK generation
Learning curve	Days	Weeks
Best for	Small, fixed, single-product images	Product families, long maintenance, licence compliance, teams
Output	rootfs image	Images, packages, an SDK, an SBOM and licence manifests

THREE HABITS THAT MAKE AN EMBEDDED LINUX PRODUCT MAINTAINABLE

A read-only root filesystem with a small writable overlay: it removes an entire class of corruption and makes A/B updates simple. **Everything in version control, including the kernel configuration, the device tree and the build recipes** - a product you cannot rebuild identically in three years is a liability. **Boot-time budgeting from day one** (**systemd-analyze**, **bootgraph**): boot time is easy to keep at two seconds and very hard to reduce from thirty.

34.5 User space or a kernel driver?

Do it in user space when	Write a kernel driver when
The interface already exists (spidev , i2c-dev , gpiod , IIO)	The device needs an interrupt handled with low latency
Latency of tens to hundreds of microseconds is acceptable	It must present a standard interface (IIO, input, netdev) so other software just works

Do it in user space when	Write a kernel driver when
You want to debug with gdb and iterate quickly	Power management, DMA or clock control must be coordinated by the kernel
The code is product-specific and will never be upstreamed	The same hardware is used across several products and deserves one maintained driver

A common and effective middle path is a small kernel driver that does only the timing-critical part - the interrupt, the DMA, a FIFO - and exposes a character device or an IIO buffer, with all the policy in user space where it is easy to update and test.

```
# The user-space toolkit worth knowing before writing kernel code
dmesg -w                # driver probe messages, live
ls /sys/bus/iio/devices/ # sensors the kernel already exposes
gpioinfo / gpioset / gpioget # libgpiod, replaces sysfs GPIO
cat /proc/interrupts      # is your interrupt firing at all?
cat /proc/device-tree/... # what the kernel actually parsed
cyclicttest -p 80 -t1 -n  # scheduling latency under load
ftrace / trace-cmd / perf # where the time and the wake-ups go
```

34.6 Real-time on Linux, honestly

Configuration	Typical worst-case latency	Notes
Stock kernel, no tuning	Milliseconds, occasionally tens	Fine for UI and networking, not for control
PREEMPT plus priorities and mlockall	Hundreds of microseconds	Cheap improvement; no kernel patch
PREEMPT_RT	Tens of microseconds, bounded in practice	Now largely mainline; costs some throughput
RT plus CPU isolation (isolcpus, IRQ affinity)	Tens of microseconds, far fewer outliers	Dedicate a core to the real-time work
A companion MCU core	Microseconds, provable	The only option when the deadline is genuinely hard (Chapter 20)

THREE THINGS THAT RUIN REAL-TIME ON LINUX

Page faults - call `mlockall(MCL_CURRENT | MCL_FUTURE)` and pre-fault the stack, or the first touch of a page costs milliseconds. **Dynamic allocation and logging in the hot path** - the same rules as firmware apply. **Power management and frequency scaling** - a CPU that drops to a low idle state adds wake-up latency; pin the governor for the isolated core.

Exercises

- 1 Boot a board, read `dmesg`, and identify which device-tree node created each of three devices.
- 2 Add a sensor to the device tree and read it through IIO without writing any code.
- 3 Measure scheduling latency with `cyclicttest` on a stock kernel and with `PREEMPT_RT`, under load.
- 4 Build the same image with Buildroot and with Yocto and compare the effort and the result.
- 5 Implement an A/B rootfs update and power-cycle during it twenty times.

35. Production: Manufacturing, Provisioning and the Field

Firmware's responsibilities do not end at the release build. Somebody must program tens of thousands of boards, prove each one works, give each a unique identity and its calibration, lock it, and later diagnose the ones that come back. Designing that early is much cheaper than improvising it during a production ramp.

35.1 Getting firmware onto the board

Method	Speed	Notes
Gang programmer via SWD/JTAG	Seconds per board, many at once	The standard; needs test points and a fixture
Pre-programmed parts from the distributor	Zero line time	Great for the bootloader; inflexible for the application
Bootloader over USB/UART on the line	Seconds	Also exercises the connector - a genuine test benefit
In-circuit test (ICT) or flying probe	Part of the test step	Combines programming with electrical test
Boundary scan (JTAG)	Moderate	Tests solder joints as well - valuable on dense boards

A common and effective split: program a small, permanent bootloader at the earliest step (or buy parts pre-programmed with it), then have the factory test firmware and the application loaded through that bootloader. The line then never needs a debug probe again, and the same path is used for field updates - so it is tested thousands of times before a customer ever uses it.

35.2 Factory test firmware

What the production test must prove

- ☐ Every rail is within tolerance and quiescent current is in range.
- ☐ The oscillator starts and is within its frequency tolerance.
- ☐ Every external device answers on its bus (ID register read).
- ☐ Every connector and switch is exercised, with an operator prompt where needed.
- ☐ Every output can drive and every input can sense - loopbacks where possible.
- ☐ The radio transmits at the expected power and receives a known signal.
- ☐ Calibration is performed and stored, with the reference instrument's reading recorded.
- ☐ A unique identity and its credentials are provisioned.
- ☐ The result, with all measured values - not just pass/fail - is uploaded to a database keyed by serial number.
- ☐ The final step programs the application image, locks the debug port, and verifies the lock.

TEST TIME IS A COST, SO BUDGET IT

A line making 2,000 units a day over two shifts has roughly 57,600 seconds of line time, so a single test station allows about **28 seconds per unit** including handling. If the firmware's self-test takes 40 seconds because a sensor needs to stabilise, the choices are: parallel test fixtures (more capital), a shorter test (less coverage), or overlapping test with handling. **Deciding this after the line is running costs far more than designing the test to a budget**, so ask for the target unit rate before writing the test firmware.

35.3 Identity, keys and calibration

- **Serial number:** unique, printed on the label and readable over the interface, tied to the manufacturing record.

- **Keys and certificates:** generated in the device where the part supports it (so the private key never exists elsewhere), or injected from an HSM in a controlled environment. Log which key went to which unit, and never in plain text on a factory PC.
- **Calibration:** stored in a dedicated, versioned, CRC-protected region (Chapter 16), with the date, the operator, the instrument, and the raw measurements - not only the fitted coefficients. When a cohort drifts in the field, the raw data is what lets you correct it.
- **Locking:** enable readout protection and disable debug at the last step, then verify that it took effect. Getting this wrong in either direction - locking too early or never locking - is a common and expensive production defect.

35.4 The field, and what comes back

Activity	What firmware provides
Diagnosing a returned unit	Crash records, reset causes, error counters, uptime, firmware version, and the manufacturing record keyed by serial number (Chapter 28)
Fleet health	Version distribution, check-in rate, crash rate per version, battery statistics
Field service	A documented recovery mode, a way to reflash locally, and a technician interface that does not require a developer
Root cause across units	Correlating field failures with manufacturing test values often finds a marginal component before it becomes a recall
End of life	A documented support window, final firmware, and a plan for the cloud services the device depends on

THE TWO PRODUCTION SURPRISES

Yield is a firmware problem too. If 3% of units fail a test because a timeout is marginal at the edge of the component tolerance, that is 60 units a day scrapped or reworked - and the fix is usually one constant. Get the test's measured distributions, not just its pass rate. **Firmware version skew.** Units built in March have a different image from those built in June, and both are in the field. Every diagnostic must start by reporting its own version, and every fix must be evaluated against every version still in circulation.

Exercises

- 1 Write the production test specification for your product, with a time budget per step.
- 2 Make your factory test emit measured values rather than pass/fail and plot the distribution over fifty boards.
- 3 Implement provisioning of a unique identity and confirm two units never receive the same one.
- 4 Lock a development board's debug port and then follow your own documented recovery procedure.
- 5 Take a returned or failed unit and diagnose it using only what the firmware records - then add whatever was missing.

PART VI

Practice

Connected products and machine learning at the edge, the professional habits that keep a firmware codebase alive for a decade, and one complete project taken from an empty repository to a maintained product in the field.

36. Connected Devices and Edge Intelligence

A connected product is not a device with a radio bolted on; it is a device whose firmware must manage identity, connectivity, telemetry, commands, updates and time, all while the link comes and goes. Increasingly it also runs inference locally. This chapter is the architecture for both.

36.1 The anatomy of connected-device firmware

```
+-----+
| application: sensing, control, user experience |
+-----+
| device model:  telemetry | commands | desired vs reported |
|                  |           | state (the 'shadow')         |
+-----+
| services: identity/provisioning | OTA (Ch 31) | time sync |
|          store-and-forward buffer | diagnostics upload |
+-----+
| connectivity manager: a state machine with backoff |
|   DOWN -> LINK_UP -> AUTHENTICATED -> READY -> (error) -> DOWN |
+-----+
| radio stack and driver (Ch 14) |
+-----+
```

Every box is testable on its own, and the application never talks to the radio directly - which is what allows the same product to ship with BLE on one variant and cellular on another.

Connectivity rules that survive contact with reality

- ☐ The application enqueues data; it never waits for a link.
- ☐ Every outbound item is timestamped at capture, not at transmission.
- ☐ The buffer is bounded and has a documented drop policy (oldest, newest, or by priority).
- ☐ Reconnection uses exponential backoff with jitter and a maximum interval.
- ☐ Commands are idempotent and carry an identifier, so a duplicate is harmless.
- ☐ The device authenticates the server and the server authenticates the device (Chapter 32).
- ☐ Clock skew is handled: the device knows whether its time is synchronised, and says so in the data.
- ☐ There is a supported path to reconfigure connectivity locally, for when the network settings are wrong.

36.2 Machine learning on a microcontroller

Running a model on the device rather than in the cloud buys four things: latency measured in milliseconds, privacy (raw audio or video never leaves the device), operation without connectivity, and a dramatic reduction in radio energy - which, as Chapter 29 showed, is usually the whole power budget. What it costs is memory and cycles, both of which must be budgeted before the model is chosen.

Stage	Where it happens	Firmware's job
Data collection	On the device, in the field	Reliable, timestamped, labelled capture - usually the hardest and most valuable part
Training	Off the device, on a workstation	Nothing - but provide honest, representative data
Compression	Off the device	Quantise to INT8 (or lower), prune, and check the accuracy cost
Deployment	On the device	Fit the weights in flash and the activations in RAM; meet the latency budget
Verification	Both	Bit-exact test vectors: the same input must give the same output on host and target

Stage	Where it happens	Firmware's job
Update and monitoring	Both	Ship models as signed assets (Chapter 31); monitor input drift and confidence in the field

BUDGETING AN INFERENCE

A keyword-spotting model with **2.5 million multiply-accumulates** per inference, quantised to INT8. With CMSIS-NN on a Cortex-M4F, int8 kernels achieve roughly 1.5 MAC per cycle using the SIMD instructions, so one inference costs about 1.7 million cycles = **21 ms at 80 MHz**. At 10 inferences per second that is **21% of the CPU** - feasible, but it must be accounted for in the response-time analysis of Chapter 20, and it will dominate the energy budget unless the device sleeps between inferences. Memory: 20,000 INT8 weights need 20 KB of flash, and the activation arena - the peak of concurrent intermediate tensors, not their sum - might be **16-32 KB of RAM**, which on a 64 KB part is the binding constraint. **Compute all three numbers (cycles, flash, arena) before committing to a model architecture**, because a model that does not fit is discovered late and expensively.

Technique	Effect	Reference
INT8 post-training quantisation	4x smaller, 2-4x faster, typically under 1% accuracy loss	Standard first step
Quantisation-aware training	Recovers accuracy at very low bit widths	When PTQ loses too much
Structured pruning	Removes whole channels, so the speed-up is real on an MCU	Unstructured sparsity rarely helps without hardware support
Knowledge distillation	A small model trained to mimic a large one	Often the best accuracy per byte
Operator fusion and CMSIS-NN kernels	2-5x over naive C	Always use the vendor/CMSIS kernels
Cascaded models	A tiny always-on model gates a larger one	The standard low-power pattern for wake words and event detection

VERIFY THE DEPLOYED MODEL, NOT THE TRAINED ONE

The model that runs on the device has been converted, quantised and compiled by a different toolchain from the one that trained it. Keep a set of **golden test vectors** - inputs with their expected outputs - and run them on the target as part of the power-on self-test and in CI. Silent numerical differences between host and target are common, and without vectors they surface as 'the product got worse in the field' months later.

36.3 On-device adaptation, briefly

Training on the device itself - personalising a gesture model to one user, adapting a threshold to a specific installation - is now feasible on larger microcontrollers using the same efficiency techniques: quantised training, updating only the last layers, and sparse updates. The firmware considerations are the ones this book has already established: a bounded memory arena, a bounded time slice so the control loop still meets its deadlines, power-fail-safe storage of the updated weights (Chapter 16), a way to reset to the factory model, and signed distribution of the base model it starts from.

Exercises

- 1 Draw your product's connectivity state machine, including every error transition, and implement the backoff.
- 2 Measure the energy cost of one telemetry message and compare it with one hour of sleep.
- 3 Take any pretrained small model, quantise it to INT8, and compute its flash, arena and cycle budget for your target before deploying it.
- 4 Build golden test vectors for a model and run them on the target; compare bit-for-bit with the host.
- 5 Implement a cascaded detector: a cheap always-on stage gating an expensive one, and measure the resulting average current.

37. Professional Practice

Firmware outlives the people who write it. The habits in this chapter are what let a codebase be picked up in five years, on hardware that has revised three times, by an engineer who was not there - and what make the difference between a team that ships predictably and one that is permanently debugging.

37.1 Version control and the things that are not source

Artefact	Practice
Source	One repository per product where possible; short-lived branches; every commit builds
Vendor HAL and SDKs	Vendored into the repository at a pinned version - not 'whatever the IDE downloaded'
Toolchain	Pinned by version and, ideally, containerised, so a build in three years is byte-identical
Binaries per release	ELF, map, bin/hex, and the build configuration, archived permanently - they are how you decode a field crash
Hardware documents	Schematic revision, BOM, errata and datasheet revisions referenced from the firmware repository
Configuration and calibration formats	Versioned schemas with migration code (Chapter 16)

VERSION NUMBERS THAT ANSWER QUESTIONS

Ship **MAJOR.MINOR.PATCH+<git-hash>** with the build date and hardware compatibility, printed at boot, reported in telemetry, and stored in the crash record. The hash is what matters: it maps a unit in the field to an exact tree, an exact map file and an exact toolchain, and without it every field investigation begins with a guess.

37.2 Reviewing firmware

What to look for in a firmware code review

- ☐ Every register write: is the clock enabled, are reserved bits preserved, is it write-1-to-clear?
- ☐ Every shared variable: who else touches it, and under what protection?
- ☐ Every loop that waits: does it have a timeout?
- ☐ Every buffer: who owns it, how long does it live, can it be written past its end?
- ☐ Every ISR: how long is it, does it block, does it call anything unsafe?
- ☐ Every error return: is it checked, and does the handling do something real?
- ☐ Every new global or static: is it necessary, and is it initialised?
- ☐ Stack impact: recursion, large locals, deep call chains.
- ☐ Units and types: is that milliseconds or ticks, signed or unsigned, and can it overflow?
- ☐ Does it still build and pass tests at the release optimisation level?

37.3 Documentation that is worth writing

- **A pin map**, generated or maintained as one table, shared with the hardware team. Pin conflicts discovered at bring-up are pure waste.
- **Interface documents** for every protocol the device speaks, versioned alongside the code that implements them.
- **Architecture decision records**: one page each - the decision, the alternatives, the reason, the date. These are what a future engineer needs and what nobody writes.
- **A memory and timing budget** (Chapters 20 and 24), kept current. It converts vague concern into a number.

- **A bring-up log and an errata list** for the hardware, including your workarounds and why they exist. Undocumented workarounds get removed by someone who thinks they are dead code.

37.4 Estimation, honestly

Phase	Where the time really goes
Bring-up	Doubles when the board is new. Budget for at least one hardware bug you must work around
Driver work	The datasheet is optimistic; errata and undocumented sequencing are normal
Integration	Where the timing interactions appear - the hardest phase to estimate and the most often underestimated
Low power	Always longer than planned; measurement equipment and many small discoveries
Certification and production	Calendar time you do not control: test houses, audits, line availability
The last 10%	Field behaviour, update mechanisms, recovery paths, documentation - routinely 30% of the effort

THE PROFESSIONAL OBLIGATION

Firmware controls things that move, heat, charge and connect. An engineer who knows a safety mechanism is untested, a key is shared across a fleet, or a watchdog is disabled has an obligation to say so clearly, in writing, to someone who can act - and to keep saying it. Most firmware disasters were known to somebody first.

37.5 Working with the hardware team

Firmware and hardware fail together and succeed together, and the interface between the two disciplines is where schedule is most often lost. A few habits remove most of that friction.

- **Own the pin map jointly, in one file.** Every pin, its function, its alternate-function number, its default state, and who requested it. Review it before layout, when a change is free.
- **Ask for the things that cost nothing at design time:** test points on every bus and rail, a spare GPIO or two for instrumentation, a current-measurement jumper, a debug connector, and pull-ups where firmware cannot substitute for them.
- **Report measurements, not impressions.** 'SCL rise time is 900 ns against a 300 ns limit' is a hardware finding; 'I2C is flaky' is a conversation.
- **Track errata and workarounds in the firmware repository,** each with a link to the erratum and the conditions under which it can be removed. Undocumented workarounds are deleted by well-meaning engineers.
- **Expect the first boards to be wrong** and plan for it: two or three revisions is normal, so keep a per-board rework log (Chapter 27) and make firmware detect the board revision where possible - a resistor strap read at boot costs nothing and prevents shipping the wrong image.

BOARD REVISION DETECTION IS WORTH ONE GPIO

Two pins strapped to ground or supply give four board revisions, read once at boot and reported in telemetry and in the crash record. When half the fleet behaves differently, the first question is always which hardware they are, and this answers it without opening a single enclosure.

Exercises

- 1 Reproduce a build from a release six months old. If you cannot, fix that before anything else.
- 2 Write three architecture decision records for decisions already made in your project.
- 3 Run your review checklist over a recent merged pull request and count what it finds.
- 4 Produce the pin map for your board and check it against the schematic, pin by pin.
- 5 Estimate a task you have already completed, then compare with the actual. Repeat monthly - calibration is a learnable skill.

38. A Complete Product, Stage by Stage

This chapter runs one project end to end: a **battery-powered wireless sensor node** that measures temperature and vibration, reports over Bluetooth Low Energy, runs for years on a coin cell, and can be updated in the field. Every stage references the chapter that covers it in depth. The point is the order, the decisions, and the numbers - the same skeleton fits a motor controller or a medical device.

38.1 Stage 0 - Requirements, before anything else

Requirement	Value	Consequence
Measurement	Temperature +/-0.5 degC, vibration RMS to 1 kHz	Sets the ADC rate (≥ 2.5 kHz) and the sensor choice
Reporting	Every 10 minutes, or immediately on an alarm	Sets the radio duty cycle and the power budget
Battery life	≥ 2 years on a 220 mAh coin cell	Gives an average current budget of 12.6 μ A
Latency	Alarm reported within 3 s	Constrains sleep depth and connection interval
Update	Field updatable over BLE, must never brick	A/B slots plus a bootloader (Chapter 31)
Security	Only signed firmware runs; data authenticated	Secure boot, per-device keys (Chapter 32)
Environment	-20 to +70 degC, IP54	Crystal tolerance, calibration over temperature
Volume and cost	10,000 units, BOM under a limit	Part choice, test time per unit (Chapter 35)

TWO NUMBERS DECIDE THE ARCHITECTURE

12.6 μ A average and **3 s alarm latency**. Together they rule out staying connected continuously, rule out a 1 ms RTOS tick that never sleeps, and require a wake-on-interrupt sensor rather than polling. Almost every later decision follows from these two lines - which is why they are written before any part is chosen.

38.2 Stage 1 - Choose the parts and the software architecture

- **MCU**: a BLE SoC with 512 KB flash, 64 KB RAM, a low-power comparator, DMA, and deep-sleep current under 2 μ A with RAM retention. Radio-on-chip beats a module here on power and cost at this volume (Chapter 14).
- **Sensors**: a digital temperature sensor with a threshold interrupt (so no polling), and an accelerometer with an internal FIFO and a wake-on-motion interrupt (so the CPU sleeps while it fills).
- **Software**: an RTOS, because the vendor BLE stack assumes one - but with each task written as a run-to-completion state machine (Chapter 17), and tickless idle enabled from the start (Chapter 21).
- **Storage**: internal flash for configuration and calibration, A/B records with CRC (Chapter 16). No filesystem: there is nothing to put in one.

38.3 Stage 2 - Review the schematic while it can still change

Apply the checklist from Chapter 27 before the board is laid out. On this design it produced four changes: SWD brought to a test connector; a series resistor and a test point on the sensor interrupt line; the I2C pull-ups changed from 10 k Ω to 2.2 k Ω because the bus runs at 400 kHz; and a 100 μ F bulk capacitor next to the radio, because a 220 mAh coin cell cannot supply a 15 mA transmit pulse without its voltage dipping into the brown-out threshold (Chapter 29).

38.4 Stage 3 - Repository, toolchain and CI on day one

Before the first driver is written

- ☐ Repository with the pinned toolchain, vendor SDK vendored at a fixed version, and a one-command build.
- ☐ CI that builds, runs host unit tests, and records flash and RAM usage (Chapter 26).
- ☐ The version string with the git hash compiled in and printed at boot (Chapter 5).
- ☐ A `port/` directory so drivers can be compiled for the host with fakes (Chapter 23).
- ☐ A fault handler capturing PC, LR, CFSR into `.noinit` before anything else exists to crash (Chapter 25).

38.5 Stage 4 - Bring up the board

Follow Chapter 27's sequence: power, probe, blink, clocks, console, one peripheral at a time. On this project it took a day, and the log recorded one real finding: the accelerometer did not answer on I2C until the enable line was driven, which the schematic showed but the firmware had assumed was pulled up.

38.6 Stage 5 - Drivers, one at a time, each with tests

Each driver follows Chapter 23's shape: a handle, an injected bus, explicit errors, no globals, a blocking API and an interrupt-driven one. Each gets host tests with a fake bus and a golden byte sequence before it is integrated. The temperature driver's conversion arithmetic is a pure function with ten unit tests, including both extremes and the sign boundary (Chapter 26).

38.7 Stage 6 - The timing budget, written down and checked

Activity	C	T	D	Priority
Accelerometer FIFO ISR	40 us	80 ms	80 ms	Interrupt
BLE stack (vendor, radio timing)	reserved 20%	-	-	Highest - never delayed
Sampling task (FFT of 256 samples)	3.2 ms	80 ms	40 ms	1
Alarm evaluation	0.2 ms	80 ms	80 ms	2
Telemetry / connection task	5 ms	600 s	3 s	3
Housekeeping (watchdog, stats)	0.3 ms	1 s	1 s	4

Utilisation excluding the BLE reservation is $0.04/80 + 3.2/80 + 0.2/80 + 0.3/1000 =$ about **4.3%**, so the deadlines are met with enormous margin - and that margin is the point: it is the budget the radio stack, the future features and the worst-case paths will consume (Chapter 20).

38.8 Stage 7 - Power engineering, measured not assumed

With the design of Chapter 29 - sleep at 3 uA, a 20 ms sampling wake-up each minute, a 100 ms transmission every ten minutes - the predicted average is **8.8 uA** against the 12.6 uA budget. The first measurement said **41 uA**. The current trace found three causes, all typical: a GPIO left floating on an unused sensor pin (12 uA), the debug interface left enabled in the build (15 uA), and the RTOS tick still running at 1 kHz because tickless idle needed a low-power timer that had not been configured (5 uA). Fixing all three brought it to **9.4 uA**, within budget. **Every one of those would have been invisible without a current waveform.**

38.9 Stage 8 - Storage, configuration and calibration

Two A/B record pairs in separate sectors (Chapter 16): factory calibration, written once in production and then write-protected, and user configuration, written whenever settings change. Each record carries a magic number, a schema version, a sequence number and a CRC-32, with the CRC written last. Migration code exists for version 1 to 2 before version 2 ever ships.

38.10 Stage 9 - Bootloader and OTA, built before they are needed

A 32 KB bootloader, two 224 KB application slots, an A/B metadata sector (Chapter 31). The application downloads into the inactive slot over BLE at about 40 kB/s - a 180 KB image takes 4.5 s of connected time - verifies an ECDSA P-256 signature, marks the image pending, and resets. The new image must run for 60 seconds and complete one successful server exchange before it is confirmed; otherwise the next boot reverts. This was tested by power-cycling a unit randomly through 200 updates.

38.11 Stage 10 - Security and provisioning

Secure boot with the public key in write-protected flash; a per-device key pair generated on-device during production so the private key never leaves it; anti-rollback via a monotonic counter; the debug port locked as the final production step (Chapters 32 and 35). The threat model fits on one page and explicitly excludes laboratory attacks on a physically-held device, which is a decision made and recorded rather than a gap.

38.12 Stage 11 - Testing, on the host and in the loop

140 host unit tests covering conversions, the parser, the ring buffer, the state machines and the configuration migration; a fuzzer on the BLE command parser that found two out-of-bounds reads in the first minute; a HIL fixture with a programmable power supply, a temperature chamber and a relay that can cut power at any moment during an update; and a 72-hour soak with telemetry, whose first run exposed a counter wrap after 49.7 days that only the arithmetic of Chapter 10 predicted.

38.13 Stage 12 - Production test and calibration

A factory test build that measures every rail, reads every device ID, checks the crystal against a reference, exercises the radio at a known power, performs a two-point temperature calibration against a reference instrument, stores the raw readings as well as the coefficients, provisions the identity, then flashes the application, locks the part and verifies the lock - within the **28-second** per-unit budget of Chapter 35. Every measured value goes to a database keyed by serial number.

38.14 Stage 13 - Release, roll out, and watch

Staged rollout: 1%, then 10%, then all, halted automatically if the crash rate or the check-in rate moves. Telemetry reports firmware version, reset cause, boot count, stack high-water marks, error counters and battery voltage. The first field release produced exactly one surprise - devices in one customer's freezer showed a slow rise in I2C errors, traced to the pull-up value at -25 degC - which was found because the error counters existed (Chapter 28).

38.15 Stage 14 - Maintain

For as long as the product is supported: security updates when a component in the SBOM gets a CVE, a documented support window, and every ELF and map file kept forever so that a crash report from a three-year-old unit can still be decoded. The last engineering task of a product is not shipping it.

38.16 The timeline, honestly

Week	Activity	What usually goes wrong
0-1	Requirements, part selection, schematic review	Skipped, and paid for in week 12
2	Repo, toolchain, CI, fault handler	Deferred as 'overhead'
3-4	Bring-up and clocks	One hardware bug, one datasheet erratum
5-8	Drivers and their tests	The sensor's documented initialisation sequence is incomplete

Week	Activity	What usually goes wrong
9-11	Architecture, tasks, protocol, storage	Timing interactions appear here, not earlier
12-13	Power optimisation	The first measurement is always 3-5x the prediction
14-15	Bootloader, OTA, security	Underestimated by half, every time
16-18	Test, HIL, soak, production test	Soak finds what nothing else does
19+	Pilot build, staged release, field learning	The real requirements finally reveal themselves

38.17 The one-page master checklist

Foundations

- ☐ Requirements, budgets (power, memory, timing) and the safe state are written down.
- ☐ The startup path, linker script and memory map are understood, not inherited.
- ☐ Reset cause, version and a crash record are captured from the first build.
- ☐ Every wait has a timeout; every buffer has an owner; every queue is bounded.

Hardware interface

- ☐ Every peripheral: clock enabled, pins configured, errata read.
- ☐ ISRs are short, non-blocking, and clear their flags with a barrier.
- ☐ Shared data has a documented mechanism; the longest critical section is measured.
- ☐ DMA buffers are static, aligned, and cache-maintained where required.

Architecture and timing

- ☐ No blocking delays in periodic paths.
- ☐ Task priorities are assigned by deadline, and the response-time analysis has been done.
- ☐ Stack sizes are measured with high-water marks and guarded.
- ☐ Worst case, not average, is what was measured.

Shipping

- ☐ Bootloader with A/B slots, signature verification, and confirm-or-revert.
- ☐ Debug locked, secrets provisioned per device, SBOM produced.
- ☐ Production test measures values, not just pass/fail, and fits the line's time budget.
- ☐ Telemetry reports version, resets, errors and headroom; rollout is staged.
- ☐ Every release artefact is archived and reproducible.

IF YOU REMEMBER ONE THING FROM THIS BOOK

Measure. The current trace, the scope on a GPIO, the cycle counter, the stack high-water mark, the map file, the response-time calculation. Firmware rewards the engineer who converts opinions into numbers faster than anything else in software - because the hardware is always telling the truth, and it is usually willing to tell you in about ten minutes.

Exercises

- 1 Write stage 0 for a product you want to build. If you cannot fill in the power and timing budgets, that is the first task, not a formality.
- 2 Take an existing project of yours and run stages 6, 7 and 11 on it - timing budget, power measurement, host tests. Report what you find.

- 3 Implement the update path (stage 9) on a spare board and try to brick it deliberately. Keep trying until you cannot.
- 4 Run the master checklist against your current firmware and count the unticked boxes. That count is your backlog, in priority order.

APPENDICES

Appendices

A formula and register reference you can work from, a glossary of every term used in this book, and a study roadmap with projects and tools.

Appendix A. Formula and Reference Sheet

Clocks, timers and serial

$$\begin{aligned}
 \text{Timer period} &= (\text{PSC} + 1) * (\text{ARR} + 1) / f_{\text{timer}} \\
 \text{PWM frequency} &= f_{\text{timer}} / ((\text{PSC} + 1) * (\text{ARR} + 1)) \\
 \text{PWM duty} &= (\text{CCR}) / (\text{ARR} + 1) \\
 \text{PWM resolution} &= \log_2(\text{ARR} + 1) \text{ bits} \\
 \text{APB timer clock} &= 2 \times \text{PCLK} \text{ when the APB prescaler is not 1} \\
 \text{UART divider} &= f_{\text{clk}} / (\text{oversample} * \text{baud}) \\
 \text{Baud error} &= (\text{actual} - \text{target}) / \text{target}, \text{ keep } |\text{total}| < 2\% \\
 \text{Byte time (8N1)} &= 10 / \text{baud} \\
 \text{PLL} &f_{\text{out}} = f_{\text{in}} / M * N / P \\
 \text{Crystal drift} &\text{ppm} \times 86400 / 1e6 = \text{seconds of error per day} \\
 &(20 \text{ ppm} \rightarrow 1.73 \text{ s/day}, 100 \text{ ppm} \rightarrow 8.64 \text{ s/day})
 \end{aligned}$$

Analogue

$$\begin{aligned}
 \text{LSB} &= V_{\text{ref}} / 2^N \quad (12\text{-bit}, 3.3 \text{ V} \rightarrow 0.806 \text{ mV}) \\
 \text{Ideal SNR} &= 6.02 N + 1.76 \text{ dB} \\
 \text{ENOB} &= (\text{SNR}_{\text{measured}} - 1.76) / 6.02 \\
 \text{Oversampling} &4^k \text{ samples averaged} \rightarrow +k \text{ bits (needs dither)} \\
 \text{Acquisition time} &t = R_{\text{source}} \times C_{\text{hold}} \times \ln(2^{(N+1)}) \\
 \text{RC filter} &f_c = 1 / (2 \pi R C), \quad \tau = R C \\
 \text{Settling to 0.1\%} &\sim 7 \tau \quad (3 \tau = 95\%, 5 \tau = 99.3\%) \\
 \text{I2C pull-up} &t_{\text{rise}} = 0.8473 \times R \times C_{\text{bus}} \\
 &R_{\text{max}} = t_{\text{rise_spec}} / (0.8473 \times C_{\text{bus}}) \\
 &R_{\text{min}} = V_{\text{dd}} / I_{\text{sink_max}} \quad (3 \text{ mA})
 \end{aligned}$$

Power and energy

$$\begin{aligned}
 I_{\text{avg}} &= \text{SUM}(I_{\text{state}} \times t_{\text{state}}) / T_{\text{cycle}} \\
 \text{Battery life} &= \text{capacity_mAh} / I_{\text{avg_mA}} \quad (\text{then derate}) \\
 \text{Charge} &1 \text{ mA for 1 hour} = 1 \text{ mAh} = 3.6 \text{ coulombs} \\
 \text{Energy} &E = V \times I \times t; \quad \text{dynamic CPU power} \sim C V^2 f \\
 \text{Coin cell} &220 \text{ mAh, high internal resistance: buffer TX pulses} \\
 \text{Rule} &\text{one radio transmission can cost more than an hour asleep}
 \end{aligned}$$

Real-time

```

Utilisation      U = SUM C_i / T_i

RM bound        U <= n (2^(1/n) - 1)    n=3: 0.780, inf: 0.693

Response time    R = C + B + SUM_j CEIL(R / T_j) * C_j    (iterate)

Interrupt latency entry (12 cycles typical)
                + longest interrupts-disabled region
                + higher-priority ISR duration

Context switch   80-200 cycles on Cortex-M

Stack per nested 8 words (32 B) + 17 words (68 B) if FPU used
                interrupt level + the handler's own frame

```

Cortex-M quick reference

Item	Value
Exception numbers	1 Reset, 2 NMI, 3 HardFault, 4 MemManage, 5 BusFault, 6 UsageFault, 11 SVC, 14 PendSV, 15 SysTick, 16+ IRQ0..
Priority	Lower number = higher priority ; only the upper bits are implemented (4 bits = 16 levels)
Exception stack frame	R0, R1, R2, R3, R12, LR, PC, xPSR - in that order from low address upward
EXC_RETURN bit 2	0 = return using MSP, 1 = return using PSP
EXC_RETURN bit 4	0 = FPU context was stacked
Common values	0xFFFFFFFF9 thread/MSP, 0xFFFFFFFFD thread/PSP, 0xFFFFFFFF1 handler/MSP
CFSR at 0xE000ED28	byte 0 = MemManage, byte 1 = BusFault, halfword 1 = UsageFault
MemManage bits	0 IACCVIOL, 1 DACCVIOL, 3 MUNSTKERR, 4 MSTKERR, 7 MMARVALID
BusFault bits	0 IBUSERR, 1 PRECISERR, 2 IMPRECISERR, 3 UNSTKERR, 4 STKERR, 7 BFARVALID
UsageFault bits	0 UNDEFINSTR, 1 INVSTATE, 2 INVPC, 3 NOCP, 8 UNALIGNED, 9 DIVBYZERO
Barriers	<code>__DMB()</code> orders memory accesses, <code>__DSB()</code> waits for completion, <code>__ISB()</code> flushes the pipeline
Enable the cycle counter	<code>DEMCR = TRCENA; DWT->CYCCNT = 0; DWT->CTRL = CYCCNTENA</code>

Bit manipulation

```

Set / clear / toggle / test

r |= m;   r &= ~m;   r ^= m;   (r & m) != 0

Extract field  (r >> lo) & ((1u << width) - 1u)

Lowest set bit  x & -x          Clear lowest set  x & (x - 1)

Power of two?  x && !(x & (x - 1))

Align up      (x + (a - 1)) & ~(a - 1)    a = power of two

Count leading zeros  __CLZ(x)    Byte reverse  __REV(x)

Wrap-safe compare  (int32_t)(a - b) > 0

Ring buffer count  (uint32_t)(head - tail)    size = power of two

```

Toolchain commands

Command	Purpose
<code>arm-none-eabi-size fw.elf</code>	Flash = text + data; RAM = data + bss
<code>arm-none-eabi-nm --print-size --size-sort -r fw.elf</code>	Biggest symbols
<code>arm-none-eabi-objdump -d fw.elf</code>	Disassembly
<code>arm-none-eabi-addr2line -e fw.elf 0x08001234</code>	Address to source line - the crash-report workflow
<code>arm-none-eabi-readelf -S fw.elf</code>	Section addresses and sizes
<code>-Wl, -Map=fw.map</code>	Produce the map file
<code>-fstack-usage</code>	Per-function stack frames (.su files)
<code>gdb: monitor reset halt / load / x/8xw 0x20000000</code>	Reset, flash, examine memory
<code>gdb: p/x *(uint32_t*)0xE000ED28</code>	Read CFSR after a fault
<code>gdb: watch variable</code>	Data watchpoint on a corrupted variable
<code>openocd -f interface/... -f target/...</code>	Start a debug server

Appendix B. Glossary

Term	Definition
AAPCS	The ARM calling convention: which registers pass arguments, which must be preserved, how the stack is aligned.
ADC	Analogue-to-digital converter; turns a voltage into a code.
Alternate function	The peripheral a pin is muxed to, selected by a per-pin number from the datasheet's table.
Anti-rollback	A monotonic counter preventing an older, vulnerable signed firmware image from being reinstalled.
ARR	Auto-reload register: the value at which a timer wraps; the period is (ARR + 1) counts.
Atomic	An operation that cannot be observed half-done; on Cortex-M, an aligned 32-bit load or store.
Bare metal	Firmware with no operating system.
BASEPRI	A register that masks interrupts at or below a chosen priority, leaving more urgent ones enabled.
Baud rate	Symbols per second on a serial line; with 8N1 framing, one byte costs ten bits.
Bootloader	The small, rarely-changed program that validates and starts an application image, and applies updates.
BOR	Brown-out reset: the supply fell below a threshold.
BSRR	Bit set/reset register: writes set or clear GPIO pins atomically, avoiding read-modify-write races.
Bus-off	The CAN state a node enters after too many errors, in which it stops transmitting.
CAN	A multi-master automotive/industrial bus with non-destructive, identifier-based arbitration.
CFSR	Configurable Fault Status Register: says which fault occurred and why.
Clock gating	Disabling a peripheral's clock so it consumes (almost) no power - and ignores register writes.
CMSIS	ARM's standard headers and libraries for Cortex-M: core access functions, DSP and NN kernels.
COBS	Consistent Overhead Byte Stuffing: removes zero bytes from a payload so 0x00 can delimit frames, at one byte per 254.
Context switch	Saving one task's registers and restoring another's; on Cortex-M it happens in PendSV.
CRC	Cyclic redundancy check: a remainder from polynomial division used to detect corruption (not tampering).
Critical section	A region executed with interrupts (or preemption) disabled so shared data cannot be seen half-updated.
DMA	Direct memory access: a controller that moves data without the CPU.
DWT	Data Watchpoint and Trace unit; contains the cycle counter used for profiling.
ECDSA	An elliptic-curve signature algorithm; P-256 signatures are 64 bytes and verify in milliseconds.
ENOB	Effective number of bits: the resolution an ADC really delivers once noise is included.
Erratum	A documented silicon defect and its workaround; the errata sheet is a separate document from the datasheet.
EXC_RETURN	The special value in LR during an exception that encodes which stack and mode to return to.
Fixed point	An integer with an implied binary point; Q15 stores values in [-1, 1) scaled by 32768.
Flash wait states	Extra cycles the core waits for flash at high clock speeds; must be raised before the clock is.
FPU	Floating-point unit; must be enabled in CPACR before the first float instruction, and its context costs 17 stacked words.
Framing	Turning a byte stream into messages, by length prefix, delimiter, or idle time.

Term	Definition
HAL	Hardware abstraction layer: vendor code wrapping peripheral registers.
HardFault	The catch-all exception, and the escalation target when another fault cannot be taken.
Heartbeat	A periodic check-in from a task, used by a supervisor to decide whether to feed the watchdog.
HIL	Hardware in the loop: automated tests driving real hardware with simulated inputs.
I2C	A two-wire open-drain bus with addressing, ACKs, clock stretching and multi-master arbitration.
Idempotent	A command whose repetition has no additional effect - the property that makes retries safe.
IIO	Linux's Industrial I/O subsystem for sensors and converters.
Interrupt latency	Time from the hardware event to the first instruction of the handler.
ISR	Interrupt service routine.
ITM / SWO	Instrumentation trace over a single pin: low-cost logging and event timestamps.
Jitter	Variation in the timing of a periodic event.
Lazy stacking	The Cortex-M optimisation that reserves space for FPU context but saves it only if the handler uses the FPU.
Linker script	The file that assigns sections to memory regions and defines the symbols startup code uses.
LMA / VMA	Load address (where a section is stored, usually flash) and virtual address (where it runs, usually RAM).
Lock-free	A data structure safe for concurrent use without disabling interrupts or taking a lock.
MISRA C	A coding standard for safety-related C, enforced by a checker with documented deviations.
MMIO	Memory-mapped I/O: peripheral registers accessed as addresses.
MPU	Memory Protection Unit: a few region-based permission rules; no address translation, unlike an MMU.
NVIC	Nested Vectored Interrupt Controller: enables, prioritises and dispatches interrupts.
OTA	Over-the-air update: delivering firmware over the product's own communication link.
PendSV	The lowest-priority exception used by an RTOS to perform context switches after all real work is done.
Ping-pong buffer	Two buffers used alternately so one can be processed while the other fills.
Posted write	A write that completes after the instruction that issued it, requiring a read-back or a barrier before relying on it.
PRIMASK	The register that disables all configurable interrupts.
Priority inheritance	Temporarily raising a mutex owner's priority to that of the highest waiter, bounding priority inversion.
Priority inversion	A high-priority task blocked by a low-priority one holding a resource, while a medium-priority task runs.
PSC	Prescaler register: divides the timer clock by $PSC + 1$.
PSP / MSP	Process and main stack pointers; tasks run on PSP, handlers on MSP.
Reentrant	A function that can be safely called again before the first call completes.
Ring buffer	A circular FIFO; safe without locks for exactly one producer and one consumer.
RTOS	A small preemptive scheduler with tasks, priorities and blocking primitives, linked into the firmware.
Run to completion	A design rule: each step runs to the end without blocking, so latency for everything else is bounded.
Safe state	The defined output condition the system enters when it detects a fault it cannot handle.

Term	Definition
SAR ADC	Successive-approximation converter: samples onto a hold capacitor, then converts bit by bit.
Secure boot	Verifying a cryptographic signature on each stage before executing it, rooted in immutable hardware.
Seqlock	A lock-free pattern where a reader retries if a sequence counter shows the writer was active.
SPI	A four-wire full-duplex synchronous bus with four clock-polarity/phase modes.
Stack painting	Filling stacks with a pattern at boot so unused depth can be measured later.
Startup code	The reset handler: copies <code>.data</code> , zeroes <code>.bss</code> , initialises the system, calls <code>main()</code> .
SWD	Serial Wire Debug: the two-pin ARM debug interface.
Tail-chaining	Servicing a second pending interrupt without unstacking and restacking.
TCM	Tightly-coupled memory: single-cycle RAM attached directly to the core, ideal for hot ISRs.
Tickless idle	Programming a timer for the next expiry instead of waking periodically, so the device sleeps deeply.
TrustZone-M	ARMv8-M hardware separation of secure and non-secure code on one core.
Vector table	The array of exception handler addresses, beginning with the initial stack pointer and the reset handler.
volatile	Tells the compiler a value can change outside program flow; it provides neither atomicity nor ordering across the bus.
VTOR	Vector Table Offset Register: relocates the vector table, as a bootloader must before starting an application.
Watchdog	A timer that resets the device unless firmware proves liveness by feeding it.
WCET	Worst-case execution time: the longest time an activity can take, alone, on the CPU.
Wear levelling	Spreading flash erases across many sectors so no single sector reaches its endurance limit early.
WFI	Wait For Interrupt: the instruction that stops the core until an interrupt arrives.
Write-1-to-clear	A flag cleared by writing a one to it; <code>SR &= ~FLAG</code> on such a register clears every other pending flag.
XIP	Execute in place: running code directly from (usually external) flash without copying it to RAM.
Zephyr	An RTOS and platform with a device model, devicetree, and networking, storage and Bluetooth subsystems.

Appendix C. Study Roadmap, Projects and Tools

A 16-week study plan

Weeks	Focus	Deliverable
1	Chapters 1-3: the machine, bits, numbers	Blink, then toggle a pin and measure it on an analyser
2	Chapters 4-6: C dialect, toolchain, startup	Build without an IDE; read your map file; write a startup file
3	Chapters 7-8: registers, clocks, pins	Configure the PLL by hand and verify it on MCO
4-5	Chapters 9-10: interrupts and timers	A jitter-free 1 kHz timebase and a measured interrupt latency
6	Chapter 11: analogue	Timer-triggered ADC with DMA, oversampled, calibrated
7-8	Chapters 12-13: serial buses	Drive one sensor over each of I2C and SPI, from the datasheet only
9	Chapters 15-16: DMA and storage	A DMA UART console and power-fail-safe configuration storage
10-11	Chapters 17-19: architecture, concurrency, RTOS	The same application written twice: cooperative and RTOS
12	Chapters 20-21: real-time analysis and RTOS internals	A measured timing budget for your own system
13	Chapters 24-25: memory and debugging	Stack budget, MPU guard, and a decoded hard fault
14	Chapters 26-27: testing and bring-up	Host unit tests in CI for three modules
15	Chapters 28-30: reliability, power, performance	A measured energy budget and one optimisation with numbers
16	Chapters 31-32, 38: update, security, the whole project	A signed A/B update you cannot brick by pulling power

A ladder of projects

- **1. Blink without a HAL** - registers only, and know why each write is there.
- **2. A UART console** with a command parser, a ring buffer and DMA.
- **3. A sensor driver from the datasheet alone**, with host unit tests and a fake bus.
- **4. A data logger**: timer-triggered ADC, DMA, power-fail-safe flash records, and a way to extract the data.
- **5. A control loop** - motor, heater or LED - with a measured sampling jitter and a written timing budget.
- **6. A battery-powered node** that lasts a measured month, with a current trace to prove where the energy goes.
- **7. A bootloader** with A/B slots, CRC and then signature verification, that survives random power cuts during 200 updates.
- **8. A connected product**: telemetry, commands, OTA, provisioning and staged rollout - the whole of Chapter 38 on your own hardware.

The tools worth owning

Tool	Why
A debug probe with SWO	Halting debug plus low-cost trace logging - the baseline
A logic analyser with protocol decoders	Settles bus arguments in seconds; the best value per unit cost in this list
An oscilloscope	Anything analogue, edges, ringing, supply dips

Tool	Why
A power analyser or current probe	The only way to do Chapter 29's work honestly
A programmable power supply with current limiting	Protects boards during bring-up and enables brown-out testing
A soldering iron, hot air and a microscope	Rework, bodge wires, and reading part markings
A second board	So you can compare a failure against a known-good unit

How to keep learning

- **Read the reference manual, not the example code.** Examples teach one path; the manual teaches the peripheral.
- **Read the errata sheet** for every part you use, at the start rather than after two days of confusion.
- **Reimplement something you rely on** - a scheduler, a ring buffer, a bootloader, a driver. It is the only reliable comprehension test.
- **Keep a bug notebook.** Symptom, cause, and how you found it. After a year it is the most valuable document you own.
- **Measure something every week.** Cycles, microamps, microseconds. The habit compounds faster than any reading list.